

Algorithms

Prof. Dr. Christoph Berkholz

Summer term 2026



TECHNISCHE UNIVERSITÄT
ILMENAU



Preface

This document contains slides and further notes for the *Algorithms* lecture within the *Research in Computer & Systems Engineering* (RCSE) master study program at TU Ilmenau. This document **does not contain everything that is relevant for the course and the exam**. Relevant for the final exam is everything that was taught and discussed in lecture and exercise classes in the respective semester.

Copyright. The lecture notes are published under the *Creative Commons Attribution-ShareAlike 4.0 International Public License (CC BY-SA 4.0)*. The legally binding license agreement can be found at <https://creativecommons.org/licenses/by-sa/4.0/>.

Literature & Sources Part of the material is based on the slides from Martin Dietzfelbinger and some additions where contributed by Maximilian Merz, who taught this course in the summer term 2025. The lecture follows an own plan and not a specific book. The most relevant background literature are the books *Erickson: Algorithms* [Eri19] and *Cormen, Leiserson, Rivest, Stein: Introduction to Algorithms* [CLRS09]. Picture credits are provided at the end of this document.

Title image The titel image is the picture *algorists vs. abacists* from Gregor Reisch, depicted in an etching from 1503 CE. It is taken from the wikipedia page of the mathematician Muḥammad ibn Mūsā al-Khwārizmī (<https://en.wikipedia.org/wiki/Al-Khwarizmi>), who popularized the Indo-Arabic numeral system (depicted on the left in the picture) which has been proven more versatile than roman numerals (depicted on the right). His name is the origin of the English word *algorithm*.

Christoph Berkholz

Version: 14th April, 2026

The lecture notes are currently in a preliminary status. Use them with a critical mindset and check out the moodle course for updates.

Contents

1	Introduction	6
I	Divide-and-Conquer	11
2	The Divide-and-Conquer Paradigm: MergeSort & Recursion Trees	11
3	Integer & Matrix Multiplication	13
4	The Divide-and-Conquer Paradigm	18
4.1	Selection algorithm of Blum, Floyd, Pratt, Rivest, Tarjan	22
5	Polynomial Multiplication and Fast Fourier Transform	23
5.1	Multiplication of Polynomials	23
5.2	Fast Fourier Transformation	27
5.3	Inverse Fouriertransformation	28
5.4	The master theorem	30
II	Dynamic Programming	31
6	Dynamic Programming	31
6.1	Introduction	31
6.2	Text segmentation	33
6.3	Edit distance	34
6.4	Longest Increasing Subsequence	36
6.5	Matrix chain multiplication	38
6.6	Maximum-Independent-Set on Trees	39
III	Greedy Algorithms	41
7	Greedy	41
7.1	Introduction	41
7.2	Scheduling	43
7.3	Huffman-Codes	45
8	Minimum Spanning Trees	50
8.1	Jarník/Prim algorithm	53
8.2	The cut property	54
8.3	Kruskal's algorithm	57
8.3.1	Correctness of Kruskal's algorithm	58
8.4	Auxiliary data structure: Union-Find	58
8.4.1	Tree implementation of Union-Find	60
9	Dijkstra's algorithm	63
IV	Hard Problems	73
10	The TSP and NP-completeness	73
10.1	Shortest path versus shortest round tour	73

10.1.1 TSP and the Bellman-Held-Karp Algorithm	73
10.2 Theory of NP-completeness	76

Picture credits	84
------------------------	-----------

Literatur	84
------------------	-----------

List of problems

1.1 Finding summer periods	7
1.2 Print-1	10
3.1 Integer multiplication	13
3.2 Matrix multiplication	16
4.1 Selection Problem	20
5.1 Polynomial Multiplication	23
5.2 Discrete Fourier Transformation (DFT)	27
6.1 Text segmentation (decision version)	33
6.2 Text segmentation	34
6.3 Edit distance	35
6.4 Longest Increasing Subsequence	36
6.5 Matrix chain multiplication	38
7.1 Event Scheduling	43
7.2 Minimum cost prefix code	45
9.1 SSSP with nonnegative edge weights	64
10.1 Traveling Salesman Problem	75
10.2 Traveling Salesman Problem	77
10.3 Traveling Salesman Decision Problem	78
10.4 Independent Set Decision	79
10.5 Clique Decision	79
10.6 Independent Set Decision	79
10.7 Vertex Cover Decision	79
10.8 Directed Hamiltonian Cycle Decision	82

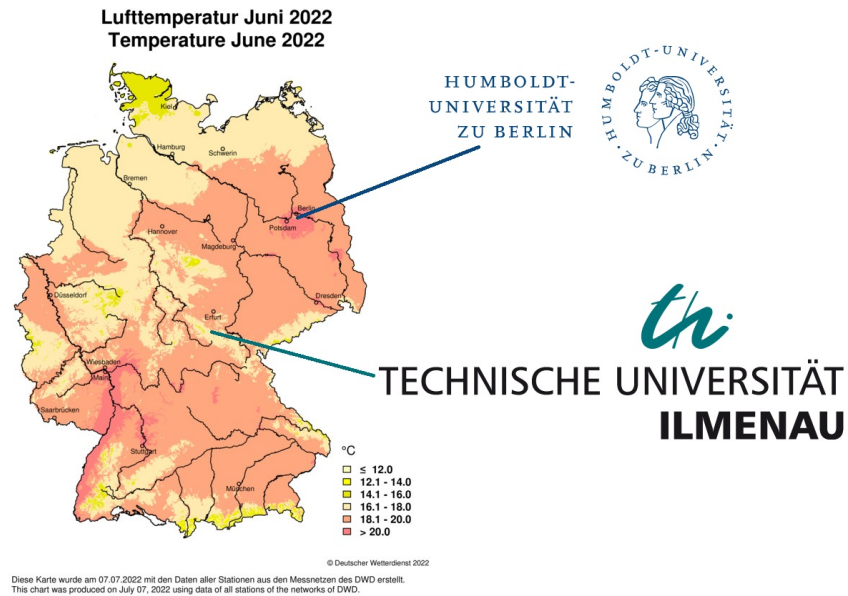
1 INTRODUCTION

1 Introduction

slide 1.1

slide 1.2

slide 1.3



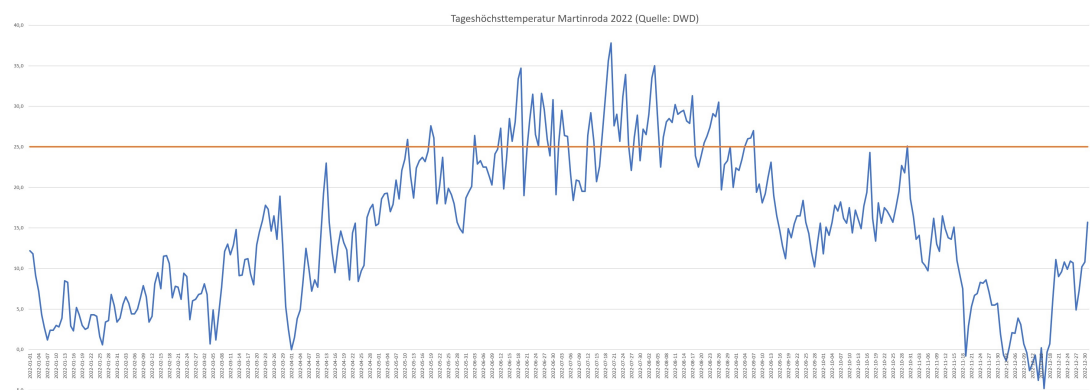
slide 1.4

How long is the summer in Ilmenau?

According to the Germany's National Meteorological Service, a *summer day* is defined to be a day with maximum temperature of at least 25°C.

slide 1.5

Maximum daily temperature 2022 in Ilmenau



<https://cdc.dwd.de/portal/>

- In total there were 66 “summer days”, but how long was “the summer”?

slide 1.6

Summer – attempt of a definition

Goal: Define a time interval that characterizes the summer.

slide 1.7

1 INTRODUCTION

- We don't want warm days in spring/autumn to be included.
- Short cold periods in the summer are ok, though.

Attempt:

- For every day t with maximum temperature MaxTemp_t , define $\text{Diff}_t := \text{MaxTemp}_t - 25^\circ\text{C}$.
 - Non-negative values of Diff_t correspond to summer days.
- The *summer value* of the interval $[t_1, t_2]$ is $\sum_{t_1 \leq t \leq t_2} \text{Diff}_t$.
- A *summer period* is an interval with highest (and non-negative) summer value.

slide 1.8

Finding the summer: Abstraction and Specification

slide 1.9

- We can easily compute the Diff_t -values in linear time. Moreover, we can also make sure that there is at least one summer day.
- For the main algorithmic task of finding the maximum interval we abstract from our application scenario and specify the general *algorithmic task*:

Problem: Finding summer periods

Input: A sequence $(a_1, \dots, a_n)$ of $n \geq 1$ rational numbers.

Output: Indices i, j satisfying $1 \leq i \leq j \leq n$, such that $\sum_{i \leq \ell \leq j} a_\ell$ is maximal.

slide 1.10

Summer algorithm 1: The naive approach

slide 1.11

```
1: procedure SUMMERNATIVE( $(a_1, \dots, a_n)$ )
2:    $\text{max\_sum} \leftarrow -\infty$ 
3:    $\text{max\_pair} \leftarrow \text{"no summer period"}$ 
4:   for  $i = 1, \dots, n$  do
5:     for  $j = i, \dots, n$  do
6:        $s \leftarrow 0$ 
7:       for  $\ell = i, \dots, j$  do
8:          $s \leftarrow s + a_\ell$ 
9:       if  $s > \text{max\_sum}$  then
10:         $\text{max\_sum} \leftarrow s$ 
11:         $\text{max\_pair} \leftarrow (i, j)$ 
12:   return  $\text{max\_pair}$ 
```

- **Running time:** $O(n^3)$
- **Correctness:** straightforward $\checkmark$ (formally: formulate loop invariant and prove inductively)

Recap: asymptotic notation

slide 1.12

We fix the following standard notation for arbitrary functions $g, f: \mathbb{N} \rightarrow \mathbb{R}^+$.

1 INTRODUCTION

- $f(n) = O(g(n))$ abbreviates There are $n_0 \in \mathbb{N}$ and $c > 0$ such that for all $n \geq n_0$:
 $f(n) \leq c \cdot g(n)$.
 $f(n)$ grows asymptotically at most as fast as $g(n)$.
- $f(n) = \Omega(g(n))$ abbreviates There are $n_0 \in \mathbb{N}$ and $c > 0$ such that for all $n \geq n_0$:
 $f(n) \geq c \cdot g(n)$.
 $f(n)$ grows asymptotically at least as fast as $g(n)$.
- $f(n) = \Theta(g(n))$ abbreviates $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
 $f(n)$ grows asymptotically in the same way as $g(n)$.

Note that $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$.

slide 1.13

slide 1.14

Summer algorithm 2: The almost-naive approach

```
1: procedure SUMMERNAIVE( $(a_1, \dots, a_n)$ )
2:    $max\_sum \leftarrow -\infty$ 
3:    $max\_pair \leftarrow$  "no summer period"
4:   for  $i = 1, \dots, n$  do
5:     for  $j = i, \dots, n$  do
6:        $s \leftarrow 0$ 
7:       for  $\ell = i, \dots, j$  do
8:          $s \leftarrow s + a_\ell$ 
9:       if  $s > max\_sum$  then
10:         $max\_sum \leftarrow s$ 
11:         $max\_pair \leftarrow (i, j)$ 
12:   return  $max\_pair$ 
```

- Calculates for all candidates (i, j) the sum $s_{(i,j)} = \sum_{\ell=i}^j a_\ell = a_i + \dots + a_j$
- **Runtime:** $O(n^3)$

```
1: procedure SUMMERALMOST-NAIVE( $(a_1, \dots, a_n)$ )
2:    $max\_sum \leftarrow -\infty$ 
3:    $max\_pair \leftarrow$  "no summer period"
4:   for  $i = 1, \dots, n$  do
5:      $s \leftarrow 0$ 
6:     for  $j = i, \dots, n$  do
7:        $s \leftarrow s + a_j$ 
8:       if  $s > max\_sum$  then
9:         $max\_sum \leftarrow s$ 
10:         $max\_pair \leftarrow (i, j)$ 
11:   return  $max\_pair$ 
```

- **Also** calculates for all candidates (i, j) the sum $s_{(i,j)} = \sum_{\ell=i}^j a_\ell = a_i + \dots + a_j$
- **Runtime:** $O(n^2)$
- **Correctness:** almost straightforward ✓
(formulate loop invariant and prove inductively)

slide 1.15

slide 1.16

Summer algorithm 3: Divide-and-Conquer

1 INTRODUCTION

struct Candidate with fields i, j, sum
(start- and end-index, sum of entries $a_i, \dots, a_j$)

```
1: procedure SUMMERRECVRAP(( $a_1, \dots, a_n$ ))
2:    $opt \leftarrow SUMMERREC((a_1, \dots, a_n))$ 
3:   return ( $opt.i, opt.j$ )
```

```
1: procedure SUMMERREC(( $a_1, \dots, a_n$ ))
2:   if  $n = 1$  then return Candidate(1, 1,  $a_1$ )
3:    $cand\_L \leftarrow SUMMERREC((a_1, \dots, a_{\lfloor n/2 \rfloor}))$ 
4:    $cand\_R \leftarrow SUMMERREC((a_{\lfloor n/2 \rfloor + 1}, \dots, a_n))$ 
5:    $cand\_M \leftarrow SUMMERMID((a_1, \dots, a_n))$ 
6:    $opt \leftarrow$  choose the best of three candidates
        $cand\_L, cand\_R, cand\_M$ 
       (so that  $opt.sum$  is maximal)
7:   return  $opt \triangleright$  a Candidate struct
```

```
1: procedure SUMMERMID(( $a_1, \dots, a_n$ ))
2:    $s \leftarrow 0$ ;  $maxL \leftarrow -\infty$ ;  $maxL\_i \leftarrow \lfloor n/2 \rfloor + 1$ 
3:   for  $i = \lfloor n/2 \rfloor, \dots, 1$  do
4:      $s \leftarrow s + a_i$ 
5:     if  $s > maxL$  then
6:        $maxL \leftarrow s$ ;  $maxL\_i \leftarrow i$ 
7:    $s \leftarrow 0$ ;  $maxR \leftarrow -\infty$ ;  $maxR\_j \leftarrow \lfloor n/2 \rfloor$ 
8:   for  $j = \lfloor n/2 \rfloor + 1, \dots, n$  do
9:      $s \leftarrow s + a_j$ 
10:    if  $s > maxR$  then
11:       $maxR \leftarrow s$ ;  $maxR\_j \leftarrow j$ 
12:   return Candidate( i:  $maxL\_i$ ,
                     j:  $maxR\_j$ ,
                     sum:  $maxL + maxR$ )
```

- **Correctness:** ☞ blackboard.
- **Runtime:** $O(n \cdot \log n)$, ☞ blackboard.

slide 1.17

Summer algorithm 4: Linear Scan (Kadane's Algorithm)

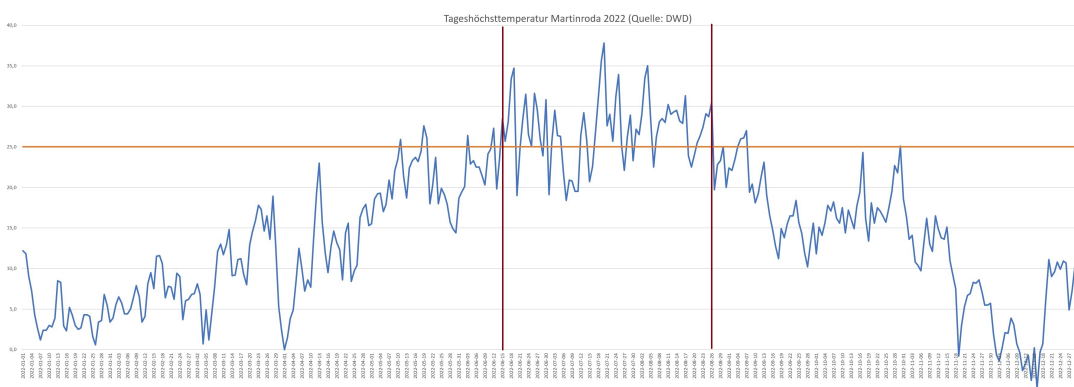
slide 1.18

```
1: procedure SUMMERLIN(( $a_1, \dots, a_n$ ))
2:    $global \leftarrow$  Candidate( $\perp, \perp$ , sum:  $-\infty$ )
3:    $local \leftarrow$  Candidate(1,  $\perp$ , sum: 0)
4:   for  $\ell = 1, \dots, n$  do
5:      $local.j \leftarrow \ell$ 
6:      $local.sum \leftarrow local.sum + a_\ell$ 
7:     if  $local.sum < a_\ell$  then
8:        $local.i \leftarrow \ell$ 
9:        $local.sum \leftarrow a_\ell$ 
10:    if  $local.sum > global.sum$  then
11:       $global \leftarrow local$ 
12:   return ( $global.i, global.j$ )
```

- Like before:
struct Candidate with fields i, j, sum
(start- and end-index, sum of entries $a_i, \dots, a_j$)
– Symbol $\perp$ as “None” / “no value”
- Iterate over the end-index ℓ of the interval. After line 6 the following hold:
 1. $local.j = \ell$.
 2. $local.sum = a_{local.i} + \dots + a_{local.j}$.
- Compare $local.sum$ with just a_ℓ .
If $local.sum < a_\ell$, then we get better candidates by letting the interval $local$ begin at ℓ !
- **Runtime:** $O(n)$
- **Correctness:** ☞ blackboard.

Alright, how long was the summer in Ilmenau?

slide 1.19



1 INTRODUCTION

- The summer period 2022 lasted 73 days, from June 15 to August 26.
- The maximum summer value in this interval was 154,8.

slide 1.20

slide 1.21

General approach of solving problems algorithmically

1. Model the problem (“What is a summer?”)
2. Abstraction (temperature database and time span → sequence of rational numbers and subinterval)
3. Specification (exact description of algorithmic task and input/output behavior).
4. Algorithm design
 - Generally the most difficult (= interesting!) task.
 - First make it work, then make it fast.
 - Analyze resource consumption (usually runtime and memory).
 - Prove the correctness.
5. Implementing the algorithm (not in this lecture)
 - testing the implementation
 - verify runtime behavior on “real world” instances
 - maybe adapt algorithm design based on experimental findings

slide 1.22

Partially covered at the end of this lecture: *investigating the complexity of the algorithmic task.*

- Can the algorithmic task in principle be solved with efficient algorithms?
- Is my algorithm optimal?
- Can the algorithmic task be solved with limited resources?
 - with simple automata
 - with little memory
 - in distributed settings
 - with artificial neural networks
 - in database systems with certain query languages
 - ...

slide 1.23

Runtime analysis & Correctness: it can be challenging ...

Problem: Print-1

Input: A positive integer n .

Output: Print “1”.

```
1: procedure PRINTONESIMPLE( $n$ )
2:   return 1
```

- Correctness: trivial ✓
- Runtime: $O(1)$

```
1: procedure PRINTONERECURSIVE( $n$ )
2:   if  $n = 1$  then
3:     return  $n$ 
4:   else if  $n$  is even then
5:     PRINTONERECURSIVE( $n/2$ )
6:   else
7:     PRINTONERECURSIVE( $3n + 1$ )
```

- For all $n \leq 2 \cdot 95 \cdot 10^{20}$ correct.
- Correctness (for all n) & runtime: open mathematical problem since 1937 (Collatz conjecture).

slide 1.24

Part I

Divide-and-Conquer

2 The Divide-and-Conquer Paradigm: MergeSort & Recursion Trees

slide 2.1

Divide and Conquer

slide 2.2

Algorithm design using the **divide-conquer-merge** principle. **divide**

- Decompose instance into smaller subinstances.
- Repeat decomposition recursively until instance size is small enough.

conquer

- Solve tiny (constant size) instances directly.

merge

- Combine solutions of smaller instances to a solution for the larger instance.

slide 2.3

```

procedure MERGESORT( $A[1 \dots n]$ )
  if  $n = 1$  then
    return  $A$ 
  else
     $m \leftarrow \lfloor \frac{n}{2} \rfloor$ 
     $B_{\text{left}} \leftarrow \text{MERGESORT}(A[1 \dots m])$ 
     $B_{\text{right}} \leftarrow \text{MERGESORT}(A[m + 1 \dots n])$ 
    return  $\text{MERGE}(B_{\text{left}}, B_{\text{right}})$ 

```

↓ **divide!** ↓

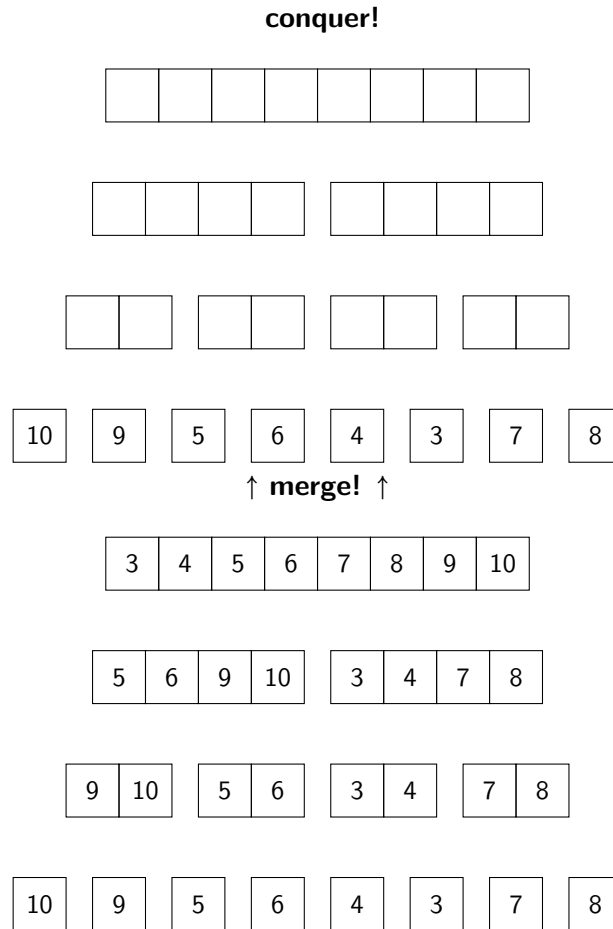
10	9	5	6	4	3	7	8
----	---	---	---	---	---	---	---

10	9	5	6	4	3	7	8
----	---	---	---	---	---	---	---

10	9	5	6	4	3	7	8
----	---	---	---	---	---	---	---

10	9	5	6	4	3	7	8
----	---	---	---	---	---	---	---

2 THE DIVIDE-AND-CONQUER PARADIGM: MERGESORT & RECURSION TREES



slide 2.4

The merge subroutine

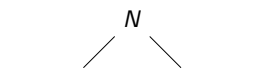
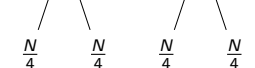
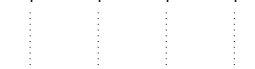
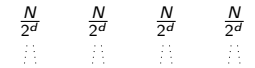
```
procedure MERGE( $C[1 \dots c]$ ,  $D[1 \dots d]$ )  
  Init array  $B[1 \dots c + d]$   
   $i \leftarrow 1$ ;  $j \leftarrow 1$   
  for  $k = 1 \dots c + d$  do  
    if  $j > d$  then  
       $B[k] \leftarrow C[i]$ ;  $i \leftarrow i + 1$   
    else if  $i > c$  then  
       $B[k] \leftarrow D[j]$ ;  $j \leftarrow j + 1$   
    else if  $C[i] < D[j]$  then  
       $B[k] \leftarrow C[i]$ ;  $i \leftarrow i + 1$   
    else  
       $B[k] \leftarrow D[j]$ ;  $j \leftarrow j + 1$   
  return  $B$ 
```

Runtime $O(c + d) = O(n)$

slide 2.5

Merge sort runtime analysis with recursion tree

3 INTEGER & MATRIX MULTIPLICATION

level		# calls/level	runtime/call	total runtime of level
0		1	$c \cdot N$	$c \cdot N$
1		2	$c \cdot \frac{N}{2}$	$c \cdot N$
2		4	$c \cdot \frac{N}{4}$	$c \cdot N$
d		2^d	$c \cdot \frac{N}{2^d}$	$c \cdot N$
$\lceil \log(N) \rceil$		N	c	$c \cdot N$
total runtime:				$O(N \log N)$

Design of divide-and-conquer algorithms

slide 2.6

Design of the **divide**-phase

- very common: halving strategy (e.g. merge sort)
- clever decompositions may avoid redundancies (e.g. fast matrix multiplication)

Design of the conquer-phase

- usually simplest part
- tiny instances can be solved directly in $O(1)$
- in practice: size of “tiny instances” depends on hardware (can be determined experimentally)

Design of the **merge**-phase

- clever combination of results for subinstances
- exploit (in)dependencies between subinstances

3 Integer & Matrix Multiplication

slide 3.1

Integer multiplication

slide 3.2

Problem: Integer multiplication

Input: Two integers x and y as bit strings.

Output: The product $x \cdot y$ (as bit string).

Remark 3.1. Any other b -ary representation for $b \geq 2$ would also be possible. [E, sec. 1.9] uses decimal numbers ($b = 10$) to describe the problem and the algorithm.

Conventions:

- In algorithm analysis we consider storing, adding, multiplying integers as $O(1)$ operations, because they “usually” occupy only one (or few) 32-/64-bit registers. In particular, this is justified when the binary representation is logarithmic in the overall “input length”.

3 INTEGER & MATRIX MULTIPLICATION

- In *this section*, however, we only consider bit-level operations as $O(1)$.

slide 3.3

The basic school method

factors x, y :	1	0	0	1	1	0	.	1	1	0	0	1	1
		1	0	0	1	1	0	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>
			1	0	0	1	1	0	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>
				0	0	0	0	0	0	<i>o</i>	<i>o</i>	<i>o</i>	<i>o</i>
					0	0	0	0	0	0	<i>o</i>	<i>o</i>	<i>o</i>
						1	0	0	1	1	0	<i>o</i>	<i>o</i>
							1	0	0	1	1	0	<i>o</i>
								1	0	0	1	1	0
product:		1	1	1	1	0	0	1	0	0	1	0	0

Multiplication of two n -bit number $\hat{=}$ addition of n binary numbers of length $\leq 2n$. $\Rightarrow$ Total run-

time: $O(n^2)$. Addition of two $2n$ -bit numbers in $O(n)$:

$$\begin{array}{r}
 1\ 0\ 0\ 1\ 1\ 0\ 0 \\
 +\quad 1\ 0\ 0\ 1\ 1\ 0\ 0 \\
 \hline
 1\ 1\ 1\ 0\ 0\ 1\ 0
 \end{array}$$

slide 3.4

Divide-and-conquer multiplication

Let x, y be two n -bit numbers and $k = \lceil \frac{n}{2} \rceil$. Decompose bitstring $x = a \circ b$ into an $(n-k)$ -bit piece a and a k -bit piece b . Similarly, $y = c \circ d$.

$$\begin{aligned}
 x &= 2^k a + b \\
 y &= 2^k c + d \\
 x \cdot y &= (2^k a + b) \cdot (2^k c + d) \\
 &= 2^{2k} ac + 2^k (ad + bc) + bd
 \end{aligned}$$

```

1: procedure  $MULT(x, y, n)$ 
2:   if  $n = 1$  then return  $x \cdot y$ 
3:   else
4:      $k \leftarrow \lceil \frac{n}{2} \rceil$ 
5:      $a \leftarrow x[1 \dots \lfloor \frac{n}{2} \rfloor]$ ,  $b \leftarrow x[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$ 
6:      $c \leftarrow y[1 \dots \lfloor \frac{n}{2} \rfloor]$ ,  $d \leftarrow y[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$ 
7:      $v_{ac} \leftarrow MULT(a, c, k)$ ,  $v_{ad} \leftarrow MULT(a, d, k)$ 
8:      $v_{bc} \leftarrow MULT(b, c, k)$ ,  $v_{bd} \leftarrow MULT(b, d, k)$ 
9:     return  $2^{2k} v_{ac} + 2^k (v_{ad} + v_{bc}) + v_{bd}$ 

```

$\triangleright x, y$ are n -bit numbers

Recurrence: $T[n] \leq 4 \cdot T[\lceil \frac{n}{2} \rceil] + c \cdot n$

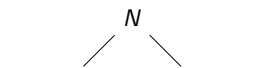
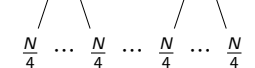
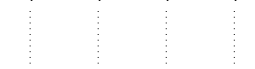
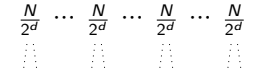
$$\Rightarrow T[n] = O(n^2)$$

partial sum formula

$$\sum_{k=0}^n q^k = \frac{q^{n+1} - 1}{q - 1} \text{ for } n \in \mathbb{N}, q \in \mathbb{R}^+ \setminus \{1\}$$

Solving the recurrence with the recursion tree method:

3 INTEGER & MATRIX MULTIPLICATION

Level		# Calls	Merge $c \cdot n$	Total runtime
0		1	$c \cdot N$	$1 \cdot c \cdot N$
1		4	$c \cdot \frac{N}{2}$	$4 \cdot c \cdot \frac{N}{2}$
2		16	$c \cdot \frac{N}{4}$	$16 \cdot c \cdot \frac{N}{4}$
d		4^d	$c \cdot \frac{N}{2^d}$	$4^d \cdot c \cdot \frac{N}{2^d}$
$\lceil \log(N) \rceil$		N	c	$c \cdot N$

$$\sum_{d=0}^{\lceil \log(N) \rceil} 4^d \cdot c \cdot \frac{N}{2^d}$$

Applying the partial sum formula:

$$\sum_{d=0}^{\lceil \log(N) \rceil} 4^d \cdot c \cdot \frac{N}{2^d} = c \cdot N \cdot \sum_{d=0}^{\lceil \log(N) \rceil} 2^d = c \cdot N \cdot \frac{2^{\lceil \log(N) \rceil + 1} - 1}{2 - 1} = O(N^2)$$

Multiplying faster than $O(n^2)$?

slide 3.5

The following decomposition, found by Anatoli Alexejewitsch Karatsuba, uses only 3 instead of 4 multiplications:

$$\begin{aligned}
 x &= 2^k a + b \\
 y &= 2^k c + d \\
 x \cdot y &= (2^k a + b) \cdot (2^k c + d) \\
 &= 2^{2k} ac + 2^k (ad + bc) + bd \\
 &= 2^{2k} ac + bd \\
 &\quad + 2^k (ac + bd - (a - b)(c - d))
 \end{aligned}$$

```

1: procedure  $KMULT(x, y, n)$   $\triangleright x, y$  are  $n$ -bit numbers
2:   if  $n = 1$  then return  $x \cdot y$ 
3:   else
4:      $k \leftarrow \lceil \frac{n}{2} \rceil$ 
5:      $a \leftarrow x[1 \dots \lfloor \frac{n}{2} \rfloor], b \leftarrow x[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$ 
6:      $c \leftarrow y[1 \dots \lfloor \frac{n}{2} \rfloor], d \leftarrow y[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$ 
7:      $\triangleright (\text{sign}(a) = \text{sign}(b) = \text{sign}(x); \text{sign}(c) = \text{sign}(d) = \text{sign}(y))$ 
8:      $v_{ac} \leftarrow KMULT(a, c, k), v_{bd} \leftarrow KMULT(b, d, k)$ 
9:      $w \leftarrow KMULT(a - b, c - d, k)$ 
10:    return  $2^{2k} v_{ac} + 2^k (v_{ac} + v_{bd} - w) + v_{bd}$ 

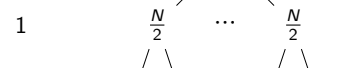
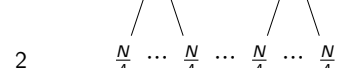
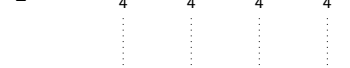
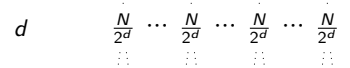
```

Recurrence: $T[n] \leq 3 \cdot T[\lceil \frac{n}{2} \rceil] + c \cdot n$

$$\Rightarrow T[n] = O(n^{\log_2(3)}) = O(n^{1.585})$$

3 INTEGER & MATRIX MULTIPLICATION

Applying the recursion tree method:

Level		# Calls	Merge $c \cdot n$	Total runtime
0		1	$c \cdot N$	$1 \cdot c \cdot N$
1		3	$c \cdot \frac{N}{2}$	$3 \cdot c \cdot \frac{N}{2}$
2		9	$c \cdot \frac{N}{4}$	$9 \cdot c \cdot \frac{N}{4}$
d		3^d	$c \cdot \frac{N}{2^d}$	$3^d \cdot c \cdot \frac{N}{2^d}$
$\lceil \log(N) \rceil$		N	c	$c \cdot N$

$$\sum_{d=0}^{\lceil \log(N) \rceil} 3^d \cdot c \cdot \frac{N}{2^d}$$

Solving the sum with the partial sum formula:

$$\sum_{d=0}^{\lceil \log(N) \rceil} 3^d \cdot c \cdot \frac{N}{2^d} = c \cdot N \cdot \sum_{d=0}^{\lceil \log(N) \rceil} \frac{3^d}{2^d} = c \cdot N \cdot \frac{\frac{3}{2}^{\lceil \log(N) \rceil + 1} - 1}{\frac{3}{2} - 1} = O(N \cdot \frac{3}{2}^{\log(N)}) = O(N^{1 + \log(\frac{3}{2})}) = O(N^{\log(3)})$$

slide 3.6

Even faster multiplication

School method:	$O(n^2)$
Karatsuba (1960/62):	$O(n^{1.585})$
Toom-Cook (1963/69):	$O(f(\epsilon) \cdot n^{1+\epsilon})$ for all $\epsilon > 0$
Schönhage-Strassen (1971):	$O(n \log n \cdot \log \log n)$
Fürer (2007):	$O(n \log n \cdot \log^* n)$
Harvey-VanDerHoeven (2019/21):	$O(n \log n)$

Conjecture

Multiplication of two n -bit numbers requires $\Theta(n \log n)$.

slide 3.7

Matrix multiplication

Recap from linear algebra

The product $C = A \cdot B$ of two $n \times n$ matrices $A = (a_{ij})_{i \in [n], j \in [n]}$ and $B = (b_{ij})_{i \in [n], j \in [n]}$ over a ring (e.g. $\mathbb{Z}, \mathbb{Q}, \mathbb{R}$) is the matrix $C = (c_{ij})_{i \in [n], j \in [n]}$ with $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$.

Problem: Matrix multiplication

Input: Two $n \times n$ matrices A and B with entries from a ring.

Output: The matrix product $C = A \cdot B$.

- Convention: Here we consider addition and multiplication of two ring elements as $O(1)$ operations.
- The trivial algorithm has runtime $O(n^3)$.
- We assume: $n = 2^x$ for some $x \in \mathbb{N}$ (otherwise fill up the matrix with 0 entries).

Divide-and-conquer approach: decompose into $\frac{n}{2} \times \frac{n}{2}$ block matrices:

$$A = \left(\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right) \cdot \left(\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right) = \left(\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right) = \left(\begin{array}{c|c} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ \hline A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{array} \right)$$

3 INTEGER & MATRIX MULTIPLICATION

- Solve recursively 8 multiplications of $\frac{n}{2} \times \frac{n}{2}$ -matrices and combine them in $O(n^2)$.
- $T[n] \leq 8 \cdot T[\frac{n}{2}] + c \cdot n^2 \Rightarrow T[n] = O(n^3)$.

Fast matrix multiplication

slide 3.8

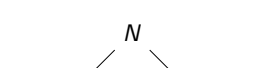
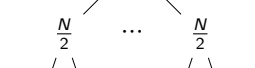
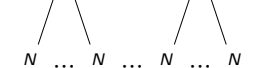
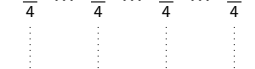
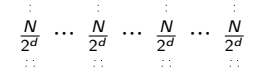
Improvement of divide-strategy using only 7 multiplications (Strassen 1969)

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22})B_{11} \\ M_3 &= A_{11}(B_{12} - B_{22}) \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12})B_{22} \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

$$\begin{aligned} C &= \left(\begin{array}{c|c} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ \hline A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{array} \right) \\ &= \left(\begin{array}{c|c} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ \hline M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{array} \right) \end{aligned}$$

- Solve recursively 7 multiplications of $\frac{n}{2} \times \frac{n}{2}$ -matrices combine them in $O(n^2)$.
- $T[n] \leq 7 \cdot T[\frac{n}{2}] + c \cdot n^2 \Rightarrow T[n] = O(n^{\log_2 7}) = O(n^{2.8074})$.

Recursion tree:

Level		# Calls	Merge $c \cdot n^2$	Total runtime
0		1	$c \cdot N^2$	$1 \cdot c \cdot N^2$
1		7	$c \cdot (\frac{N}{2})^2$	$7 \cdot c \cdot (\frac{N}{2})^2$
2		49	$c \cdot (\frac{N}{4})^2$	$49 \cdot c \cdot (\frac{N}{4})^2$
d		7^d	$c \cdot (\frac{N}{2^d})^2$	$7^d \cdot c \cdot (\frac{N}{2^d})^2$
$\lceil \log(N) \rceil$		N	c	$c \cdot N$

$$\sum_{d=0}^{\lceil \log(N) \rceil} 7^d \cdot c \cdot (\frac{N}{2^d})^2$$

Using the partial sum formula:

$$\sum_{d=0}^{\lceil \log(N) \rceil} 7^d \cdot c \cdot (\frac{N}{2^d})^2 = c \cdot N^2 \cdot \sum_{d=0}^{\lceil \log(N) \rceil} \frac{7^d}{4^d} = c \cdot N^2 \cdot \frac{\frac{7}{4}^{\lceil \log(N) \rceil + 1} - 1}{\frac{7}{4} - 1} = O(N^2 \cdot \frac{7}{4}^{\log(N)}) = O(N^{2 + \log(\frac{7}{4})}) = O(N^{\log(7)})$$

Even faster matrix multiplication

slide 3.9

The matrix multiplication exponent ω

The matrix multiplication exponent ω is the smallest real number ω , such that for all $\epsilon > 0$ two $n \times n$ -matrices can be multiplied with $O(n^{\omega + \epsilon})$ ring operations.

4 THE DIVIDE-AND-CONQUER PARADIGM

Strassen (1969)	$\omega < 2.8074$	Conjecture $\omega = 2$
Pan (1978)	$\omega < 2.796$	
Bini, Capovani, Romani (1979)	$\omega < 2.780$	
Schönhage (1981)	$\omega < 2.522$	
Romani (1981)	$\omega < 2.517$	
Coppersmith, Winograd (1981)	$\omega < 2.496$	
Strassen (1986)	$\omega < 2.479$	
Coppersmith, Winograd (1990)	$\omega < 2.3755$	
Stothers (2010)	$\omega < 2.3737$	
Williams (2013)	$\omega < 2.3729$	
Le Gall (2014)	$\omega < 2.3728639$	
Alman, Williams (2020)	$\omega < 2.3728596$	
Duan, Wu, Zhou (2022)	$\omega < 2.37188$	
Williams, Xu, Xu, Zhou (2024)	$\omega < 2.371552$	
Alman, Duan, Williams, Xu, Xu, Zhou (2024)	$\omega < 2.371339$	

(Data from https://en.wikipedia.org/wiki/Computational_complexity_of_matrix_multiplication)

slide 3.10

An application: counting triangles

A *triangle* in an (undirected) graph $G = (V, E)$ is a set of three nodes u, v, w , that are pairwise connected.

Easy: counting triangles of n -vertex graphs in $O(n^3)$.

- Let $A = (a_{uv})$ the $n \times n$ -adjacency matrix of G and $A^2 = (c_{uv})$ the squared adjacency matrix.
- Then, $c_{uv} = \sum_{w \in V(G)} a_{uw} \cdot a_{wv}$ is the *number of common neighbours of u and v* .

```

1: procedure COUNTTRIANGLES( $G$ )                                ▷  $G = (V, E)$  is undirected graph
2:   Let  $A = (a_{uv})$  adjacency matrix of  $G$ . Compute  $A^2 = (c_{uv})$  using fast matrix mult.
3:    $cnt \leftarrow 0$ 
4:   for  $(u, v) \in V^2$  do
5:     if  $a_{uv} = 1$  then  $cnt \leftarrow cnt + c_{uv}$ 
6:   return  $\frac{cnt}{6}$ 

```

Runtime: $O(n^{2.371339} + n^2) = O(n^{2.371339})$

slide 4.1

4 The Divide-and-Conquer Paradigm

Lecture 3: QuickSort & the Selection Problem

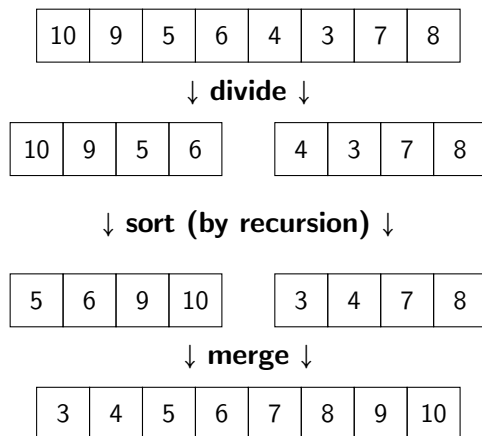
slide 4.2

slide 4.3

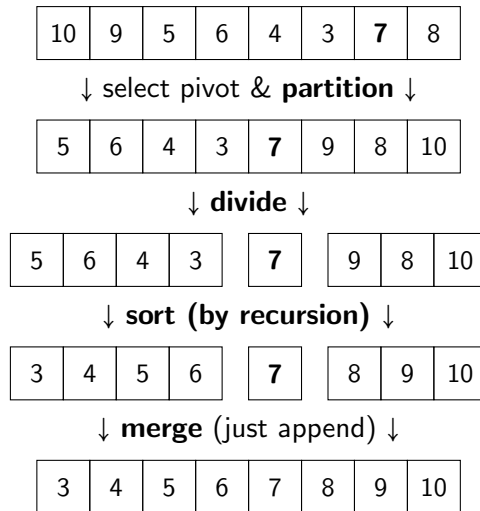
Quicksort: Intuition

4 THE DIVIDE-AND-CONQUER PARADIGM

MergeSort



QuickSort



Quicksort [C. A. R. Hoare 1961]

slide 4.4

```

1: procedure QUICKSORTWRAP( $A[1 \dots n]$ )
2:   Call QUICKSORT(1,  $n$ ) on global array  $A$ 
3:   return  $A$ 

4: procedure QUICKSORT( $a, b$ )
5:   if  $a < b$  then
6:     Select a pivot  $A[p]$  with  $a \leq p \leq b$ .
7:      $r \leftarrow \text{PARTITION}(a, b, p)$ 
8:     QUICKSORT( $a, r - 1$ )
9:     QUICKSORT( $r + 1, b$ )

```

```

1: procedure PARTITION( $a, b, p$ )
2:   swap  $A[p] \leftrightarrow A[b]$ 
3:    $\ell \leftarrow a$ 
4:   for  $i \leftarrow a \dots b - 1$  do
5:     if  $A[i] < A[b]$  then
6:       swap  $A[\ell] \leftrightarrow A[i]$ 
7:        $\ell \leftarrow \ell + 1$ 
8:   swap  $A[b] \leftrightarrow A[\ell]$ 
9:   return  $\ell$ 

```

$\text{PARTITION}(a, b, p)$ receives indices a, b, p with $1 \leq a \leq p \leq b \leq n$. We call $P := A[p]$ the *pivot*, while p is the *pivot's index*. The call to PARTITION rearranges the subarray $A[a \dots b]$ and returns an index ℓ such that:

1. $A[\ell] = P$ (the pivot is now at index ℓ)
2. $A[i] < P$ for all $a \leq i < \ell$ (elements smaller than the pivot P are now sorted left of it)
3. $A[i] \geq P$ for all $\ell < i \leq b$ (elements greater than the pivot P are now sorted right of it)

Runtime analysis of Quicksort

slide 4.5

Runtime crucially depends on the choice of the pivot!

- Worst case
 - pivot is always smallest/largest element
 - $T[n] \leq T[0] + T[n-1] + n$
 - $\Rightarrow T[n] = O(n^2)$
- Best case
 - pivot is always *median*
 - $T[n] \leq T[\frac{n}{2}] + T[\frac{n}{2}] + n$
 - $\Rightarrow T[n] = O(n \log n)$

Good strategy: Choose pivot uniformly at random.

Theorem 4.1. If the input elements are pairwise distinct, then randomized quicksort has *expected* runtime $O(n \log n)$.

4 THE DIVIDE-AND-CONQUER PARADIGM

slide 4.6

Proof: Let $\mathbb{E}[n]$ denote the expected runtime. ($\mathbb{P}[X]$ denotes the probability of event X .)

- $\mathbb{E}[0] = \mathbb{E}[1] = 1 = O(n \log n)$ ✓
- For $n \geq 2$:

$$\mathbb{E}[n] = \sum_{i=1}^n (\mathbb{P}[\text{pivot index is } i] \cdot \overbrace{(\text{runtime of recursive calls} + \text{runtime of } PARTITION))}^{\substack{\text{we don't know size of the calls!} \\ =n}})$$

Idea: re-phrase in terms of the pivot index **after** the partition!

$$= \sum_{i=1}^n (\underbrace{\mathbb{P}[\text{pivot index after partition is } i]}_{=1/n} \cdot (\mathbb{E}[i-1] + \mathbb{E}[n-i] + n)) \quad // \text{ entries are disjoint}$$

$$= \frac{1}{n} \sum_{i=1}^n \mathbb{E}[i-1] + \frac{1}{n} \sum_{i=1}^n \mathbb{E}[n-i] + \frac{1}{n} \sum_{i=1}^n n$$

$$= 2 \frac{1}{n} \sum_{j=0}^{n-1} \mathbb{E}[j] + n$$

– Now, I claim that (for all $n \geq 0$):

$$\textbf{(claim)}_n \quad \mathbb{E}[n] \leq 2 \cdot (n+1) \cdot H_{n+1}$$

Here, $H_n = \sum_{i=1}^n \frac{1}{i}$ is the n 'th harmonic number.

- We know that $H_n = O(\log n)$.
- We can use this $\textbf{(claim)}_n$ to prove the expected runtime:

$$\mathbb{E}[n] = 2 \frac{1}{n} \sum_{j=0}^{n-1} \mathbb{E}[j] + n \leq 2 \cdot (n+1) \cdot H_{n+1} = 2 \cdot (n+1) \cdot O(\log n) = O(n \log n)$$

- It remains to prove the $\textbf{(claim)}_n$!

slide 4.7

$$\textbf{(claim)}_n \quad \mathbb{E}[n] \leq 2 \cdot (n+1) \cdot H_{n+1}$$

We prove $\textbf{(claim)}_n$ by induction.

Induction start.

- $\mathbb{E}[0] = 1 \leq (0+1) \cdot 2 \cdot H_{0+1} = 2$
- $\mathbb{E}[1] = 1 \leq (1+1) \cdot 2 \cdot H_{1+1} = 6$

Induction step $(n) \rightarrow (n+1)$

for $n \geq 1$:

- **Induction assumption (IA).**

We assume $\textbf{(claim)}_n$ holds:

$$\mathbb{E}[n] \leq 2 \cdot (n+1) \cdot H_{n+1}.$$

- We need to prove $\textbf{(claim)}_{(n+1)}$:

$$\mathbb{E}[n+1] \leq 2 \cdot (n+2) \cdot H_{n+2}.$$

- Recurrence for $\mathbb{E}[n]$:

$$\mathbb{E}[n] = 2 \frac{1}{n} \sum_{j=0}^{n-1} \mathbb{E}[j] + n$$

$$\begin{aligned} \mathbb{E}[n+1] &= 2 \frac{1}{n+1} \sum_{j=0}^n \mathbb{E}[j] + (n+1) \\ &= 2 \frac{1}{n+1} \sum_{j=0}^{n-1} \mathbb{E}[j] + 2 \frac{1}{n+1} \mathbb{E}[n] + (n+1) \\ &= n \frac{1}{n+1} \left(2 \frac{1}{n} \sum_{j=0}^{n-1} \mathbb{E}[j] \right) + 2 \frac{1}{n+1} \mathbb{E}[n] + (n+1) \\ &= n \frac{1}{n+1} (\mathbb{E}[n] - n) + 2 \frac{1}{n+1} \mathbb{E}[n] + (n+1) \\ &= n \frac{1}{n+1} \mathbb{E}[n] - \frac{1}{n+1} n^2 + 2 \frac{1}{n+1} \mathbb{E}[n] + (n+1) \\ &= (n+2) \frac{1}{n+1} \mathbb{E}[n] + \frac{1}{n+1} ((n+1)^2 - n^2) \\ &\leq (n+2) \frac{1}{n+1} 2(n+1) H_{n+1} + \frac{1}{n+1} (2(n+1) - 1) \\ &= 2(n+2) H_{n+1} + 2 - \frac{1}{n+1} \\ &= 2(n+2) \left(H_{n+1} + \frac{1}{n+2} \right) - \frac{1}{n+1} \\ &= 2(n+2) H_{n+2} - \frac{1}{n+1} \\ &\leq 2(n+2) H_{n+2} \quad \square \end{aligned}$$

slide 4.8

Selection

Finding medians and k -th smallest elements in linear time

Problem: Selection Problem

Input: A list of integers $a_1, \dots, a_n$ and a number k with $1 \leq k \leq n$.

Output: The k -th smallest element in this list, i. e., the value x with $|\{i: a_i < x\}| < k \leq |\{i: a_i \leq x\}|$.

4 THE DIVIDE-AND-CONQUER PARADIGM

- Special case: **Median** $k = \lfloor \frac{n+1}{2} \rfloor$.
- Simple solution: Sort list in $\tilde{O}(n \log n)$ and look at position k .
- Goal: Solving the selection problem in *linear* time
 1. We start with a randomized algorithm that has linear runtime in expectation.
 2. Afterwards we discuss a selection algorithm with worst-case linear time.

Quickselect [C. A. R. Hoare 1961]

slide 4.9

<pre> 1: procedure QUICKSELECT($A[1 \dots n], k$) 2: if $n = 1$ then return $A[1]$ 3: else 4: Choose pivot $A[p]$ with $1 \leq p \leq n$ 5: $(A[1 \dots n], r) \leftarrow \text{PARTITION}(A[1 \dots n], p)$ 6: if $k < r$ then QUICKSELECT($A[1 \dots r-1], k$) 7: if $k > r$ then QUICKSELECT($A[r+1 \dots n], k-r$) 8: if $k = r$ then return $A[r]$ </pre>	$\triangleright 1 \leq k \leq n$ (Slightly adapted notation:) <pre> 1: procedure PARTITION($A[1, n], p$) 2: swap $A[p] \leftrightarrow A[n]$ 3: $\ell \leftarrow 1$ 4: for $i \leftarrow 1 \dots n-1$ do 5: if $A[i] < A[n]$ then 6: swap $A[\ell] \leftrightarrow A[i]$ 7: $\ell \leftarrow \ell + 1$ 8: swap $A[n] \leftrightarrow A[\ell]$ 9: return $(A[1 \dots n], \ell)$ </pre>
--	---

Runtime analysis of Quickselect

slide 4.10

Runtime again depends on subarray size $n = b - a + 1$ and pivot position $r_n = r - a + 1$:

$$T[n] \leq \max(T[r_n-1], T[n-r_n]) + O(n)$$

- *Best-Case* ($r_n = \lfloor \frac{n+1}{2} \rfloor$): $T[n] \leq T[\frac{n}{2}] + O(n) \Rightarrow O(n)$
- *Worst-Case* ($r_n = 1$): $T[n] \leq T[n-1] + O(n) \Rightarrow O(n^2)$

For a linear time running time a sufficiently good pivot suffices,
e. g.. $0,01n \leq r_n \leq 0,99n$:

$$T[n] \leq T[0,99n] + O(n) \Rightarrow O(n)$$

A *random pivot choice* does this with high probability:

Theorem 4.2. Quickselect with random pivot choice finds the k -th smallest element in expected linear time.

Proof of Theorem 4.2. Let $\mathbb{E}[n]$ be the expected running time for arrays of size n . Furthermore, let “pivot central” be the event that the randomly chosen pivot, after the call of *PARTITION*, has an index in the range of $0,01n \dots 0,99n$. Since all elements are pairwise distinct, as in the analysis of QuickSort, the position of a randomly chosen pivot is uniformly distributed both before and after the call of *PARTITION*. Therefore the event “pivot central” has probability 0,98. We again ignore rounding issues and are only interested in the asymptotic behavior. Hence the arbitrary and not optimal choice of 0,01 and 0,99.

- (1) $\mathbb{E}[n] \leq c \cdot n + \mathbb{P}[\text{pivot central}] \cdot \text{runtime of the recursive call if pivot is central}$
- (2) $\quad + \mathbb{P}[\text{pivot not central}] \cdot \text{runtime of the recursive call if pivot is not central}$
- (3) $\leq c \cdot n + 0,98 \cdot \mathbb{E}[0,99n] + 0,02 \cdot \mathbb{E}[n]$
- (4) $\Leftrightarrow 0,98 \cdot \mathbb{E}[n] \leq c \cdot n + 0,98 \cdot \mathbb{E}[0,99n] \quad // 0,02 \cdot \mathbb{E}[n] \text{ subtracted}$
- (5) $\Leftrightarrow \mathbb{E}[n] \leq \frac{1}{0,98} c \cdot n + \mathbb{E}[0,99n] \quad // \text{divided by } 0,98$

4 THE DIVIDE-AND-CONQUER PARADIGM

Solving with the recursion tree method (for $\ell_{\max} = \lceil \log(n) \cdot \frac{1}{\log(0.99^{-1})} \rceil$ since the maximum level is the smallest natural number $\ell_{\max}$ such that $0.99^{\ell_{\max}} n \leq 1$ holds - the exact value is not important here):

$$(6) \quad \mathbb{E}[n] \leq \sum_{\ell=0}^{\ell_{\max}} c \cdot (0.99^{\ell} n)$$

$$\leq cn \cdot \sum_{\ell=0}^{\infty} 0.99^{\ell} = cn \frac{1}{1-0.99} = O(n)$$

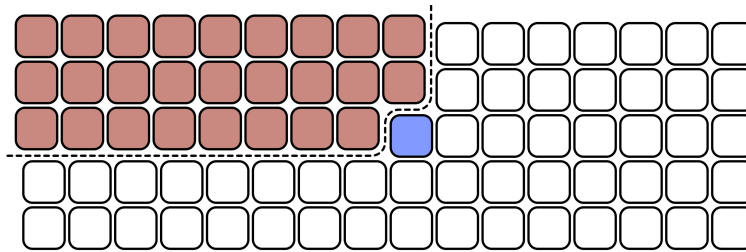
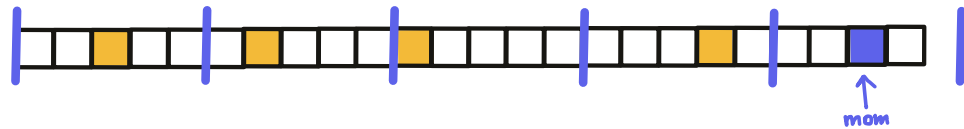
□

4.1 Selection algorithm of Blum, Floyd, Pratt, Rivest, Tarjan

Median-of-Medians [Blum, Floyd, Pratt, Rivest, Tarjan 1972]

The *Median-of-Medians* is a Selection algorithm that *deterministically* chooses a good pivot:

1. Split list into $\frac{n}{5}$ blocks of size 5. (only implicitly) In every block: compute **block median** (median-of-five). *const. time per block $\rightarrow O(n)$*
2. Compute median of $\frac{n}{5}$ block medians (median-of-medians, **mom**). *recursion: $T[\#blocks]$*
3. Use mom as pivot. *recursion: $T[\#smaller / larger\ elements]$*



- Number of elements smaller than **mom**:
at least $\frac{1}{2} \cdot \frac{3}{5} \cdot n - 1 = \frac{3}{10}n$
- Number of elements larger than **mom**:
also at least $\frac{3}{10}n$

Abbildung aus dem Buch "Algorithms" von Jeff Erickson (<http://algorithms.wtf>)

Momselect

- 1: **procedure** $MOMSELECT_{POS}(A[1..n]; k)$
 $\triangleright$ Computes **position** r such that $A[r]$ is a k th smallest element in $A[1..n]$.
- 2: **if** $n = 1$ **then return** 1
- 3: **else**
- 4: $m \leftarrow \lceil \frac{n}{5} \rceil$; init arrays $M[1..m]$ and $M_{pos}[1..m]$. $\triangleright$ Compute **block medians**.
- 5: **for** $i \leftarrow 1 \dots m$ **do**
- 6: Let $A[j]$ median of block $A[5i-4..5i]$; $M[i] \leftarrow A[j]$; $M_{pos}[i] \leftarrow j$;
- 7: $momblock \leftarrow MOMSELECT_{POS}(M[1..m], \lfloor \frac{m+1}{2} \rfloor)$ $\triangleright$ Compute **mom**.
- 8: $mompos \leftarrow M_{pos}[momblock]$
- 9: $(A[1..n], r) \leftarrow PARTITION(A[1..n], mompos)$ $\triangleright$ Use **mom** as pivot.
- 10: **if** $k < r$ **then** $MOMSELECT_{POS}(A[1..r-1], k)$
- 11: **if** $k > r$ **then** $MOMSELECT_{POS}(A[r+1..n], k-r)$
- 12: **if** $k = r$ **then return** r
- 13: **procedure** $MOMSELECT(A[1..n]; k)$
- 14: **return** $A[MOMSELECT_{POS}(A[1..n]; k)]$

Runtime of Momselect

slide 4.13

Let r_n be the *position* of the median-of-medians **mom**. We know: $\frac{3}{10}n \leq r_n \leq \frac{7}{10}n$.¹

Overall we get:

$$\begin{aligned}
 T[n] &\leq T[\text{\#blocks}] + T[\text{\#smaller/larger elements}] + \text{time for computing block medians} \\
 &\leq T\left[\frac{1}{5}n\right] + \max(T[r_n], T[n-r_n]) + c \cdot n \\
 &\leq T\left[\frac{1}{5}n\right] + T\left[\frac{7}{10}n\right] + c \cdot n
 \end{aligned}$$

→ recursion tree analysis gives us the runtime:

Theorem 4.3. Momselect finds the k -th smallest element in a list of n objects in time $O(n)$.

5 Polynomial Multiplication and Fast Fourier Transform

slide 5.1

5.1 Multiplication of Polynomials**Multiplication of Polynomials**

slide 5.2

Problem: Polynomial Multiplication

Input: Polynomials $A(x) = \sum_{i=0}^{\deg A} a_i x^i$ and $B(x) = \sum_{j=0}^{\deg B} b_j x^j$.

Output: Product $C = A \cdot B = \sum_{k=0}^{\deg C} c_k x^k$.

What should we choose as the problem size n ?

For reasons which we will understand later, we set n to be the *smallest* number with:

1. $n \geq \deg C + 1 = \deg A + \deg B + 1$
2. $n = 2^x$ for some $x \in \mathbb{N}$
(n is a power of two)

Algorithms for this problem?

- Simple expansion of product: $c_k = \sum_{i+j=k} a_i \cdot b_j \Rightarrow \text{Runtime } O(n^2)$.
- **Goal:** more clever $O(n \log n)$ algorithm.

Different Representations of Polynomials

slide 5.3

¹Note that $\frac{3}{10}n \leq r_n$ only holds if the pivot does not occur multiple times.

5 POLYNOMIAL MULTIPLICATION AND FAST FOURIER TRANSFORM

Any polynomial $A(x)$ of degree $\deg A$ is uniquely determined by...

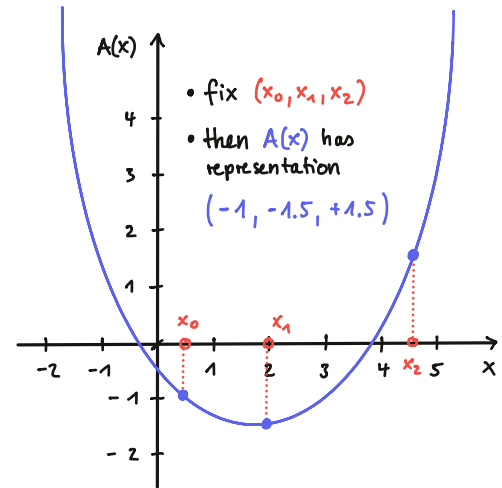
- ... $(\deg A + 1)$ many **coefficients**:

$$A(x) = \sum_{i=0}^{\deg A} a_i x^i.$$

- ... $(\deg A + 1)$ many **interpolation points**:

$(x_0, y_0), (x_1, y_1), \dots, (x_{\deg A}, y_{\deg A})$
[.4em] such that $A(x_i) = y_i$ for all
 $i \in \{0, 1, \dots, \deg A\}$.

- For fixed x_i is $(y_0, \dots, y_{n-1})$ a *interpolation point representation* of the polynomial.
- The x_i must be pairwise different!
Other than that, they can be chosen arbitrarily.



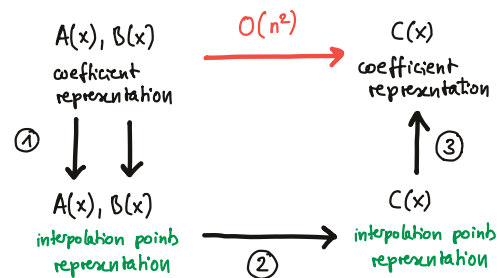
If we have polynomials $A(x)$ and $B(x)$ in *interpolation point representation* (for the same x_i), then we can compute $C = A \cdot B$ in linear time! (multiply element-wise)

slide 5.4

Plan for a faster polynomial multiplication algorithm

Plan:

1. Convert $A(x)$ and $B(x)$:
coefficient r. $\rightarrow$ interpolation points r.
 $O(n^2) \rightsquigarrow O(n \cdot \log n)$
2. Compute product $C(x) = A(x) \cdot B(x)$
in interpolation points r.
3. Convert $C(x)$:
interpolation points r. $\rightarrow$ coefficient r.
 $O(n^2) \rightsquigarrow O(n \cdot \log n)$



First idea for 1. :

- Compute all points $y_i = A(x_i)$
in linear time for each point.

Better: Fast Fourier Transform!

We need a faster algorithm
for converting between repr.'s!

First idea for 3. :

- Construct linear equation system $y_\ell \cdot z_\ell = \sum_{k=0}^{n-2} (x_\ell)^k \cdot c_k$
with $n-1$ variables c_k
and $n-1$ linear equalities for $\ell = 0, \dots, n-2$.
- Solve system with Gaussian elimination in $O(n^3)$.

Better: Inverse Fast Fourier Transform!

Divide-and-Conquer approach for evaluating polynomials

- Converting a polynomial to interpolation points representation = **evaluating the polynomial!**

$$A(x) = a_0 x^0 + a_1 x^1 + a_2 x^2 + a_3 x^3 + a_4 x^4 + a_5 x^5 + a_6 x^6 + a_7 x^7$$

$$= a_0 x^0 + a_2 x^2 + a_4 x^4 + a_6 x^6 + a_1 x^1 + a_3 x^3 + a_5 x^5 + a_7 x^7$$

$$= a_0 (x^2)^0 + a_2 (x^2)^1 + a_4 (x^2)^2 + a_6 (x^2)^3 + a_1 (x^2)^0 x + a_3 (x^2)^1 x + a_5 (x^2)^2 x + a_7 (x^2)^3 x$$

$$= a_0 (x^2)^0 + a_2 (x^2)^1 + a_4 (x^2)^2 + a_6 (x^2)^3 + x \cdot \left(a_1 (x^2)^0 + a_3 (x^2)^1 + a_5 (x^2)^2 + a_7 (x^2)^3 \right)$$

slide 5.5

Maybe this could be our **Divide** step:

$$A(x) = A_0(x^2) + x \cdot A_1(x^2)$$

$$\text{with } A_0(x) = a_0 x^0 + a_2 x^1 + a_4 x^2 + a_6 x^3 \text{ and } A_1(x) = a_1 x^0 + a_3 x^1 + a_5 x^2 + a_7 x^3$$

Divide-and-Conquer approach for evaluating polynomials

slide 5.6

General formula:

$$\begin{aligned} A(x) &= \sum_{i=0}^{n-1} a_i x^i \\ &= \underbrace{\sum_{\ell=0}^{\frac{n}{2}-1} a_{2\ell} x^{2\ell}}_{\text{even monomials}} + \underbrace{\sum_{\ell=0}^{\frac{n}{2}-1} a_{2\ell+1} x^{2\ell+1}}_{\text{odd monomials}} \\ &= \underbrace{\sum_{\ell=0}^{\frac{n}{2}-1} a_{2\ell} (x^2)^\ell}_{=: A_0(x^2)} + x \cdot \underbrace{\left(\sum_{\ell=0}^{\frac{n}{2}-1} a_{2\ell+1} (x^2)^\ell \right)}_{=: A_1(x^2)} \\ &= A_0(x^2) + x \cdot A_1(x^2) \end{aligned}$$

Draft Algorithm:

EVALUATE($A, (x_0, \dots, x_{\deg A})$):

- $A_0(x) := \dots$ of degree $\approx \frac{1}{2} \deg A$
 $A_1(x) := \dots$ of degree $\approx \frac{1}{2} \deg A$

• Recursion:

EVALUATE($A_0, (x_0^2, \dots, x_{\deg A}^2)$)

EVALUATE($A_1, (x_0^2, \dots, x_{\deg A}^2)$)

• Merge: For each x_i :

Compute $A(x_i) = A_0(x_i^2) + x_i \cdot A_1(x_i^2)$

• Problem:

Runtime of the Merge phase?

- $O(1)$ time to compute each $A(x_i)$ – good!
- **but:** $\deg A$ many x_i at each recursion level!
- $O(\deg A)$ **at every recursion level!**

• At

each level, we also need to halve the # of interpolation points to get a good runtime overall!

• Require n

to be a power of 2.

• Require

the interpolation points $\{x_0, \dots, x_{n-1}\}$ to be the n -th roots of unity $\{\omega^0, \dots, \omega^{n-1}\}$.

• The algo-

rithm we just invented is the **Fast Fourier Transformation (FFT)!**

We are done!

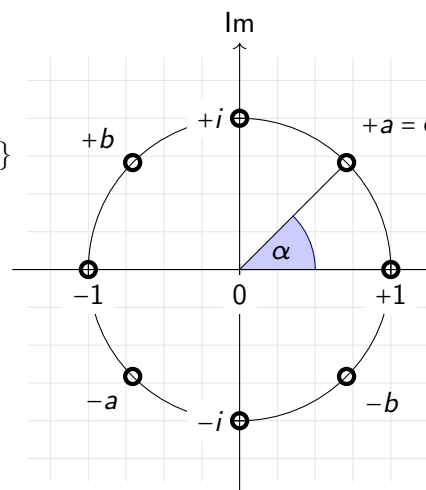
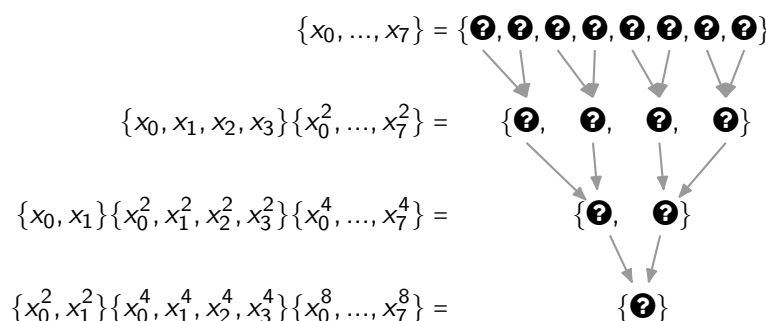
Squaring should halve the number of interpolation points

slide 5.7

5 POLYNOMIAL MULTIPLICATION AND FAST FOURIER TRANSFORM

Goal: Choose interpolation points so that their number halves with every squaring.

Case $n = 2$: Case $n = 4$: Case $n = 8$:



Strategy in general: Choose all n solutions of $x^n = 1$! These numbers are the n -th roots of unity in $\mathbb{C}$. How do they look like?

slide 5.8

The (Primitive) n -th Roots of Unity $\sqrt[n]{1}$

Definition 5.1. • n -th root of unity in $\mathbb{C}$:

complex number x with $x^n = 1$.

• **primitive** n -th root of unity in $\mathbb{C}$:

complex number ω with $\omega^n = 1$

and $\omega^\ell \neq 1$ for $\ell \in \{1, \dots, n-1\}$.

The following are true for every ω that is a primitive n -th root of unity:

1. ω^ℓ for $\ell \in \{0, \dots, n-1\}$ are n different n -th roots of unity. (Otherwise there would be $\omega^\ell = \omega^{\ell'}$ for some $\ell > \ell'$, and therefore $\omega^{\ell-\ell'} = 1$, a contradiction!)

2. $\omega^{\frac{n}{2}} = -1$. (We know $(\omega^{\frac{n}{2}})^2 = \omega^n = 1$, and this implies $\omega^{\frac{n}{2}} \in \{-1, 1\}$! Also, $\omega^{\frac{n}{2}} \neq 1$ because ω is primitive!)

3. $(-1) \cdot \omega^\ell = (\omega^{\frac{n}{2}}) \cdot \omega^\ell = \omega^{\frac{n}{2}} \omega^\ell = \omega^{\ell + \frac{n}{2}}$

How do we use this?

4. $(\omega^\ell)^2 = \omega^{2\ell}$
 $(\omega^{\ell + \frac{n}{2}})^2 = (-\omega^\ell)^2 = \omega^{2\ell}$

→ When both ω^ℓ and $\omega^{\ell + \frac{n}{2}}$ are in the interpolation point set, only one remains after squaring!

We have an explicit formula for such a primitive n -th root of unity:

$$\begin{aligned} \omega &= \exp\left(\frac{1}{n} \cdot 2\pi i\right) = e^{(1/n) \cdot 2\pi i} \\ &= \cos\left(\frac{1}{n} 2\pi\right) + i \sin\left(\frac{1}{n} 2\pi\right) \end{aligned}$$

(In general, there is more than one primitive root of unity! But we only need one of them, so let's just agree to use $\omega = \exp(\frac{1}{n} \cdot 2\pi i)$.)

slide 5.9

The (Primitive) n -th Roots of Unity $\sqrt[n]{1}$

Key facts from the last slide:

- **primitive** n -th root of unity in $\mathbb{C}$: complex number ω with $\omega^n = 1$ and $\omega^\ell \neq 1$ for $\ell \in \{1, \dots, n-1\}$.
 - Formula: $\omega = \exp(\frac{1}{n} \cdot 2\pi i)$
4. $(\omega^\ell)^2 = \omega^{2\ell}$
 $(\omega^{\ell+\frac{n}{2}})^2 = (-\omega^\ell)^2 = \omega^{2\ell}$

- Choose some number n .
- Fix a primitive n -th root of unity ω .
- Fix **interpolation points**: $\{\omega^1, \omega^2, \dots, \omega^{n-1}, \omega^n\}$
 - We want to apply our **Fact 4**!
 - For every point ω^ℓ , its counterpart $\omega^{\ell+\frac{n}{2}}$ must also be in the set! $\rightarrow \frac{n}{2}$ must be an integer!

Goal: Choose interpolation points so that their number halves with every squaring.

For this step to work at every level, the numbers $n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots$ all have to be divisible by 2!

Requirement: n must be a power of 2!

– **Requirement:** n must be divisible by 2!

Square the interpolation points:

$$\begin{aligned} & \text{square } \{\omega^1, \omega^2, \dots, \omega^{n-1}, \omega^n\} \\ &= \text{square } \{\omega^1, \omega^2, \dots, \omega^{\frac{n}{2}}, \omega^{1+\frac{n}{2}}, \omega^{2+\frac{n}{2}}, \dots, \omega^{\frac{n}{2}+\frac{n}{2}}\} \\ &= \{(\omega^1)^2, (\omega^2)^2, \dots, (\omega^{\frac{n}{2}})^2, (\omega^{1+\frac{n}{2}})^2, (\omega^{2+\frac{n}{2}})^2, \dots, (\omega^{\frac{n}{2}+\frac{n}{2}})^2\} \\ &= \{(\omega^1)^2, (\omega^2)^2, \dots, (\omega^{\frac{n}{2}})^2\} \\ &= \{(\omega^2)^1, (\omega^2)^2, \dots, (\omega^2)^{\frac{n}{2}}\} \text{ square } \{\omega^1, \dots, \omega^n\} = \{(\omega^2)^1, \dots, (\omega^2)^{\frac{n}{2}}\} \end{aligned}$$

For the next recursive level, we get:

- Number $\frac{n}{2}$.
- ω^2 is primitive $\frac{n}{2}$ -th root of unity. (black-board)
- Interpolation points $\{(\omega^2)^1, (\omega^2)^2, \dots, (\omega^2)^{\frac{n}{2}}\}$.

Proof (that ω^2 is a primitive $\frac{n}{2}$ -th root of unity): Since ω is n -th root of unity it follows that $(\omega^2)^{\frac{n}{2}} = \omega^{\frac{2n}{2}} = \omega^n = 1$. It remains to show that $(\omega^2)^l \neq 1$ for $l \in \{1, \dots, \frac{n}{2}-1\}$. To prove this we will use the following claim:

Claim. Let ω be an n -th root of unity. Then:

$$\omega^m = 1 \Leftrightarrow n \mid m$$

“ $\Leftarrow$ ”: From $n \mid m$ it follows that $m = k \cdot n$ for some $k \in \mathbb{Z}$. We obtain $\omega^m = \omega^{n \cdot k} = (\omega^n)^k = 1^k = 1$.

“ $\Rightarrow$ ”: We perform the division of m by n with remainder and obtain $m = q \cdot n + r$ with $0 \leq r < n$. It follows that $\omega^m = \omega^{q \cdot n + r} = (\omega^n)^q \cdot \omega^r = 1 \cdot \omega^r = \omega^r$ and thus $\omega^r = \omega^m = 1$. Since ω is a primitive n -th root of unity r has to be 0 and $m = q \cdot n$ which shows the claim.

With this we can easily finish the proof. Let $(\omega^2)^k = 1$. Then $\omega^{2k} = 1$ and we conclude that $n \mid 2k$ holds using the claim shown above. Since n is a power of 2 (and thus divisible by 2) we obtain $\frac{n}{2} \mid k$. Hence we conclude that either $k = 0$ or $k \geq \frac{n}{2}$. $\square$

5.2 Fast Fourier Transformation

Fast Fourier Transformation

slide 5.10

Problem: Discrete Fourier Transformation (DFT)

Input: coefficient vector $(a_0, \dots, a_{n-1})$ of a polynomial $A(x)$

Output: interpolation point representation $(y_0, \dots, y_{n-1})$ of $A(x)$, where $y_\ell = A(\omega^\ell)$ is evaluated over the n -th complex roots of unity

5 POLYNOMIAL MULTIPLICATION AND FAST FOURIER TRANSFORM

The following Divide-and-Conquer algorithm for DFT is called *Fast Fourier Transformation (FFT)*:

```

procedure  $FFT(a_0, \dots, a_{n-1})$ 
  if  $n = 1$  then return  $(a_0)$ 
   $\omega = e^{2\pi i/n}$  ▷ primitive  $n$ -th root of unity
   $(s_0, \dots, s_{\frac{n}{2}-1}) \leftarrow FFT(a_0, a_2, \dots, a_{n-2})$  ▷  $s_\ell = A_0((\omega^2)^\ell)$ 
   $(t_0, \dots, t_{\frac{n}{2}-1}) \leftarrow FFT(a_1, a_3, \dots, a_{n-1})$  ▷  $t_\ell = A_1((\omega^2)^\ell)$ 
  for  $\ell = 0, \dots, \frac{n}{2} - 1$  do
     $y_\ell \leftarrow s_\ell + \omega^\ell \cdot t_\ell$  ▷  $y_\ell = A(\omega^\ell) = A_0((\omega^\ell)^2) + \omega^\ell \cdot A_1((\omega^\ell)^2)$ 
     $y_{\ell+\frac{n}{2}} \leftarrow s_\ell - \omega^\ell \cdot t_\ell$  ▷  $y_{\ell+\frac{n}{2}} = A(\omega^{\ell+\frac{n}{2}}) = A_0((\omega^{\ell+\frac{n}{2}})^2) + \omega^{\ell+\frac{n}{2}} \cdot A_1((\omega^{\ell+\frac{n}{2}})^2)$ 
  return  $(y_0, \dots, y_{n-1})$  ▷  $\dots = A_0((-\omega^\ell)^2) - \omega^\ell \cdot A_1((-\omega^\ell)^2)$ 

```

slide 5.11

Fast Fourier Transformation

Theorem 5.2. Let $n = 2^d$ for a $d \in \mathbb{N}$
 and $(a_0, \dots, a_{n-1})$ be the coefficient representation
 of a polynomial $A(x) = \sum_{i=0}^{n-1} a_i x^i$ of degree $\leq n-1$.
 Then FFT computes the interpolation point representation $(A(\omega^0), \dots, A(\omega^{n-1}))$
 over the n -th complex roots of unity ω^ℓ as interpolation points
 in time $O(n \log n)$.

Proof. Correctness follows from previous considerations. Runtime follows by solving the recurrence $T[n] \leq 2 \cdot T[\frac{n}{2}] + c \cdot n$; $T[1] \leq c$. $\square$

5.3 Inverse Fourier transformation

slide 5.12

Discrete Fourier Transformation as Matrix-Vector-Multiplication

We know: Coefficients $(a_0, \dots, a_{n-1})$

FFT computes: $y_\ell = \sum_{\ell=0}^{n-1} a_\ell \omega^\ell$

As matrix-vector product: $M(\omega) \cdot \vec{a} = \vec{y}$

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & (\omega^2)^2 & \dots & (\omega^2)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & (\omega^{n-1})^2 & \dots & (\omega^{n-1})^{n-1} \end{pmatrix} \cdot \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

slide 5.13

Interpolation

From interpolation point representation over ω^ℓ to coefficient representation

We know: n value $y_\ell = C(\omega^\ell)$

We want to compute: coefficients c_i with $\sum_{i=0}^{n-1} c_i \cdot (\omega^\ell)^i = y_\ell$ for $\ell = 0, \dots, n-1$.

5 POLYNOMIAL MULTIPLICATION AND FAST FOURIER TRANSFORM

Linear system of equations as matrix-vector product $M(\omega) \cdot \bar{c} = \bar{y}$:

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & (\omega^2)^2 & \dots & (\omega^2)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & (\omega^{n-1})^2 & \dots & (\omega^{n-1})^{n-1} \end{pmatrix} \cdot \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

Solution strategy: Compute the inverse $M(\omega)^{-1}$ and calculate $\bar{c} = M(\omega)^{-1} \cdot \bar{y}$.

The Inverse $M(\omega)^{-1}$

slide 5.14

Let $\hat{\omega} := \omega^{n-1}$. We know: $\omega \cdot \hat{\omega} = \omega^n = 1$. Therefore, $\hat{\omega} = \omega^{-1}$ is the inverse of ω .

Lemma 5.3. $M(\omega)^{-1} = \frac{1}{n} \cdot M(\hat{\omega})$.

Proof. We show: $M(\omega) \cdot M(\hat{\omega}) = n \cdot E$.

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & (\omega^2)^2 & \dots & (\omega^2)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & (\omega^{n-1})^2 & \dots & (\omega^{n-1})^{n-1} \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \hat{\omega} & \hat{\omega}^2 & \dots & \hat{\omega}^{n-1} \\ 1 & \hat{\omega}^2 & (\hat{\omega}^2)^2 & \dots & (\hat{\omega}^2)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \hat{\omega}^{n-1} & (\hat{\omega}^{n-1})^2 & \dots & (\hat{\omega}^{n-1})^{n-1} \end{pmatrix} = n \cdot E$$

□

Interpolation using FFT

slide 5.15

Lemma 5.4. $\hat{\omega}$ is a *primitive* n -th root of unity. I.e. $\hat{\omega}^k \neq 1$ for $k = 1, \dots, n-1$.

- By Lemma 5.3 it suffices to compute the matrix-vector product $\bar{c} = M(\omega)^{-1} \cdot \bar{y} = \frac{1}{n} \cdot M(\hat{\omega}) \cdot \bar{y}$.
- By Lemma 5.4 we can use FFT again, this time with primitive root of unity $\hat{\omega}$ instead of ω !
- This variant is also called the **Inverse Fast Fourier Transformation (IFFT)**.

Multiplying polynomials in $O(n \log n)$

slide 5.16

procedure *FFT*($a_0, \dots, a_{n-1}$)

if $n = 1$ **then return** (a_0)

$\omega = e^{2\pi i/n}$

$(s_0, \dots, s_{\frac{n}{2}-1}) \leftarrow \text{FFT}(a_0, a_2, \dots, a_{n-2})$

$(t_0, \dots, t_{\frac{n}{2}-1}) \leftarrow \text{FFT}(a_1, a_3, \dots, a_{n-1})$

for $\ell = 0, \dots, \frac{n}{2} - 1$ **do**

$y_\ell \leftarrow s_\ell + \omega^\ell \cdot t_\ell$

$y_{\frac{n}{2}+\ell} \leftarrow s_\ell - \omega^\ell \cdot t_\ell$

return ($y_0, \dots, y_{n-1}$)

procedure *IFFT*($a_0, \dots, a_{n-1}$)

if $n = 1$ **then return** (a_0)

$\omega = e^{2\pi i(n-1)/n}$

$(s_0, \dots, s_{\frac{n}{2}-1}) \leftarrow \text{IFFT}(a_0, a_2, \dots, a_{n-2})$

$(t_0, \dots, t_{\frac{n}{2}-1}) \leftarrow \text{IFFT}(a_1, a_3, \dots, a_{n-1})$

for $\ell = 0, \dots, \frac{n}{2} - 1$ **do**

$y_\ell \leftarrow s_\ell + \omega^\ell \cdot t_\ell$

$y_{\frac{n}{2}+\ell} \leftarrow s_\ell - \omega^\ell \cdot t_\ell$

return ($y_0, \dots, y_{n-1}$)

procedure *FFT-POLYNOMIAL-MULTIPLICATION*(A, B)

Input: Polynomials $A(x) = \sum_{i=0}^{n/2-1} a_i x^i$ and $B(x) = \sum_{j=0}^{n/2-1} b_j x^j$.

Output: Coefficients $(c_0, \dots, c_{n-2})$ of the product $C = A \cdot B = \sum_{k=0}^{n-2} c_k x^k$.

$(y_0, \dots, y_{n-1}) \leftarrow \text{FFT}(a_0, a_1, \dots, a_{n/2-1}, 0, \dots, 0)$

$(z_0, \dots, z_{n-1}) \leftarrow \text{FFT}(b_0, b_1, \dots, b_{n/2-1}, 0, \dots, 0)$

$(c'_0, \dots, c'_{n-1}) \leftarrow \text{IFFT}(y_0 \cdot z_0, y_1 \cdot z_1, \dots, y_{n-1} \cdot z_{n-1})$

return ($\frac{1}{n} \cdot c'_0, \dots, \frac{1}{n} \cdot c'_{n-2}$)

5.4 The master theorem

slide 5.17

The master theorem

- Usefull for divide strategies splitting into equal parts & merge costs of the form $\Theta(n^\alpha)$.
- Here in a slightly simplified form (assuming $n = b^x$ to avoid rounding).

Theorem 5.5 (Master theorem). Let $a, b, c, \alpha > 0$ be rational numbers. Let T be a recurrence, that is defined for all $n = b^x$ with $x \in \mathbb{N}$ as follows:

$$T[1] = c; \quad T[n] = a \cdot T\left[\frac{n}{b}\right] + c \cdot n^\alpha$$

Then

1. $a = b^\alpha \implies T[n] = \Theta(n^\alpha \log n)$
2. $a < b^\alpha \implies T[n] = \Theta(n^\alpha)$
3. $a > b^\alpha \implies T[n] = \Theta(n^{\log_b a})$

Proof: Recursion tree gives: $T[n] = \sum_{\ell=0}^{\log_b n} a^\ell \cdot c \cdot \left(\frac{n}{b^\ell}\right)^\alpha = c \cdot n^\alpha \cdot \sum_{\ell=0}^{\log_b n} \left(\frac{a}{b^\alpha}\right)^\ell$. It remains to analyse this expression asymptotically. $\square$

Part II

Dynamic Programming

6 Dynamic Programming

slide 6.1

6.1 Introduction

Dynamic Programming

slide 6.2

The algorithm design paradigm *dynamic programming* dates back to Richard Bellman (1952).

programming ... not “coding”, but constructing a plan/schedule (a timetable, a sports or theater programme). Usually an optimization task.

dynamic ... “in multiple steps, multi-stage process” + a term that sells well:

“It also has a very interesting property as an adjective, and that is it’s impossible to use the word, *dynamic*, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It’s impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities.” [R. Bellman 1984]

slide 6.3

Fibonacci recursive

slide 6.4

```

1: procedure RECFIBO( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   else if  $n = 1$  then
5:     return 1
6:   else
7:     return RECFIBO( $n - 1$ ) + RECFIBO( $n - 2$ )

```

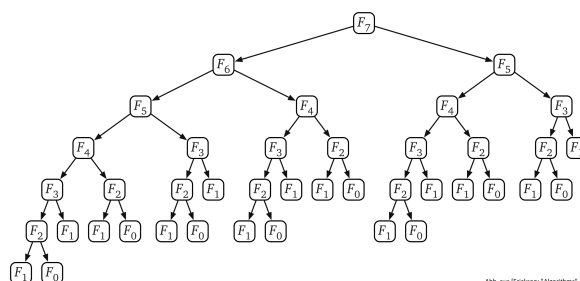


Abb. aus [Erickson: "Algorithms", 2013]

Memoization

slide 6.5

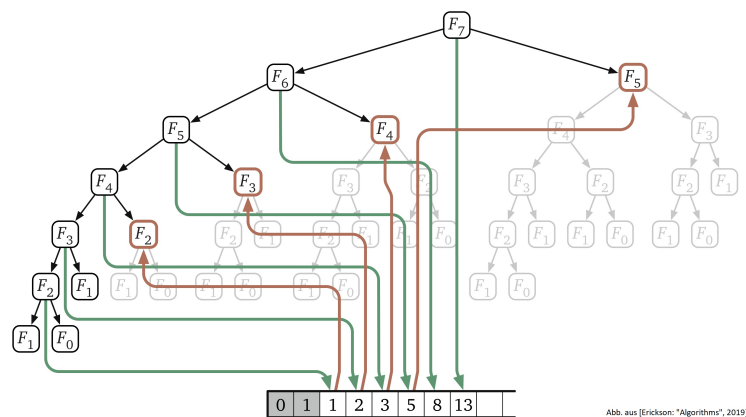
- *RECFIBO*(n) is extremely inefficient, because the same Fibonacci number is computed multiple times.
- A solution to this is *memoization*: store computed results globally for future use!
- I. e. store Fibonacci numbers in an array $F[1 \dots n]$.

6 DYNAMIC PROGRAMMING

```
1: procedure MEMFIBO( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   else if  $n = 1$  then
5:     return 1
6:   else
7:     if  $F[n]$  is undefined then
8:        $F[n] \leftarrow \text{MEMFIBO}(n-1) + \text{MEMFIBO}(n-2)$ 
9:     return  $F[n]$ 
```

slide 6.6

Fibonacci recursive using memoization



- Runtime: $O(n)$ additions^{*)}
- Memory usage: $O(n)$ numbers^{*)}

^{*)} Note how this wording deliberately ignores the huge size of these numbers!

slide 6.7

Fibonacci iteratively

- When it is clear how stored subresults depend on each other, the algorithm can be formulated iteratively in a “bottom-up” manner.
- When it is clear which subresults aren't used anymore, storage can be saved by discarding them.

```
1: procedure ITERFIBO( $n$ )
2:    $F[0] \leftarrow 0$ 
3:    $F[1] \leftarrow 1$ 
4:   for  $i \leftarrow 2 \dots n$  do
5:      $F[i] \leftarrow F[i-1] + F[i-2]$ 
6:   return  $F[n]$ 
```

- Runtime: $O(n)$ additions
- Memory usage: $O(n)$ numbers

```
1: procedure ITERFIBO2( $n$ )
2:    $prev \leftarrow 1$ 
3:    $curr \leftarrow 0$ 
4:   for  $i \leftarrow 1 \dots n$  do
5:      $next \leftarrow curr + prev$ 
6:      $prev \leftarrow curr$ 
7:      $curr \leftarrow next$ 
8:   return  $curr$ 
```

- Runtime: $O(n)$ additions
- Memory usage: $O(1)$ numbers

slide 6.8

Fibonacci: Demo in Python

```

from functools import cache

def fib_without_cache(n: int):
    # Base cases.
    if n == 0:
        return 0
    if n == 1:
        return 1

    # Recursive formula.
    f1 = fib_without_cache(n - 1)
    f2 = fib_without_cache(n - 2)
    return f1 + f2

@cache
def fib_with_cache(n: int):
    # Base cases.
    if n == 0:
        return 0
    if n == 1:
        return 1

    # Recursive formula.
    f1 = fib_with_cache(n - 1)
    f2 = fib_with_cache(n - 2)
    return f1 + f2

```

Template for dynamic programming

slide 6.9

1. Formulate the problem recursively.
 - (a) *Formalization*: Describe which problem to solve and specify the output of the algorithm.
 - (b) *Recursive solution*: Describe how to obtain an (optimal) solution for an instance by recursively computing an (optimal) solution of the same problem on smaller subinstances.
2. Combine partial solutions bottom-up:
 - (a) *Memoization*: Choose global data structure to store results for all recursive calls.
 - (b) *Dependencies*: How does a solution depend other solutions? A figure might be helpful. What are the base cases?
 - (c) Find a good *evaluation order*, so that computing the result of a subproblem only depends on already computed results.
3. Formulate the algorithm iteratively and analyse runtime and storage consumption. Optimize storage consumption.

6.2 Text segmentation

slide 6.10

Input example: SPLITTHISINTOSINGLEWORDS

Output example: SPLIT THIS INTO SINGLE WORDS

Blackbox: $IsWord(i, j)$ decides, if $A[i..j]$ is a “word”.

For simplicity we first design an algorithm that decides whether the String is *splittable*:

Problem: Text segmentation (decision version)

Input: A string $A[1..n]$ and blackbox access to $IsWord(i, j)$.

Output: Are there indices $1 = i_0 \leq i_1 \leq \dots \leq i_\ell = n + 1$ for $\ell \geq 1$ such that $IsWord(i_{s-1}, i_s - 1) = true$ for all $1 \leq s \leq \ell$?

slide 6.11

Step 1a (Formalization): We solve (for now) the decision problem, whether a sequence is splittable. The function $SPLITTABLE(i)$ shall report TRUE if $A[i..n]$ is splittable in words. Goal is to compute $SPLITTABLE(1)$.

Step 1b (Recursive solution):

$$SPLITTABLE(i) = \begin{cases} \text{True}, & \text{if } i > n, \\ \bigvee_{j=i}^n (ISWORD(i, j) \wedge SPLITTABLE(j+1)) & \text{otherwise.} \end{cases}$$

slide 6.12

Step 2a (Memoization)

- All possible calls on input $A[1..n]$: $SPLITTABLE(i)$ for $i = 1, \dots, n+1$.
- Store results in array SplitTable[1..n+1].

Step 2b (Dependencies)

- SplitTable[n+1] = true is the base case.
- SplitTable[i] requires SplitTable[l] for $\ell = i+1, \dots, n+1$.

Step 2c (Evaluation order):

- Compute SplitTable[i] for $i = n+1, \dots, 1$.

Step 3 (Algorithm and analysis):

```

1: SplitTable[n+1] ← true
2: for i ← n...1 do
3:   SplitTable[i] ← false
4:   for j ← i...n do
5:     if ISWORD(i, j) and SplitTable[j+1] then SplitTable[i] ← true
6: return SplitTable[1]
```

slide 6.13

Computing split points

Exercise: Adapt the algorithm to compute the split points, i. e., that it solves the following problem:

Problem: Text segmentation

Input: A string $A[1..n]$ and blackbox access to $ISWORD(i, j)$.

Output: Compute a list of indices $1 = i_0 \leq i_1 \leq \dots \leq i_\ell = n+1$ for $\ell \geq 1$ such that $ISWORD(i_{s-1}, i_s - 1) = \text{true}$ for all $1 \leq s \leq \ell$ or conclude that no such list exists.

6.3 Edit distance

slide 6.14

The *edit distance* of two strings is the minimal number of necessary character insertions, deletions or substitutions needed to transform one string into the other.

Example (transforming ARBEIT to FREIZEIT):

$ARBEIT \xrightarrow{\text{del B}} AREIT \xrightarrow{\text{sub A} \rightsquigarrow \text{F}} FREIT \xrightarrow{\text{ins Z}} FREIZT \xrightarrow{\text{ins E}} FREIZET \xrightarrow{\text{ins I}} FREIZEIT$
 $ARBEIT \xrightarrow{\text{sub A} \rightsquigarrow \text{F}} FRBEIT \xrightarrow{\text{ins E}} FREBEIT \xrightarrow{\text{ins I}} FREIBEIT \xrightarrow{\text{sub B} \rightsquigarrow \text{Z}} FREIZEIT$

Problem: Edit distance**Input:** Two strings $A[1..m]$ and $B[1..n]$.**Output:** The edit distance of $A[1..m]$ and $B[1..n]$.

slide 6.15

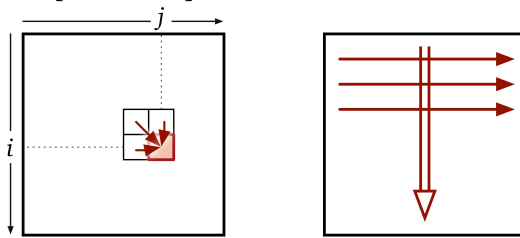
Step 1: Formalization + recursive solutionFor $A[1..m]$ and $B[1..n]$ routine $EDIT(i, j)$ returns edit distance of $A[1..i]$ and $B[1..j]$.

$$EDIT(i, j) = \begin{cases} i, & \text{if } j = 0, \\ j, & \text{if } i = 0, \text{ else:} \\ \min \begin{cases} EDIT(i, j-1) + 1 & \text{insertion} \\ EDIT(i-1, j) + 1 & \text{deletion} \\ EDIT(i-1, j-1) + (A[i] \neq B[j]) & \text{replace / keep} \end{cases} \end{cases}$$

Step 2: Memoization, dependencies, evaluation order

slide 6.16

- Memoization: Store results in 2-dim array $Edit[0..m, 0..n]$.
Base cases: $Edit[i, 0] = i$ and $Edit[0, j] = j$.
- Dependencies and evaluation order: $Edit[i, j]$ requires $Edit[i-1, j]$, $Edit[i, j-1]$, and $Edit[i-1, j-1]$

**Step 3: Algorithm and analysis**

slide 6.17

```

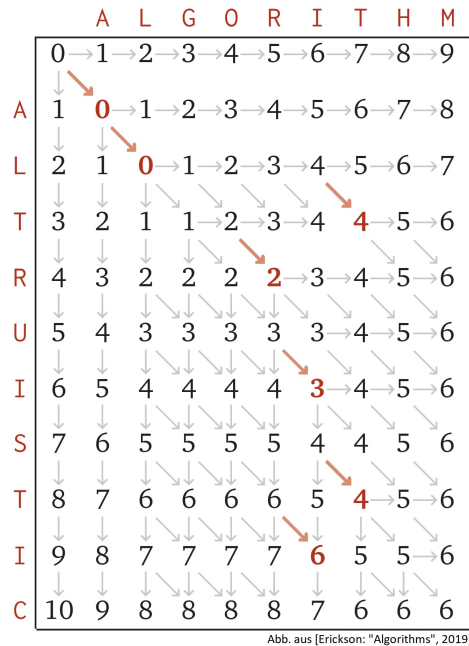
1: procedure EDITDISTANCE( $A[1..m]$ ,  $B[1..n]$ )
2:   initialize array  $Edit[0..m, 0..n]$ 
3:   for  $j \leftarrow 0 \dots n$  do
4:      $Edit[0, j] \leftarrow j$ 
5:   for  $i \leftarrow 1 \dots m$  do
6:      $Edit[i, 0] \leftarrow i$ 
7:     for  $j \leftarrow 1 \dots n$  do
8:        $ins \leftarrow Edit[i, j-1] + 1$ 
9:        $del \leftarrow Edit[i-1, j] + 1$ 
10:       $rep \leftarrow Edit[i-1, j-1] + (A[i] \neq B[j])$ 
11:       $Edit[i, j] \leftarrow \min\{ins, del, rep\}$ 
12:   return  $Edit[m, n]$ 

```

Edit distance – an example run

slide 6.18

6 DYNAMIC PROGRAMMING



- $\downarrow$ = del, $\rightarrow$ = ins, $\searrow$ = rep
- Edit distance of ALTRUISTIC und ALGORITHM is $Edit[m, n] = 6$
- Backward search from $Edit[m, n]$ to $Edit[0, 0]$ provides optimal edit sequences in $O(m + n)$ (here 3 in total).

6.4 Longest Increasing Subsequence

slide 6.19

Definition 6.1. An *longest increasing subsequence* of a sequence of integers $(a_1, \dots, a_n)$ is a sequence $(a_{i_1}, \dots, a_{i_\ell})$

- of maximal length ("longest"),
- such that $a_{i_1} < \dots < a_{i_\ell}$ ("increasing")
- and $i_1 < \dots < i_\ell$ ("subsequence").

Example: 9, 1, 3, 5, 2, 4, 7, 6, 8

- 9, 1, 3, 5, 2, 4, 7, 6, 8 (length 6) ✗ not increasing
- 9, 1, 3, 5, 2, 4, 7, 6, 8 (length 3) ✗ not of maximal length
- 9, 1, 3, 5, 2, 4, 7, 6, 8 (length 5) ✓ increasing, maximal length
- 9, 1, 3, 5, 2, 4, 7, 6, 8 (length 5) ✓ increasing, maximal length
- 9, 1, 3, 5, 2, 4, 7, 6, 8 (length 5) ✓ increasing, maximal length

slide 6.20

Problem: Longest Increasing Subsequence

Input: An array $A[1 \dots n]$ of integers.

Output: The length ℓ of a longest increasing subsequence.

Rough idea for a recursive solution:

$$\text{Solution for } [a_1, a_2, \dots, a_n] = \text{best of } \begin{cases} \text{(don't choose } a_1) & \text{Solution for } [a_2, \dots, a_n] \\ \text{(do choose } a_1) & 1 + \left(\text{Solution for } [a_2, \dots, a_n] \text{ where all } a_j \text{ in the solution are } a_j > a_1 \right) \end{cases}$$

Recursive solution depends on **starting index** and an **(optional) lower bound**. Let's formalize this idea!

slide 6.21

Step 1: Formalization + recursive solution

We **define** the function: $LISBIGGER(b, j) \dots$ Length of a longest increasing subsequence in $A[j \dots n]$, in which every number is larger than $A[b]$. (j is the starting index; b is the index of the lower bound.)

How do we make the bound optional? We define $A[0] := -\infty$! Then, $b = 0$ corresponds to no lower bound.

Goal: Compute $LISBIGGER(0, 1)$.

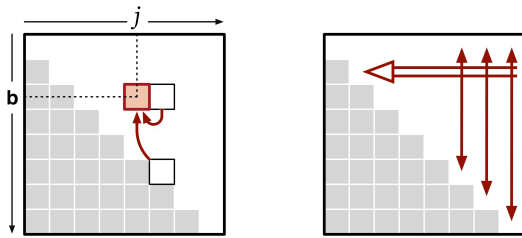
We come up with a **recursive solution** for our function $LISBIGGER(b, j)$:

$$LISBIGGER(b, j) = \begin{cases} 0 & \text{if } j > n \\ LISBIGGER(b, j+1) & \text{if } A[b] \geq A[j] \\ \max(LISBIGGER(b, j+1), 1 + LISBIGGER(j, j+1)) & \text{else.} \end{cases}$$

Step 2: Memoization, dependencies, evaluation order

slide 6.22

- Memoization: Store results in 2-dim array $LISB[0 \dots n, 1 \dots n+1]$. Base cases: $LISB[b, n+1] = 0$, for all b
- Dependencies and evaluation order: $LISB[b, j]$ requires $LISB[b, j+1]$ and $LISB[j, j+1]$



Step 3: Algorithm and analysis

slide 6.23

```

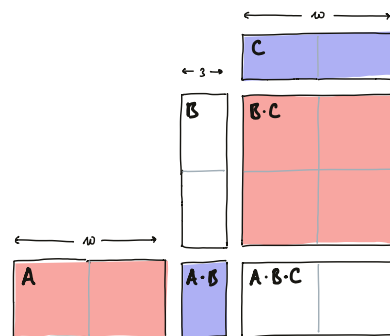
1: procedure LONGESTINCREASINGSUBSEQUENCE( $A[1 \dots n]$ )
2:    $A[0] \leftarrow -\infty$ 
3:   for  $b \leftarrow 0 \dots n$  do  $LISB[b, n+1] \leftarrow 0$ 
4:   for  $j \leftarrow n \dots 1$  do
5:     for  $b \leftarrow 0 \dots j-1$  do
6:        $keep \leftarrow 1 + LISB[j, j+1]$ 
7:        $skip \leftarrow LISB[b, j+1]$ 
8:       if  $A[b] \geq A[j]$  then
9:          $LISB[b, j] \leftarrow skip$ 
10:      else
11:         $LISB[b, j] \leftarrow \max(keep, skip)$ 
12:   return  $LISB[0, 1]$ 

```

6.5 Matrix chain multiplication

slide 6.24

- The *runtime cost* for multiplying an $r \times s$ -matrix with an $s \times t$ -matrix is $c(r, s, t) := r \cdot s \cdot t$.^a
(The result matrix has $r \cdot t$ many entries, each of which is a product of two vectors of length s .)
Example on the right: $c(5, 10, 3) = 5 \cdot 10 \cdot 3 = 150$.
- Cost for computing $A \cdot B \cdot C$:
 - $(A \cdot B) \cdot C$: $c(5, 10, 3) + c(5, 3, 10) = 150 + 150 = 300$
 - $A \cdot (B \cdot C)$: $c(10, 3, 10) + c(5, 10, 10) = 300 + 500 = 800$
- Total cost is sensitive to the order in which we multiply the matrices!
→ We want to compute an optimal order!



^aModels $\approx$ #arithmetic operations of trivial multiplication.
Using fast rectangular matrix multiplication other cost functions, e.g. $c(r, s, t) = \min(\max(r, s, t)^\omega, r \cdot s \cdot t)$ are conceivable (and also work with our algorithm).

Problem: Matrix chain multiplication

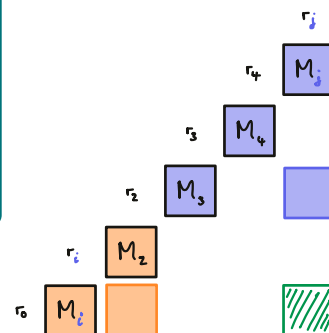
- Input:** Dimensions $(r_0, \dots, r_k)$ of a sequence $(M_1, \dots, M_k)$ of $r_{i-1} \times r_i$ -matrices M_i .
- Output:** An evaluation order so that the *total cost* of the multiplication $M_1 M_2 \dots M_k$ is minimized.

slide 6.25

Step 1: Formalization + recursive solution

Define: $COST(i, j) :=$ optimal total cost for the chain multiplication $M_i \cdot \dots \cdot M_j$.

Goal: Compute $COST(1, k)$.



$$COST(i, j) = \begin{cases} 0 & \text{if } i \geq j \\ c(r_{i-1}, r_i, r_{i+1}) & \text{if } i = j - 1 \\ \min \{ COST(i, \ell) + COST(\ell + 1, j) + c(r_{i-1}, r_\ell, r_j) : i \leq \ell < j \} & \text{otherwise} \end{cases}$$

slide 6.26

Step 2: Memoization, Dependencies, evaluation order

- Memoization: Store subresults in 2-dim array $Cost[1..k, 1..k]$. Base cases: $Cost[i, i] = 0$ and $Cost[i, i + 1] = r_{i-1} \cdot r_i \cdot r_{i+1}$
- Dependencies: $Cost[i, j]$ uses $Cost[i, \ell]$ and $Cost[\ell + 1, j]$ for all $i \leq \ell < j$
- Evaluation Order: **for** $i = k - 2 \dots 1$: **for** $j = i + 2 \dots k$: compute $Cost[i, j]$

Step 3: Algorithm and analysis

- This algorithm computes optimal *cost* in $O(k^3)$.
- Exercise: generate *arithmetic expression with brackets* for optimal order in $O(k^3)$.

slide 6.27

Input: Dimensions $r_0, \dots, r_k$.

Output: Cost of an optimal multiplication order.

```

1: procedure MATRIXCHAINMULTIPLICATIONCOST( $r_0, \dots, r_k$ )
2:   for  $i \leftarrow 1 \dots k$  do  $Cost[i, i] = 0$ 
3:   for  $i \leftarrow 1 \dots k - 1$  do  $Cost[i, i + 1] = r_{i-1} \cdot r_i \cdot r_{i+1}$ 
4:   for  $i = k - 2 \dots 1$  do
5:     for  $j = i + 2 \dots k$  do
6:        $opt \leftarrow \infty$ 
7:       for  $\ell = i \dots j - 1$  do
8:          $c \leftarrow Cost[i, \ell] + Cost[\ell + 1, j] + r_{i-1} \cdot r_\ell \cdot r_j$ 
9:         if  $c < opt$  then  $opt \leftarrow c$ 
10:       $Cost[i, j] \leftarrow opt$ 
11:   return  $Cost[1, k]$ 

```

6.6 Maximum-Independent-Set on Trees

Independent Set

slide 6.28

Definition 6.2. An independent set in an (undirected) graph $G = (V, E)$ is a subset $U \subseteq V$ s. t. $U^2 \cap E = \emptyset$.

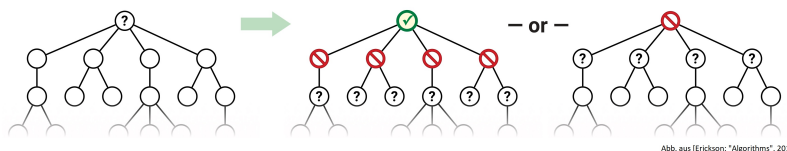
- The *Maximum Independent Set Problem* (IS) is to determine the size of the largest independent set in a graph.
- Trying all possible subsets is an $O(2^n)$ algorithm.
- It is widely believed that there is no polynomial time algorithm.
- However, when G is a tree, we can design an efficient algorithms using dynamic programming.

Let $G = (V, E)$ be a tree, i. e., an acyclic undirected graph.

- We choose an arbitrary vertex in G as root and consider the *directed rooted tree* $T(V, E)$ (arcs point from parents to children).
- For $v \in V$ let T_v be the subtree with root v .

Maximum Independent Set on Trees

slide 6.29



$\maxIS(v)$ denotes size of largest independent set in T_v . Goal: compute $\maxIS(r)$ for root r .

$$\maxIS(v) = \begin{cases} 1, & \text{if } v \text{ is a leaf,} \\ \max \left\{ \sum_{w \in N(v)} \maxIS(w), 1 + \sum_{w \in N(v)} \sum_{u \in N(w)} \maxIS(u) \right\}, & \text{else.} \end{cases}$$

slide 6.30

6 DYNAMIC PROGRAMMING

Memoisation: Store result of $\text{maxIS}(v)$ for each node v as attribute $v.\text{maxIS}$.

Evaluation order: Childs first (post-order traversal).

```
1: procedure TREEMAXIS( $v$ )  
2:   for  $w \in N(v)$  do TREEMAXIS( $w$ )       $\triangleright$  Compute  $u.\text{maxIS}$  for all  $u \in V(T_v) \setminus \{v\}$ .  
3:    $skip \leftarrow 0$   
4:    $keep \leftarrow 1$   
5:   for  $w \in N(v)$  do  
6:      $skip \leftarrow skip + w.\text{maxIS}$   
7:     for  $u \in N(w)$  do  
8:        $keep \leftarrow keep + u.\text{maxIS}$   
9:    $v.\text{maxIS} \leftarrow \max\{skip, keep\}$ 
```

- Runtime $O(n)$.

Part III

Greedy Algorithms

7 Greedy

slide 7.1

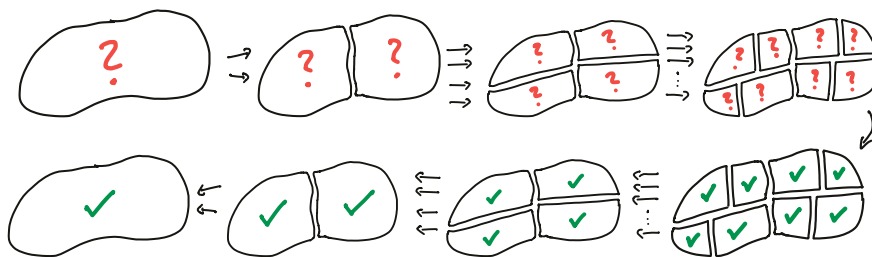
7.1 Introduction

Design techniques for algorithms

slide 7.2

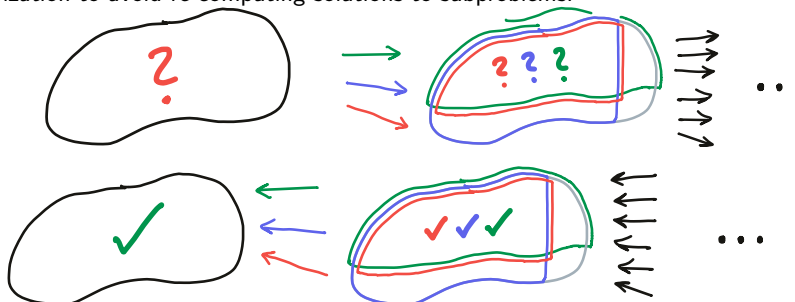
Divide-and-Conquer

- Recursively decompose instance into smaller subinstances until instance size is small enough.
- Solve tiny (constant size) instances directly.
- Combine solutions of smaller instances into a solution for the larger instance.



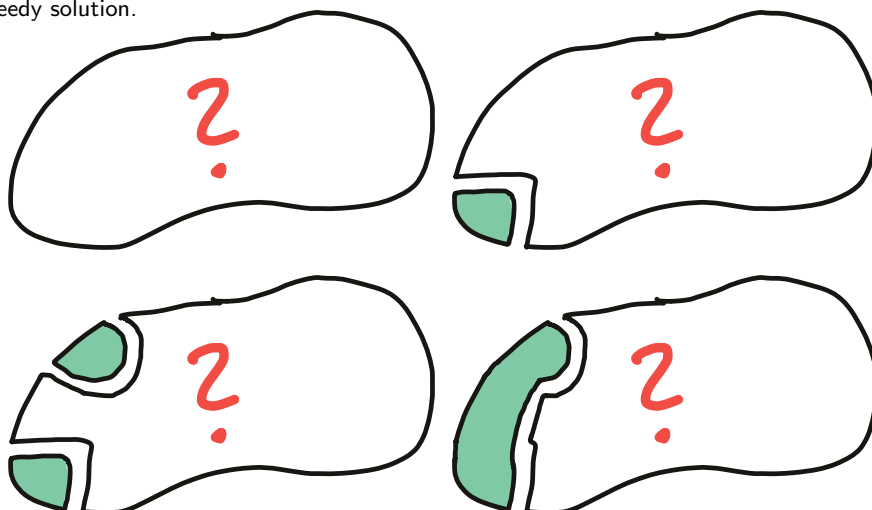
Dynamic Programming

- Solve the problem by recursively solving (overlapping!) subproblems.
- Use memoization to avoid re-computing solutions to subproblems.



Greedy Algorithms

- Construct a solution step-by-step by always adding the locally best choice to the solution.
- Leads to *recursion without branching* and is often very fast.
- WARNING: Few problems are "so easy" to solve! You always have to prove the (global) optimality of the greedy solution.



slide 7.3

Greedy Algorithms

Greedy Algorithms

- ➔ Warm-up: Check-out at the supermarket
- ➔ Warm-up: Magnetic tapes
- ➔ Scheduling
- ➔ Huffman coding

slide 7.4

Warm-up: Check-out at the supermarket

- At the supermarket, a new checkout lane opens and n customers arrive at the same time.
- Because they are buying different items, checkout will take a different time for each.
- In which order should the customers queue up to *minimize the average individual wait time*?
- Let t_i be the checkout time of customer i . Let $\sigma: [n] \rightarrow [n]$ (position $\rightarrow$ customer) an ordering of customers.
- Then the total wait time of the customer at position p is $w(p) = \sum_{j=1}^p t_{\sigma(j)}$.
- The average wait time is therefore: [5em] $\frac{1}{n} \sum_{p=1}^n w(p) = \frac{1}{n} \sum_{p=1}^n \sum_{j=1}^p t_{\sigma(j)}$.

Greedy strategy? Sort the customers by their checkout time (in increasing order).

Correctness?

$$\begin{aligned} \min_{\sigma} \frac{1}{n} \sum_{p=1}^n \sum_{j=1}^p t_{\sigma(j)} &= \frac{1}{n} \min_{\sigma} \sum_{p=1}^n (n+1-p) t_{\sigma(p)} \\ &= \frac{1}{n} \min_{\sigma} \left[\underbrace{(n) \cdot t_{\sigma(1)}}_{p=1} + \underbrace{(n-1) \cdot t_{\sigma(2)}}_{p=2} + \dots + \underbrace{(1) \cdot t_{\sigma(n)}}_{p=n} \right] \end{aligned}$$

Assume, $t_{\sigma(p)} > t_{\sigma(p+1)}$. New order: Swap p and $(p+1)$! Then the contribution of these positions to the sum changes by $t_{\sigma(p+1)} - t_{\sigma(p)} < 0$. [5pt] By swapping customers that are in the “wrong” order we can decrease the average wait time!

slide 7.5

Warm-up 2: Magnet tapes access times

- n files i of length ℓ_i are stored sequentially on a magnetic tape.
- Access is by linear scan from the start of the tape to the end of the file.
- Access frequencies of file i in a certain time span is h_i .
- Goal: Sort files in such a way that the total access time is minimised.
- For the ordering $\sigma: [n] \rightarrow [n]$ (position $\rightarrow$ file) the total access cost is $\text{TAC}(\sigma) = \sum_{k=1}^n h_{\sigma(k)} \sum_{j=1}^k \ell_{\sigma(j)}$.

Greedy strategy: Sort the files in increasing order by ℓ_i/h_i .

slide 7.6

Correctness of Magnet tapes access times

Obviously an optimal solution exists since there are only finitely many orderings and each of them has a well-defined value.

Assume towards a contradiction that $\sigma: [n] \rightarrow [n]$ is an optimal ordering that does not satisfy $\frac{\ell_{\sigma(i)}}{h_{\sigma(i)}} \leq \frac{\ell_{\sigma(i+1)}}{h_{\sigma(i+1)}}$ f.a. $1 \leq i \leq n-1$.

Then there exists a l with $\frac{\ell_{\sigma(l)}}{h_{\sigma(l)}} > \frac{\ell_{\sigma(l+1)}}{h_{\sigma(l+1)}}$.

7 GREEDY

We look at the ordering σ' with:

$$\sigma'(i) := \begin{cases} \sigma(l+1) & , \text{ if } i = l \\ \sigma(l) & , \text{ if } i = l+1 \\ \sigma(i) & , \text{ else} \end{cases}$$

(The files on positions l and $l+1$ were swapped.)

It holds that:

slide 7.7

$$\begin{aligned} \text{TAC}(\sigma') &= \text{TAC}(\sigma) - (h_{\sigma(l)} \cdot \ell_{\sigma(l)} + h_{\sigma(l+1)} \cdot \ell_{\sigma(l)} + h_{\sigma(l+1)} \cdot \ell_{\sigma(l+1)}) \\ &\quad + (h_{\sigma'(l)} \cdot \ell_{\sigma'(l)} + h_{\sigma'(l+1)} \cdot \ell_{\sigma'(l)} + h_{\sigma'(l+1)} \cdot \ell_{\sigma'(l+1)}) \\ &= \text{TAC}(\sigma) - (h_{\sigma(l)} \cdot \ell_{\sigma(l)} + h_{\sigma(l+1)} \cdot \ell_{\sigma(l)} + h_{\sigma(l+1)} \cdot \ell_{\sigma(l+1)}) \\ &\quad + (h_{\sigma(l+1)} \cdot \ell_{\sigma(l+1)} + h_{\sigma(l)} \cdot \ell_{\sigma(l+1)} + h_{\sigma(l)} \cdot \ell_{\sigma(l)}) \\ &= \text{TAC}(\sigma) + h_{\sigma(l)} \cdot \ell_{\sigma(l+1)} - h_{\sigma(l+1)} \cdot \ell_{\sigma(l)} \end{aligned}$$

Since $\frac{\ell_{\sigma(l)}}{h_{\sigma(l)}} > \frac{\ell_{\sigma(l+1)}}{h_{\sigma(l+1)}}$ by choice of l we obtain:

$$\begin{aligned} \frac{\ell_{\sigma(l+1)}}{h_{\sigma(l+1)}} &< \frac{\ell_{\sigma(l)}}{h_{\sigma(l)}} \\ \Leftrightarrow h_{\sigma(l)} \cdot \ell_{\sigma(l+1)} &< h_{\sigma(l+1)} \cdot \ell_{\sigma(l)} \\ \Leftrightarrow h_{\sigma(l)} \cdot \ell_{\sigma(l+1)} - h_{\sigma(l+1)} \cdot \ell_{\sigma(l)} &< 0 \end{aligned}$$

and thus $\text{TAC}(\sigma') < \text{TAC}(\sigma)$ which contradicts the optimality of σ . $\square$

7.2 Scheduling

Event Scheduling

slide 7.8

Problem: Event Scheduling

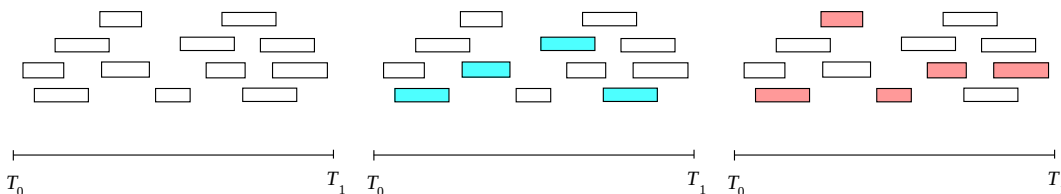
Input: Global time span T_0, T_1 and n pairs (s_i, f_i) of start and finish times such that $T_0 \leq s_i < f_i \leq T_1$ for all $1 \leq i \leq n$.

Output: A set $A \subseteq \{1, \dots, n\}$ of maximum cardinality such that the time intervals $[s_i, f_i)$ for $i \in A$ do not overlap.

$A \subseteq \{1, \dots, n\}$ is *feasible*, if all $[s_i, f_i)$, $i \in A$, are disjoint.

An example

slide 7.9



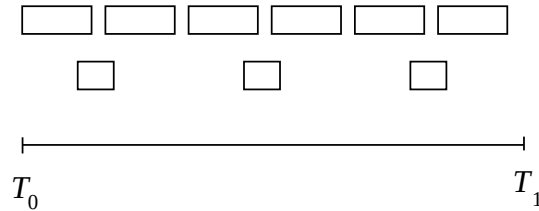
7 GREEDY

A feasible but not extendable solution (cyan) and a better feasible solution (red).

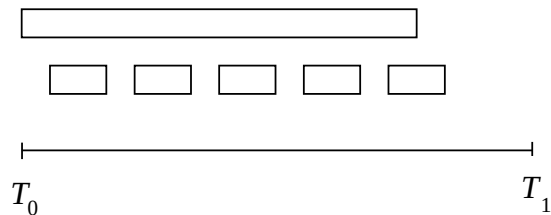
slide 7.10

Approaches that do not work

Short events first:



Events with earliest starting time first:



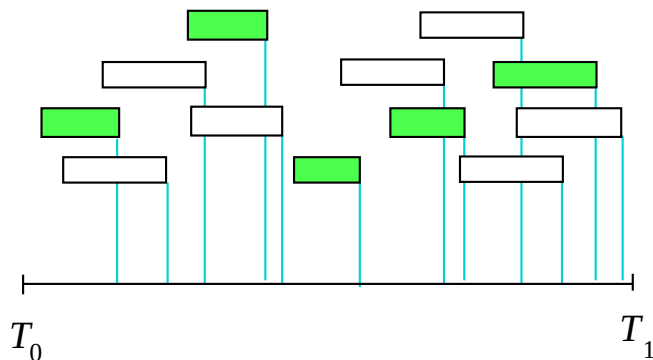
slide 7.11

Greedy strategy

Trick: Choose events in the order of their **increasing finish times**.

An event is *selectable*, if its start time is at least as large as the last finish time of an already selected event.

Greedy strategy: Choose selectable event with smallest finish time and add it to the plan.



slide 7.12

Algorithm Greedy Scheduling (GS)

Input: Real numbers $T_0 < T_1$, pairs $(s_1, f_1), \dots, (s_n, f_n)$, with

$T_0 \leq s_i < f_i \leq T_1$ for $1 \leq i \leq n$

Output: Maximum-cardinality $A \subseteq \{1, \dots, n\}$ s. t. $[s_i, f_i), i \in A$ are disjoint

- (1) Sort pairs according to $f_1, \dots, f_n$, renumber s. t. $f_1 \leq \dots \leq f_n$;
- (2) $A \leftarrow \{1\}$;
- (3) $f_{\text{last}} \leftarrow f_1$;
- (4) **for** i **from** 2 **to** n **do**
- (5) **if** $s_i \geq f_{\text{last}}$ **then** // $[s_i, f_i)$ selectable
- (6) $A \leftarrow A \cup \{i\}$;
- (7) $f_{\text{last}} \leftarrow f_i$;
- (8) **return** A

runtime: $O(n \log n)$ for sorting + $O(n)$ for the actual greedy choices.

Theorem 7.1. Greedy Scheduling computes in $O(n \log n)$ a largest feasible solution.

7.3 Huffman-Codes

slide 7.13

Problem: Minimum cost prefix code

Input: A finite alphabet A ($|A| \geq 2$) and relative frequencies $p: A \rightarrow [0, 1]$ with $\sum_{a \in A} p(a) = 1$.

Output: A prefix code $c: A \rightarrow \{0, 1\}^*$ for A with minimum cost $\text{cost}_A(c)$.

Definition 7.2. A binary *prefix code* for an alphabet A is a mapping $\text{code}: A \rightarrow \{0, 1\}^*$ such that:

- For all $a, b \in A$ with $a \neq b$:
code(a) is no prefix of code(b).

Why are prefix codes useful? 🖥 blackboard

Definition 7.3. The *cost* of a binary code c for an alphabet A with relative frequencies $p: A \rightarrow [0, 1]$ is

$$\text{cost}_A(c) := \sum_{a \in A} p(a) \cdot |\text{code}(a)|$$

Code trees

slide 7.14

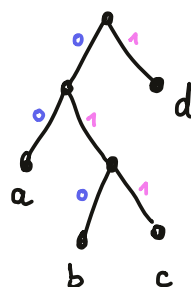
Definition 7.4. A *code tree* for A is a binary tree T , where

- every edge to the left (right) child is labelled with 0 (1);
- for every $a \in A$ there is exactly one leaf in T .

The code $\text{code}_T(a)$ for a string a is obtained by sequence of edge labels on the unique path from the root to leaf with label a in T .

It is easy to see that each prefix code corresponds to exactly one code tree. **Example:**

A	p	Code
a	0.2	00
b	0.1	010
c	0.4	011
d	0.3	1



7 GREEDY

$$\begin{aligned}
 \text{cost}_A(T) &:= \text{cost}_A(\text{code}_T) \\
 &= \sum_{a \in A} p(a) \cdot |\text{code}_T(a)| \\
 &= (0.2 \cdot 2) + (0.1 \cdot 3) + (0.4 \cdot 3) + (0.3 \cdot 1) \\
 &= 2.2
 \end{aligned}$$

slide 7.15

Example of a code tree

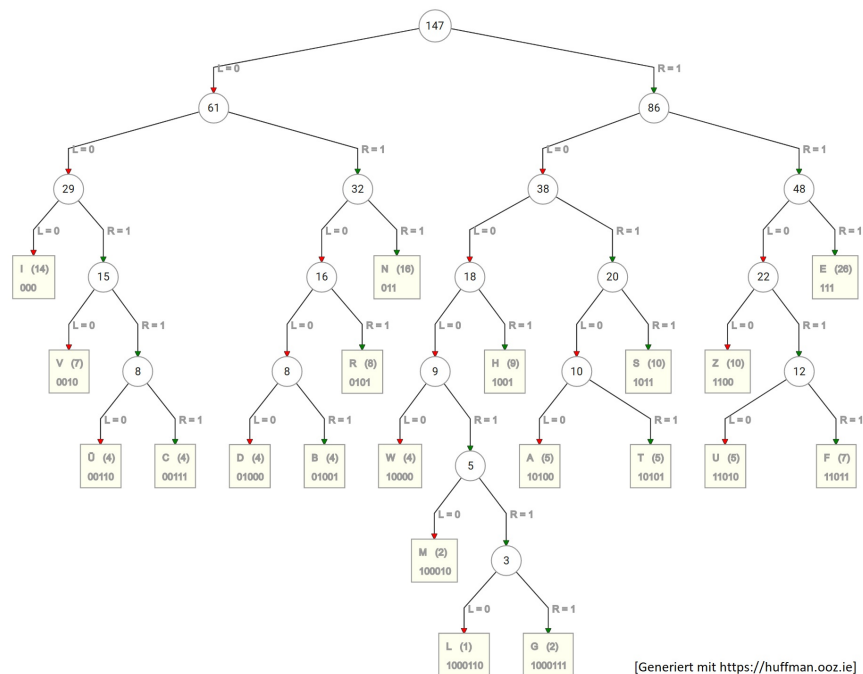
Example 7.5 (An example text [Matthias Belz 2014, autograms.net]). Dieser Satz besteht aus fünf A, vier B, vier C, vier D, sechszwanzig E, sieben F, zwei G, neun H, vierzehn I, einem L, zwei M, sechzehn N, acht R, zehn S, fünf T, fünf U, sieben V, vier W, zehn Z und vier Ü.

Based on the frequencies of the 20 different letters in the text, there is a prefix code with cost $587/147 \approx 3.99$.

When ignoring space, capitalization, and punctuation we can encode the sentence (a string of 147 letters) with 587 bits.²

An alternative encoding with a fixed-length bit strings would require 5 bits per letter (as $16 < 20 \leq 32$) and hence $5 \cdot 147 = 735$ bits.

slide 7.16



slide 7.17

Optimal Code Trees & Greedy Strategy

²I. e. we encode the string
 DIESERSATZBESTEHTAUSFÜNFAVIERBVIERCVIDERDSECHSUNDZWANZIGESIEBENFZWEIGNEUNHVERZEHN
 EINEMLZWEIMSECHZEHNACHTRZEHNSENFÜNFTFÜNFUSIEBENVVIERWZEHNZUNDVIERÜ

Definition 7.6. A prefix code c is *optimal*, if its cost is minimal.
A code tree T is *optimal*, if the corresponding prefix code code_T is optimal.

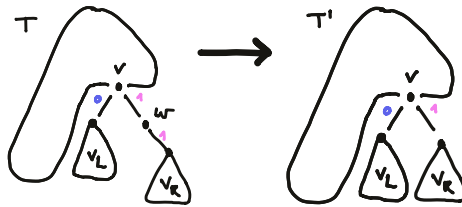
Reminder:

$$\text{cost}_A(T) := \sum_{a \in A} p(a) \cdot |\text{code}_T(a)|$$

Lemma 7.7. There is always an optimal code tree, in which each inner node has two children.

Proof:

- Assume w has only one child.



- Improve code by removing w :

$$\begin{aligned} \text{cost}_A(T) &= \sum_{a \in A} p(a) \cdot |\text{code}_T(a)| \\ \text{cost}_A(T') &= \text{cost}_A(T) + \sum_{a \in V_R} p(a) \cdot (-1) \end{aligned}$$

(The codes for all letters in V_R become shorter by 1.)

Optimal Code Trees & Greedy Strategy

slide 7.18

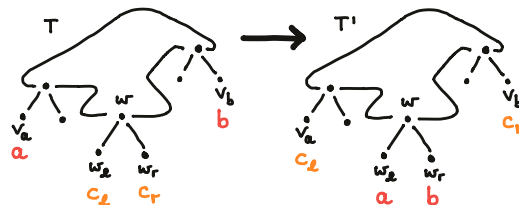
Lemma 7.8. There is always an optimal code tree, in which the two least frequent letters $a, b \in A$ are siblings.

By the “two least frequent letters” we mean $a, b \in A$ with $a \neq b$, such that for all letters $x \in A \setminus \{a, b\}$:

- $p(a) \leq p(b) \leq p(x)$

Proof:

- Assume a, b are not siblings in an optimal code tree T .
- Find a deepest inner node w . (Then: Both children of w are leaves.)



- Also: $d(v_a)$ and $d(v_b)$ are $\leq d(w_L) = d(w_R)$.
- Swap to get another optimal tree:

7 GREEDY

$$\begin{aligned}
 \text{cost}_A(T) &= \dots + p(a)d(v_a) + p(b)d(v_b) + p(c_\ell)d(w_\ell) + p(c_r)d(w_r) \\
 \text{cost}_A(T') &= \dots + p(c_\ell)d(v_a) + p(c_r)d(v_b) + p(a)d(w_\ell) + p(b)d(w_r) \\
 \text{cost}_A(T') - \text{cost}_A(T) & \quad (\geq 0 \text{ because } T \text{ optimal!}) \\
 &= p(c_\ell)d(v_a) + p(c_r)d(v_b) + p(a)d(w_\ell) + p(b)d(w_r) \\
 &\quad - p(a)d(v_a) - p(b)d(v_b) - p(c_\ell)d(w_\ell) - p(c_r)d(w_r) \\
 &= \underbrace{[p(c_\ell) - p(a)] \cdot d(v_a)}_{\geq 0} + \underbrace{[p(c_r) - p(b)] \cdot d(v_b)}_{\geq 0} \\
 &\quad + \underbrace{[p(a) - p(c_\ell)] \cdot d(w_\ell)}_{\leq 0} + \underbrace{[p(b) - p(c_r)] \cdot d(w_r)}_{\leq 0} \leq 0
 \end{aligned}$$

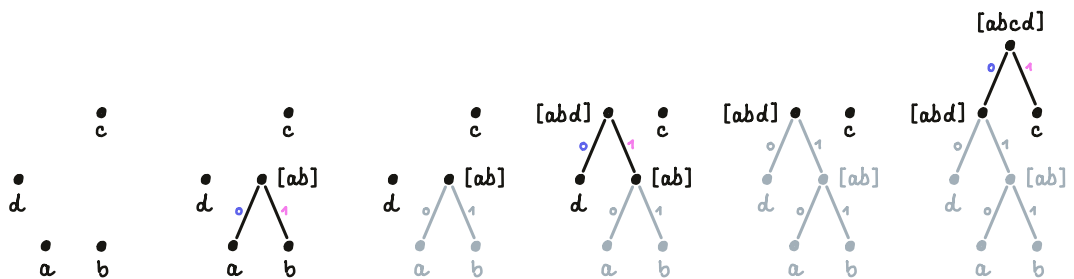
slide 7.19

Optimal Code Trees & Greedy Strategy

Greedy strategy: Make the two least frequent letters a, b into siblings and treat their new parent node as a new letter $[ab]$ with frequency $p([ab]) = p(a) + p(b)$. **Example:**

A	p	Code
	0.2	0 1 [ab] 0
	0.1	0 1 [ab] 1
	0.4	[abcd] 1
	0.3	0 [abd] 0

A	p	Code
	0.3	0 [abd] 1
	0.6	[abcd] 0
	1.0	ε



slide 7.20

Optimal Code Trees & Greedy Strategy

Greedy strategy: Make the two least frequent letters a, b into siblings and treat their new parent node as a new letter $[ab]$ with frequency $p([ab]) = p(a) + p(b)$.

Theorem 7.9 (Correctness). For each alphabet A and each $p: A \rightarrow [0, 1]$ the resulting code tree is optimal.

Proof: Induction over number of letters $|A|$. Assume the Theorem holds for $\leq n - 1$.

- $A' = \{[ab], c_3, \dots, c_n\}$

7 GREEDY

- T' optimal for A' :

- Then T'_o tree for A' : with $\text{cost}(T'_o) < \text{cost}(T') \rightarrow \text{Contradiction!}$
- $A = \{a, b, c_3, \dots, c_n\}$

- We get a tree T for A :

- Better tree for A with $\text{cost}(T_o) < \text{cost}(T)$?

Huffman coding: implementation

slide 7.21

For each tree node i (except root):

- $p[i]$ stores frequency value of node i
- $\text{pred}[i]$ stores parent node of i
- $\text{mark}[i]$ stores edge label to parent node (0 or 1)

Initialization:

- $p[1 \dots n]$ contains frequencies of letters $a_1, \dots, a_n$ sorted in increasing order. $p(a_1) \leq \dots \leq p(a_n)$. These are the leafs of the tree.
- Init empty $p[n+1 \dots 2n-1]$ for storing "frequencies" of inner nodes (corresponding to newly generated "letters"). $p[2n-1]$ corresponds to the root node (in the end: $p[2n-1] = 1$).
- Init empty $\text{pred}[1 \dots 2n-2]$ and $\text{mark}[1 \dots 2n-2]$.

Loop:

- Keep track of smallest leaf and smallest generated inner node that have no parent yet.
- In each round "generate" new inner node and attach two smallest (leaf or inner) nodes, assign edge labels (mark) and compute frequency of new inner node.

Huffman coding: implementation

slide 7.22

- 1: **procedure** *HUFFMAN*($p[1 \dots n]$) $\triangleright p$ is sorted in increasing order
- 2: Init empty arrays $p[n+1 \dots 2n-1]$, $\text{pred}[1 \dots 2n-2]$, $\text{mark}[1 \dots 2n-2]$
- 3: $\text{pred}[1] \leftarrow n+1$; $\text{pred}[2] \leftarrow n+1$; $\text{mark}[1] \leftarrow 0$; $\text{mark}[2] \leftarrow 1$
- 4: $p[n+1] \leftarrow p[1] + p[2]$
- 5: $k \leftarrow 3$ $\triangleright$ smallest non-processed leaf

8 MINIMUM SPANNING TREES

```

6:    $h \leftarrow n + 1$   $\triangleright$  smallest non-processed inner node
7:   for  $b = n + 2, \dots, 2n - 1$  do
    // compute (in  $i$  and  $j$ ) position of smallest letters:
8:     if  $k \leq n$  and  $p[k] \leq p[h]$  then  $i \leftarrow k$ ;  $k \leftarrow k + 1$  else  $i \leftarrow h$ ;  $h \leftarrow h + 1$ 
9:     if  $k \leq n$  and ( $h = b$  or  $p[k] \leq p[h]$ ) then  $j \leftarrow k$ ;  $k \leftarrow k + 1$  else  $j \leftarrow h$ ;
     $h \leftarrow h + 1$ 
    // attach  $i$  and  $j$  to new parent  $b$ 
10:     $\text{pred}[i] \leftarrow b$ ;  $\text{pred}[j] \leftarrow b$ 
11:     $\text{mark}[i] \leftarrow 0$ ;  $\text{mark}[j] \leftarrow 1$ 
12:     $p[b] \leftarrow p[i] + p[j]$ 
13:  return  $\text{pred}[1 \dots 2n - 2]$ ,  $\text{mark}[1 \dots 2n - 2]$ 

```

slide 7.23

Example run of Huffman's algorithm

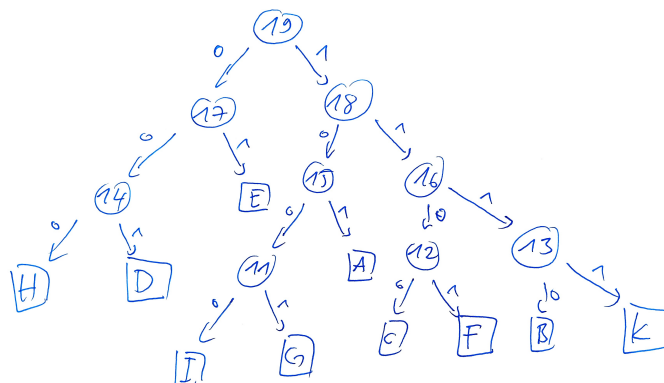
a_i	A	B	C	D	E	F	G	H	I	K
p_i	0,15	0,08	0,07	0,10	0,21	0,08	0,07	0,09	0,06	0,09
a_i	I	G	C	F	B	K	H	D	A	E
p_i	0,06	0,07	0,07	0,08	0,08	0,09	0,09	0,10	0,15	0,21
a_i	I	G	C	F	B	K	H	D	A	E
i	1	2	3	4	5	6	7	8	9	10
p_i	0,06	0,07	0,07	0,08	0,08	0,09	0,09	0,10	0,15	0,21
pred	11	11	12	12	13	13	14	14	15	17
mark	0	1	0	1	0	1	0	1	1	1

i	11	12	13	14	15	16	17	18	19
p_i	0,13	0,15	0,17	0,19	0,28	0,32	0,40	0,60	1,00
pred	15	16	16	17	18	18	19	19	—
mark	0	0	1	0	0	1	0	1	—

slide 7.24

Example run of Huffman's algorithm

Resulting optimal code tree T and optimal prefix code $c: A \rightarrow \{0, 1\}^*$:



	A	B	C	D	E	F	G	H	I	K
$p(a)$	0,15	0,08	0,07	0,10	0,21	0,08	0,07	0,09	0,06	0,09
$c(a)$	101	1110	1100	001	01	1101	1001	000	1000	1111

Cost: $B(T, p) = \sum_{a \in A} p(a) \cdot |c(a)| = 0,21 \cdot 2 + (0,1 + 0,09 + 0,15) \cdot 3 + 0,45 \cdot 4 = 3,24$.

slide 8.1

8 Minimum Spanning Trees

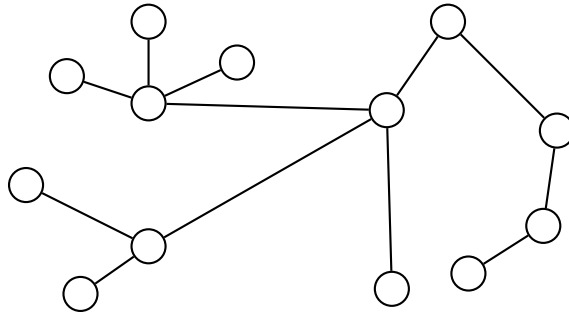
slide 8.2

Basics

8 MINIMUM SPANNING TREES

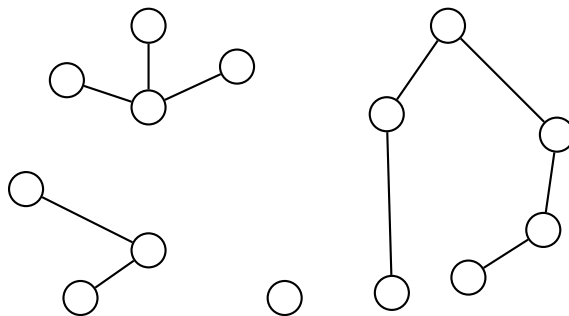
Reminder

- An undirected graph $G = (V, E)$ is called *acyclic* if there is no cycle in G .
- A graph G is called a (*free*) *tree* if it is connected and acyclic.



Basics

slide 8.3



Acyclic graphs are also called (*free*) *forests*.

Basics

slide 8.4

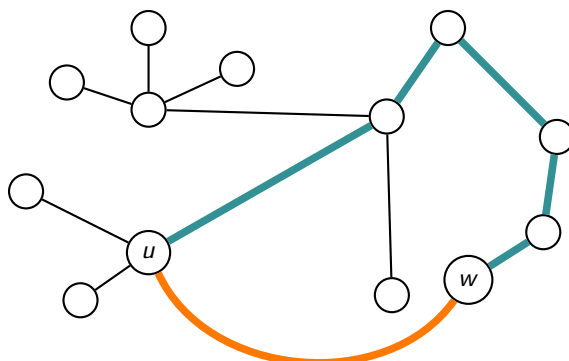
Fundamental facts about trees If $G = (V, E)$ is a tree with n nodes and m edges, the following statements hold:

1. $m = n - 1$.
2. For each pair u, v of nodes there is exactly one simple path from u to v .

Furthermore we have: If $G = (V, E)$ is a graph with $n - 1$ edges and it is acyclic, it is a tree.
If $G = (V, E)$ is a graph with $n - 1$ edges and it is connected, it is a tree.

slide 8.5

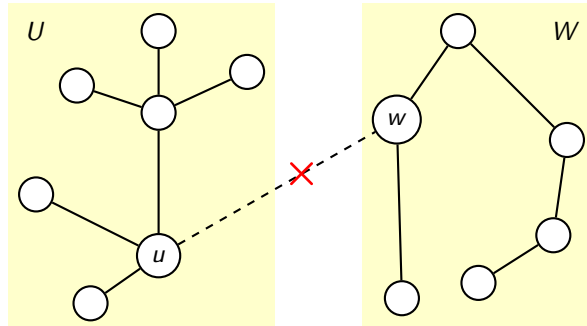
1. Adding an edge (u, w) to a tree G creates exactly one cycle (consisting of (u, w) and the unique **path** from u to w in G).



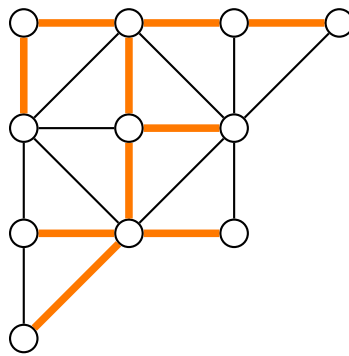
8 MINIMUM SPANNING TREES

slide 8.6

2. Removing an edge (u, w) from G makes the graph split in 2 components: $U = \{v \in V \mid v \text{ reachable from } u \text{ via edges in } E - \{(u, w)\}\}$; $W = \{v \in V \mid v \text{ reachable from } w \text{ via edges in } E - \{(u, w)\}\}$;

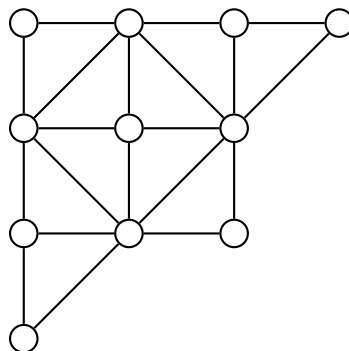


slide 8.7



Definition 8.1. For $G = (V, E)$ a connected graph a set $T \subseteq E$ of edges is called a *spanning tree* for G if (V, T) is a tree.

slide 8.8

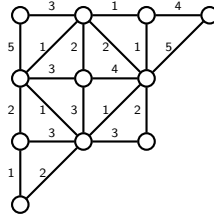


Observe: Every connected graph has a spanning tree. (Start with E . While there is a cycle, remove some edge from some cycle. At some point: no cycle is left. Taking away a cycle edge never destroys connectedness, so the final result is connected and acyclic: a tree.)

slide 8.9

Weighted Graphs

Definition 8.2. $G = (V, E, c)$ is a **weighted graph**, i.e. (V, E) is a graph and $c : E \rightarrow \mathbb{R}$ is a “weight function” or “cost function”.

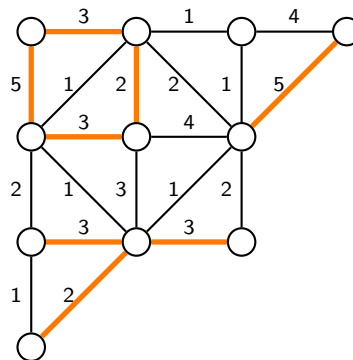


Weighted graphs model: road networks - computer networks - electric power networks...

Edge costs model: building cost - cost for leasing cable use - cost for leasing equipment for transmitting data via radio waves... Btw: Multiply edge weights by “million Euros”.

(a) The **(total) weight** of a subset $E' \subseteq E$ of edges is defined as

$$c(E') := \sum_{e \in E'} c(e)$$



slide 8.10

Total weight

$$c(E') = 3 + 5 + 2 + 5 + 3 + 3 + 3 + 2 = 26.$$

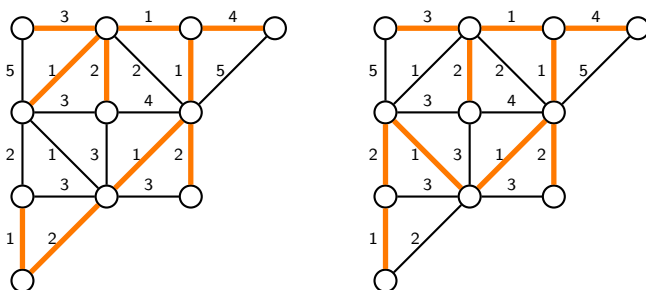
Minimum spanning tree

slide 8.11

(b) Let G be a connected graph. A spanning tree $T \subseteq E$ for G is called a *minimum spanning tree (MST)* for G if

$$c(T) = \min\{c(T') \mid T' \text{ spanning tree for } G\},$$

i.e. if $c(T)$ is minimal among all spanning trees of G .



Two MSTs, both with total weight 18.

8.1 Jarník/Prim algorithm

Jarník/Prim algorithm

slide 8.12

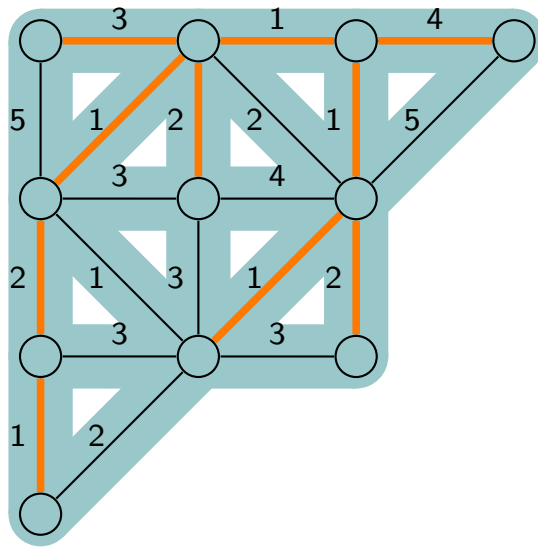
S : Set of nodes “reached so far”. R : Set of nodes “chosen so far”.

```

1: procedure SIMPLEJARNÍKPRIM
2:   choose an arbitrary start node  $s \in V$ 
3:    $S \leftarrow \{s\}; R \leftarrow \emptyset$ 
4:   for  $i$  from 1 to  $n - 1$  do
5:     Find  $w \in S$  and  $u \in V \setminus S$  such that
6:        $c(w, u)$  is minimal among all  $\{c(w', u') \mid w' \in S, u' \in V \setminus S\}$ 
7:      $S \leftarrow S \cup \{u\}$ 
8:      $R \leftarrow R \cup \{(w, u)\}$ 
9:   return  $R$ 

```

slide 8.13

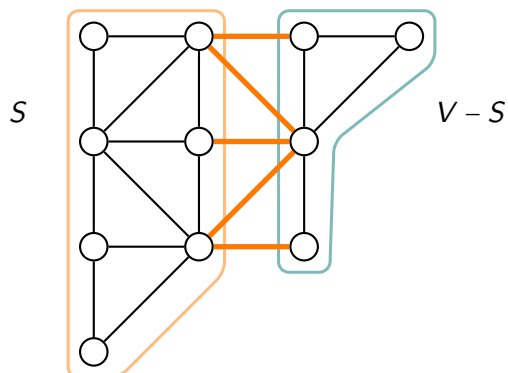
Example**8.2 The cut property**

slide 8.14

The cut property

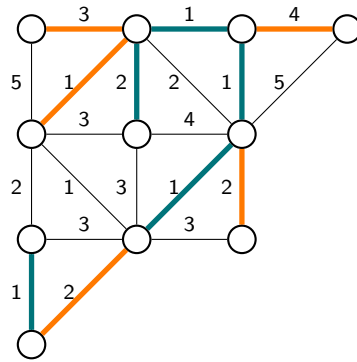
For proving the algorithm of Jarník/Prim correct we use the “*cut property*”.

A partition $(S, V \setminus S)$ of V with $\emptyset \neq S \neq V$ is called a **cut**.

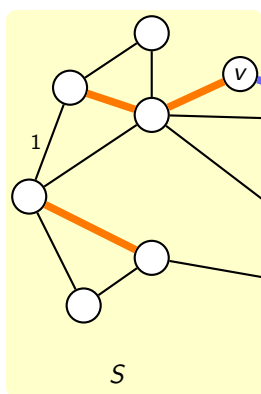


Definition 8.3. A set $R \subseteq E$ is called **extendible** (to an MST), if there is an MST T s.t. $R \subseteq T$.

8 MINIMUM SPANNING TREES



R is extendible, because ... there is an MST $T \supseteq R$.



Claim (Cut Property):

Assume $R \subseteq E$ is extendible and $(S, V \setminus S)$ is a cut, s.t. there is no edge in R from a node in S to a node in $V \setminus S$, and assume $e = \{v, w\}$, $v \in S$, $w \in V \setminus S$ is an edge that minimizes $c(\{v', w'\})$, $v' \in S$, $w' \in V \setminus S$.

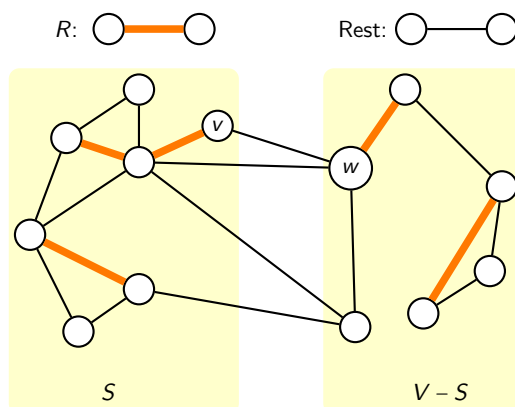
Then $R \cup \{e\}$ is also extendible.

slide 8.15

Proof of the cut property:

slide 8.16

- Let $R \subseteq E$, $T \supseteq R$ be an MST, $(S, V \setminus S)$ a cut, e as assumed.
- Case 1:** If $e \in T$, we have $R \cup \{e\} \subseteq T$, hence $R \cup \{e\}$ is extendible.



An extendible set R .

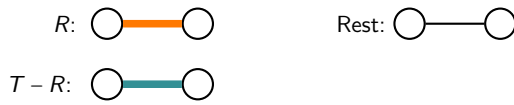
8 MINIMUM SPANNING TREES

slide 8.17

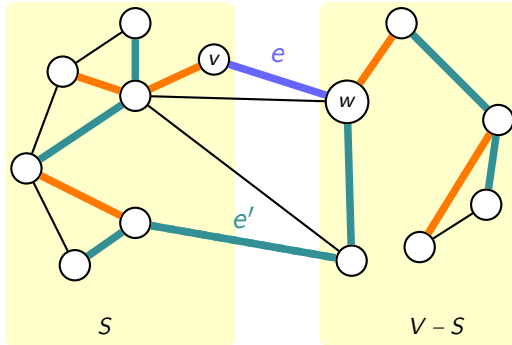
Case 2:

$e = \{v, w\} \notin T$ minimizes

$c(\{v', w'\})$, $v' \in S$, $w' \in V \setminus S$



- Path from v to w in T must change from S to $V \setminus S$ at some edge e'
- We obtain a cycle in $T \cup \{e\}$ with e and e' on it
- $T_e := (T - \{e'\}) \cup \{e\}$ is a spanning tree
- $c(T_e) - c(T) = c(e) - c(e') \leq 0$, hence T_e is also optimal
- hence $R \cup \{e\}$ is extendible. $\square$



slide 8.18

Correctness of the Jarník/Prim algorithm

- R_i : Edge set (size i), after round i in R .
- S_i : Node set (size $i + 1$), after round i in S .
- Since in every round an edge and a node is added to a connected graph, creating no cycles, every graph (S_i, R_i) is a tree.
- Since R_{n-1} is a tree with $n - 1$ edges, R_{n-1} is a spanning tree.
- Must show: Minimality, i.e. R_{n-1} is an MST for G .

slide 8.19

Inductive claim IC(i): R_i is extendible. (This is easily proved by induction on $i = 0, 1, \dots, n - 1$, with the cut property.) Then IC($n - 1$) says that $T \supseteq R_{n-1}$ for some MST T . Since $|T| = n - 1 = |R_{n-1}|$, we get $T = R_{n-1}$, hence R_{n-1} is an MST.

Missing: Details of the implementation. There is a great similarity with Dijkstra's algorithm. (We use a priority queue, for $w \notin S$ the value $\text{dist}[w]$ is the length of an edge (v, w) with $v \in S$ that minimizes $c((v, w))$.)

Theorem 8.1. *The Jarník/Prim algorithm can be implemented using a priority queue, realized as a binary heap. Then it finds a minimum spanning tree for any given weighted connected graph $G = (V, E, c)$ in time $O(m \log n)$, or $O(|E| \log |V|)$.*

slide 8.20

JarníkPrim - full version with priority queue

```

1: procedure JARNÍKPRIM( $G = (V, E, c), s$ )
2:   initialize priority queue  $PQ \leftarrow \emptyset$ 
3:   for  $w \in V$  do
4:      $\text{dist}[w] \leftarrow \infty$ ;  $\text{inS}[w] \leftarrow \text{false}$ ;  $p[w] \leftarrow -1$ 
5:    $\text{dist}[s] \leftarrow 0$ ;  $p[s] \leftarrow -2$ ;  $PQ.\text{insert}(s)$ 
6:   while  $PQ$  is not empty do
7:      $u \leftarrow PQ.\text{extractMin}()$ ;  $\text{inS}[u] \leftarrow \text{true}$ 
8:     for each vertex  $w$  with  $(u, w) \in E$  and  $\text{inS}[w] = \text{false}$  do
9:        $dd \leftarrow c(u, w)$ 
10:      if  $p[w] \geq 0$  and  $dd < \text{dist}[w]$  then
11:         $\text{dist}[w] \leftarrow dd$ ;  $p[w] \leftarrow u$ ;  $PQ.\text{decreaseKey}(w, dd)$ 

```

```

12:     else if  $p[w] = -1$  then
13:          $\text{dist}[w] \leftarrow dd$ ;  $p[w] \leftarrow u$ ;  $PQ.\text{insert}(w, dd)$ 
14:     return  $\{(w, p[w]) \mid \text{inS}[w] = \text{true}, w \neq s\}$ 

```

8.3 Kruskal's algorithm

This algorithm also solves the *MST problem*.

slide 8.21

We use a different method than Jarník/Prim, but also "greedy":

Start with $R = \emptyset$. Then do $n - 1$ rounds.

In each round:

Choose an edge $e \in E \setminus R$ of minimum weight that does not close a cycle with (V, R) , and add e to R .

It is clear that one can organize this as follows:

Scan the edges in increasing order of their weight. Add e to R if and only if e does not close a cycle with the current R .

Kruskal's algorithm

slide 8.22

Step 1:

Sort edges $e_1, \dots, e_m$ according to their weights $c(e_1), \dots, c(e_m)$ aufstein increasing order, and re-label.

Afterwards: $c(e_1) \leq \dots \leq c(e_m)$.

Step 2: Set $R \leftarrow \emptyset$.

Step 3: For $i = 1, 2, \dots, m$ do:

if $R \cup \{e_i\}$ is acyclic, then $R \leftarrow R \cup \{e_i\}$

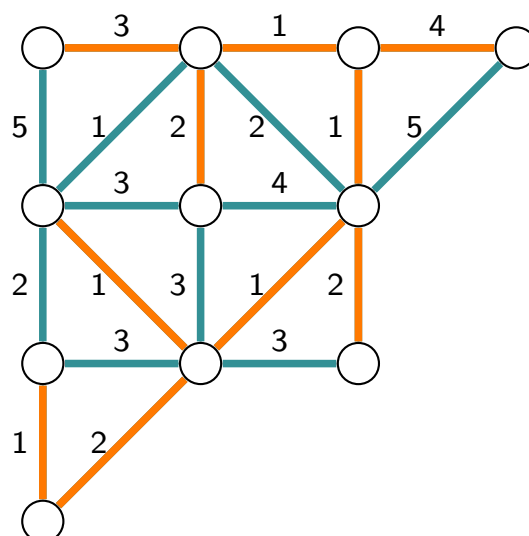
// else, i.e. if e_i closes a cycle, R does not change

// Optional: End loop as soon as $|R| = n - 1$.

Step 4: Output R .

Example for Kruskal's algorithm

slide 8.23



Step-by-step execution of the algorithm Slides with animations.

8.3.1 Correctness of Kruskal's algorithm

slide 8.24

R_i : Edge set in R after treating e_i .

One shows by induction on $i = 0, \dots, m$: R_i is extendible. (Not hard with the cut property.) Then $R_m \subseteq T$ for an MST T . But R_m is also connected, hence it is a tree, hence $R_m = T$.

Induction Claim IC(i): R_i is extendible. Proof: **Basis:** $R_0 = \emptyset$ is extendible (since there are MSTs). **I.H.:** $1 \leq i \leq m$ and R_{i-1} is extendible. **I.S.:** We execute round i with edge e_i .

Case 1: $R_{i-1} \cup \{e_i\}$ has a cycle. Then $R_{i-1} = R_i$ is extendible.

slide 8.25

Case 2: $R_{i-1} \cup \{e_i\}$ is acyclic. Let $e_i = (v, w)$. Define $S :=$ Connected component of v in (V, R_{i-1}) . Then obviously no edge in R_{i-1} connects S and $V \setminus S$. Since $R_{i-1} \cup \{e_i\}$ is acyclic, we have $w \in V \setminus S$. Easy: $c(e_i)$ is minimal among all $c(e')$ with $e' = (v', w')$, $v' \in S$, $w' \in V \setminus S$. By the cut property we get: $R_i = R_{i-1} \cup \{e_i\}$ is extendible. $\square$

IC(m) says that $R_m \subseteq T$ for some MST T . But we also have $T \subseteq R_m$. (Let $e \in T$. Then $e = e_i$ for some i and e is tested in round i . Now $R_{i-1} \subseteq R_m \subseteq T$ and $e_i \in T$, hence $R_{i-1} \cup \{e_i\} \subseteq T$, hence $R_{i-1} \cup \{e_i\}$ is acyclic, hence the algorithm puts e_i into R_i , hence $e = e_i \in R_m$.) So $R_m = T$, and R_m is an MST.

slide 8.26

Computation time of Kruskal's algorithm

With a suitable data structure ("Union-Find data structure", implementation with trees, see below) the acyclicity test in Step 3 can be carried out in time $O(\log n)$.

Total time for Kruskal's algorithm:

$$\underbrace{O(m \log m)}_{\text{sorting}} + \underbrace{m \cdot O(\log n)}_{\text{loop}} = O(m \log n)$$

(Details to follow. Note that $n - 1 \leq m < n^2/2$, hence $\log_2 m = \Theta(\log_2 n)$.)

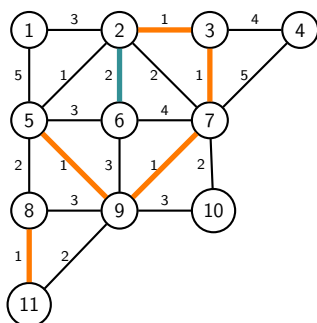
8.4 Auxiliary data structure: Union-Find

slide 8.27

Union-Find data structures are used as an auxiliary structure for several algorithms, in particular for Kruskal's algorithm.

Intermediate situation in Kruskal's algorithm: Set $R = R_{i-1} \subseteq E$, so that (V, R) is a forest. Next edge: $e = e_i = (v, w)$. Must decide whether e closes a cycle with R .

slide 8.28



For two nodes v and w decide whether (V, R) contains a path from v to w .

(Here: $v = 2$, $w = 6$.)

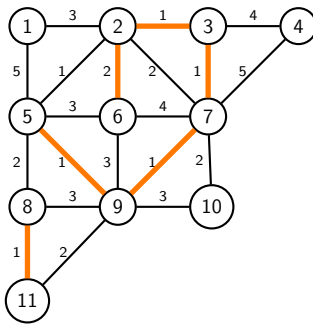
Possible, but slow: do depth-first-search in (V, R) each time.

Clever: We do not need to represent the edges in R , only the *node sets* of the connected components of (V, R) .

In the picture this are the sets $\{1\}$, $\{2, 3, 5, 7, 9\}$, $\{4\}$, $\{6\}$, $\{8, 11\}$, $\{10\}$. We wish to figure out (fast) whether *two nodes are in the same component (set)*. When the algorithm puts a new edge into R , we have to form the *union* of two of the sets (and throw away the two old sets).

💡 Auxiliary data structure: Union-Find

slide 8.29



For two nodes v and w decide whether (V, R) contains a path from v to w .
(Here: $v = 2$, $w = 6$.)
Possible, but slow: do depth-first-search in (V, R) each time.

Clever: We do not need to represent the edges in R , only the *node sets* of the connected components of (V, R) .

In the picture this are the sets $\{1\}$, $\{2, 3, 5, 7, 9\}$, $\{4\}$, $\{6\}$, $\{8, 11\}$, $\{10\}$. We wish to figure out (fast) whether *two nodes are in the same component (set)*. When the algorithm puts a new edge into R , we have to form the *union* of two of the sets (and throw away the two old sets).

New sets: $\{1\}$, $\{2, 3, 5, 6, 7, 9\}$, $\{4\}$, $\{8, 11\}$, $\{10\}$. This operation also should be fast.

A **partition** of $V = \{1, 2, \dots, n\}$ consists of disjoint nonempty subsets of V , whose union is V : slide 8.30

$$V = \{1, 2, \dots, n\} = S_1 \cup S_2 \cup \dots \cup S_\ell,$$

where $S_1, S_2, \dots, S_\ell$ are **disjoint**.

Abstract task:

Maintain a “**dynamic partition**” of the set $\{1, 2, \dots, n\}$ under operations

- init* (Initialization)
- union* (Union of two of the sets)
- find* (“In which set is v ?”).

💡 Data structure Union-Find

slide 8.31

Example: $n = 12$.

In each set S of the partition we have chosen a **representative** $r \in S$. This representative acts as S ’s **name**. We write S_r for the set with representative r .

Compact representation: Function/Array r with $r(i) =$ representative of the set of i :

i	1	2	3	4	5	6	7	8	9	10	11	12
$r(i)$	4	4	7	4	4	6	7	6	7	10	11	7

Operations:

slide 8.32

- init**(n): Given $n \geq 1$, generate the “discrete partition” with the n singleton sets $\{1\}, \{2\}, \dots, \{n\}$.
Thus $S_v = \{v\}$ and $r(v) = v$, for $1 \leq v \leq n$.
- find**(v): Given $v \in \{1, \dots, n\}$, return the representative $r(v)$ of the class $S_{r(v)}$ that currently contains v .
- union**(s, t): The arguments s and t must be representatives of different classes S_s and S_t . The operation removes S_s and S_t from the partition and adds $S_s \cup S_t$ to it. As representative of this new set $S_s \cup S_t$ use s or t .

8 MINIMUM SPANNING TREES

In the example, **union**(4, 10) removes the sets $S_4 = \{1, 2, 4, 5\}$ und $S_{10} = \{10\}$ and adds $S'_{10} = \{1, 2, 4, 5, 10\}$.

slide 8.33

Kruskal's algorithm with Union-Find data structure

Input: Weighted connected graph $G = (V, E, c)$ with $V = \{1, \dots, n\}$.

Step 1: Sort edges $e_1, \dots, e_m$ in increasing order $c_1 = c(e_1), \dots, c_m = c(e_m)$ of weights.

Result: Sorted edge list $(v_1, w_1, c_1), \dots, (v_m, w_m, c_m)$, $c_1 \leq \dots \leq c_m$.

Step 2: initialize Union-Find structure for $\{1, \dots, n\}$.

Step 3: For $i = 1, 2, \dots, m$ do:

$s \leftarrow \text{find}(v_i); \quad t \leftarrow \text{find}(w_i);$

if $s \neq t$ then begin $R \leftarrow R \cup \{e_i\}; \quad \text{union}(s, t)$ end;

// Optional: Quit loop as soon as $|R| = n - 1$.

Step 4: return R .

slide 8.34

Theorem 8.4. 1. Kruskal's algorithm in the implementation just given is correct.
2. The execution time of the algorithm is $O(m \log n)$ if one implements the Union-Find data structure with trees.

Proof:

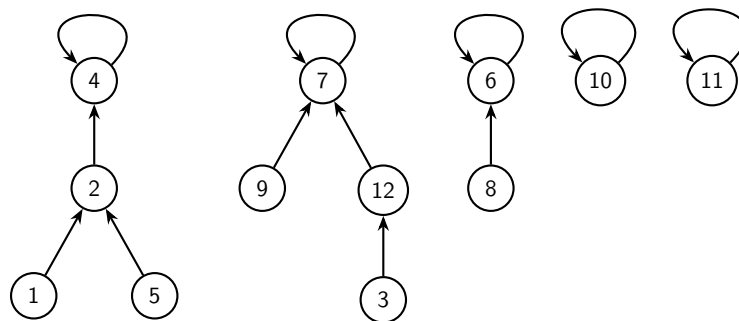
1. (Correctness) One (easily) shows by induction that after i rounds the sets in the union-find structure are the connected components of $(V, \{e_1, \dots, e_i\})$, and these are the same as the connected components of the forest (V, R_i) (R_i = content of R after i rounds).
Hence "' $s \leftarrow \text{find}(v_i); \quad t \leftarrow \text{find}(w_i);$ if $s \neq t \dots$ "' tests whether $e_i = (v_i, w_i)$ closes a cycle with (V, R_{i-1}) .
2. (Execution time): see below.

8.4.1 Tree implementation of Union-Find

slide 8.35

An attractive implementation of the Union-Find data structure uses a forest "'with edges directed towards the roots.'".

Example: Partition $\{1, 2, 4, 5\}, \{3, 7, 9, 12\}, \{6, 8\}, \{10\}, \{11\}$ is represented by:



For each set S_t there is exactly one tree B_t . Each element $v \in S_t$ is a node in tree B_t .

In each tree, all arrows point towards the root: $p(v)$ is the predecessor of v .

The root is the representative r .

The root points to itself as a predecessor: $p(v) = v$ if and only if v is a representative.

8 MINIMUM SPANNING TREES

A very efficient representation of such a forest uses only an integer array $p[1..n]$; For node v the entry $p[v]$ gives the predecessor $p(v)$. slide 8.36

The forest in the example is given by the following array:

	1	2	3	4	5	6	7	8	9	10	11	12
p :	2	4	12	4	2	6	7	6	7	10	11	7

```

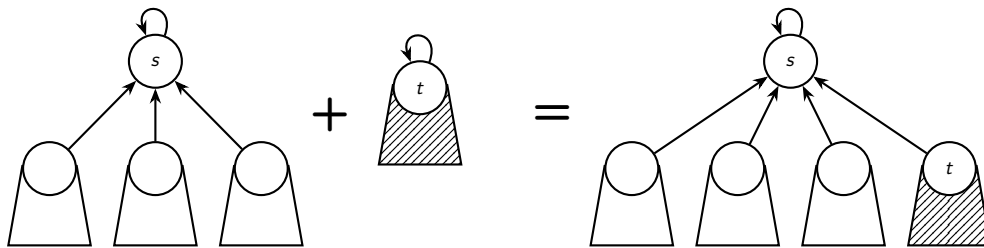
1: procedure FIND( $v$ )
2:    $current \leftarrow v$ 
3:    $parent \leftarrow p[current]$ 
4:   while  $parent \neq current$  do
5:      $current \leftarrow parent$ 
6:      $parent \leftarrow p[current]$ 
7:   return  $current$ 

```

Time: $\Theta(\text{depth}(v)) = \Theta(\text{depth of } v \text{ in its tree})$.

union(s, t): Given are two representatives s and t . slide 8.37

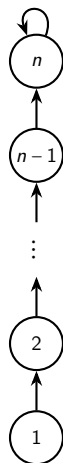
We make one of the representatives the child of the other one, i.e., we let $p(s) := t$ **or** $p(t) := s$.



? Which of the two options is preferable?

Bad choice: Carry out **union**($v, v+1$), $v = 1, \dots, n-1$, by $p(v) := v+1$.

This gives the tree

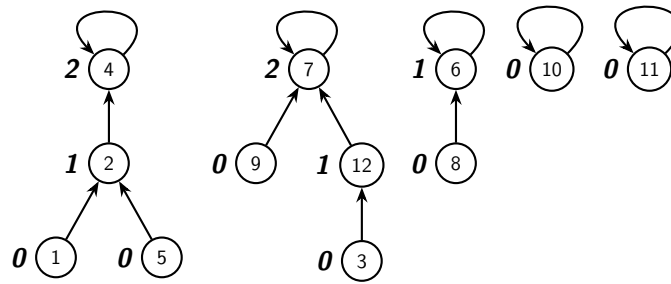


Now **find**-operations are very expensive!

find(v) for each $v \in \{1, \dots, n\}$ gives total cost $\Theta(n^2)$!

Trick: For each node v keep a number $\text{rank}(v)$ (**rank**), in an integer array $\text{rank}[1..n]$ with $\text{rank}[v] = \text{depth of subtree with root } v$. slide 8.39

8 MINIMUM SPANNING TREES

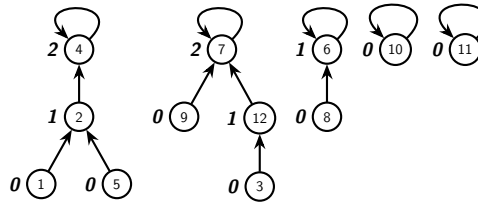


union(s, t) the root with the bigger rank becomes the root of the new, larger tree ("union by rank").

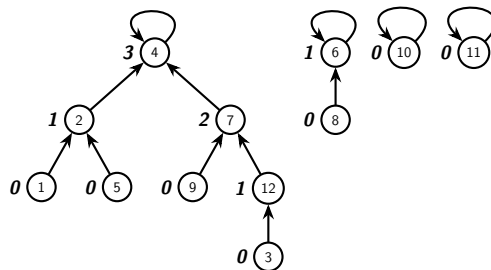
If the ranks are the same it does not matter which node we choose as new root - (precisely) in this case the rank of the node that remains a root increases by 1.

slide 8.40

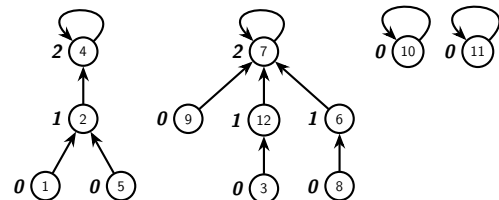
Example:



union(4, 7) would give:



union(6, 7) would give:



slide 8.41

```

1: procedure INIT( $n$ )
2:   Generate  $p$ ,  $rank$ : arrays of length  $n$  for ints
3:   for  $i$  from 1 to  $n$  do
4:      $p[i] \leftarrow i$ ;  $rank[i] \leftarrow 0$ 

```

Cost: $\Theta(n)$.

```

1: procedure UNION( $s, t$ ) ▷ "union by rank"
2:   ▷ Assumption:  $s, t$  are different representatives/roots, i.e.:  $p[s] = s$  and  $p[t] = t$ 
3:   if  $rank[s] > rank[t]$  then  $p[t] \leftarrow s$ 
4:   if  $rank[s] < rank[t]$  then  $p[s] \leftarrow t$ 
5:   if  $rank[s] = rank[t]$  then  $p[t] \leftarrow s$ ;  $rank[s] \leftarrow rank[s] + 1$ 

```

Cost: $O(1)$.

slide 8.42

Theorem 8.5. The implementation of Union-Find just described has the following properties:

1. It is correct (i.e. it has the prescribed I/O behaviour).
2. **init**(n) takes time $\Theta(n)$; **find**(v) takes time $O(\log n)$; **union**(s, t) takes time $O(1)$.

Beweis:

1. is clear.
2. **Claim:** If s is root of the tree B_s and $h = \text{rank}(s)$, then B_s contains at least 2^h nodes.
Proof of claim: A root of rank h is created when two trees are joined whose roots both have rank $h-1$. From this the claim follows by an easy induction over $h = 0, 1, \dots$.
 Since the number of nodes is n , the largest rank h that can occur satisfies $2^h \leq n$, or $h \leq \log_2 n$.
 Since $\text{rank}(s)$ also gives the depth of the tree with root s , no node v can be more than $\log_2 n$ steps away from its root, so $\text{find}(v)$ takes time $O(\log n)$. $\square$

9 Dijkstra's algorithm

slide 9.1

slide 9.2

- Definition 9.1.** 1. A *weighted digraph* is a triple $G = (V, E, c)$, where (V, E) is a digraph and $c: E \rightarrow \mathbb{R}$ is a function.
 $c(v, w)$ can be interpreted as "cost" or "length" or "weight" of edge (v, w) .
2. A directed walk $p = (v_0, v_1, \dots, v_k)$ in G has *cost/length*

$$c(p) := \sum_{i=1}^k c(v_{i-1}, v_i).$$

3. The (*directed*) *distance* of nodes $v, w \in V$ is

$$d(v, w) := \begin{cases} \infty, & \text{if } v \not\rightarrow w, \\ -\infty, & \text{if there are walks from } v \text{ to } w \text{ with costs of arbitrarily large absolute value} \\ \min\{c(p) \mid p \text{ ist Kantenzug von } v \text{ nach } w\}, & \text{else.} \end{cases}$$

slide 9.3

Bemerkung

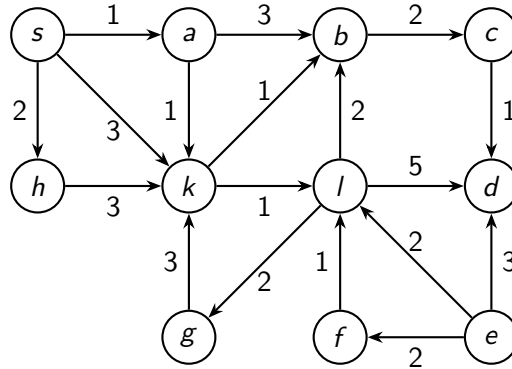
All edge weights are $\geq 0 \Rightarrow d(v, w)$ = minimal length of a **path** from v to w .

(One can take an arbitrary walk from v to w and cut out cycles without increasing the cost.)

Example: Nonnegative edge weights.

slide 9.4

9 DIJKSTRA'S ALGORITHM



$c((s, a, b, c)) = 6$, $c((s, a, k, b, c)) = 5$, $d(s, c) = 5$; $d(s, s) = 0$; $d(s, e) = d(s, f) = \infty$.

slide 9.5

The SSSP-problem

Dijkstra's Algorithm solves the problem

Single-Source-Shortest-Paths

(Shortest paths from one starting node.)

Since Dijkstra's Algorithm requires *nonnegative* edge weights to function correctly, in this course we will restrict ourselves to the following problem:

Problem: SSSP with nonnegative edge weights

Input: Weighted digraph $G = (V, E, c)$ with $c(v, w) \geq 0$ for all $(v, w) \in E$ and start node $s \in V$.

Output: For each $v \in V$ the distance $d(s, v)$ and for all $v \in V \setminus \{s\}$ with $d(s, v) < \infty$ the predecessor $v.parent$ on a shortest s - v -path.

slide 9.6

Dijkstra's Algorithm (1959)



Abb. 1: Edsger W. Dijkstra

Edsger W. Dijkstra (1930–2002)

Dutch computer scientist and pioneer of structured programming

slide 9.7

The (Sparkling) Idea Behind the Algorithm

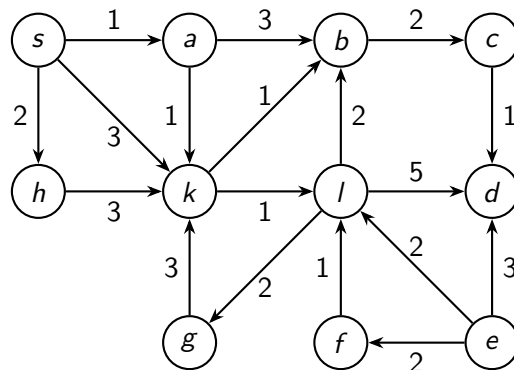
- We think of an edge (v, w) as a "one-way fuse" of length $c(v, w)$.
- Spark advances along a fuse with constant speed 1 per second.
- At time $t_0 = 0$ we ignite node s . All fuses that correspond to edges (s, v) start burning.

9 DIJKSTRA'S ALGORITHM

- The next interesting event:

At time $t = \min \{c(s, v) \mid (s, v) \in E\}$ the spark reaches (at least) one other node v .

slide 9.8



The weighted digraph.

We say: When the spark reaches v , all fuses that belong to edges (v, w) start to burn, without delay. slide 9.9

When a spark reaches v later, on another edge/fuse, nothing happens with v .

Numbers in nodes: When does the fire reach v , by *current information*?

An **orange** edge into a node v that has not been reached indicates from which direction v will be first reached. (These edges must be watched. This information can change.)

"Intuitively clear": The fire reaches v exactly at time $d(s, v)$.

Namely: The time span $[0, d(s, v)]$ is exactly the time in which the fire can walk along the shortest path from s to v , not faster, not slower.

Unfortunate: One cannot really carry out this algorithm. Afterwards the network is reduced to ashes. So we simulate! This gives an algorithm.

Observation: Only the n points in time $d(s, v), v \in V$, are interesting. (If the fire reaches v again after $d(s, v)$ nothing happens.)

Thus our algorithm has at most $n = |V|$ rounds.

In between the sparks walk along the fuses/edges without anything happening.

slide 9.10

W.l.o.g.: $V = \{1, \dots, n\}$.

The algorithm works in up to n rounds, one for each reachable node v .

Data structure:

V is split in two disjoint sets S and $V - S$.

(Nodes in S : already reached; nodes in $V - S$: not yet reached.)

Initialization: $S \leftarrow \emptyset$.

slide 9.11

Array $\text{dist}[1 \dots n]$ stores times.

For $v \in S$: $\text{dist}[v] = d(s, v)$. (Time when fire has reached v .)

For $v \notin S$: $\text{dist}[v]$ = the point in time when fire will reach v

according to the information currently available
 $= \min \{ \text{dist}[w] + c(w, v) \mid w \in S, (w, v) \in E \}.$

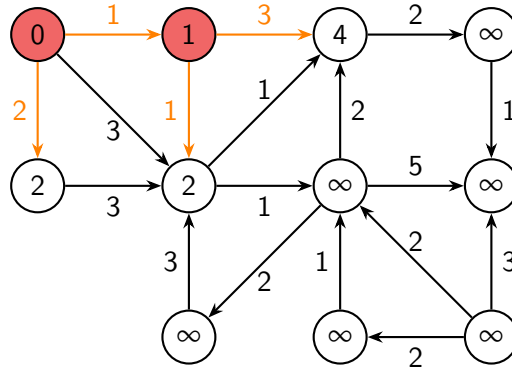
9 DIJKSTRA'S ALGORITHM

(If there is no edge (w, v) with $w \in S$, we have $\text{dist}[v] = \infty$.)

Initialization: $\text{dist}[s] \leftarrow 0$.

For all $v \neq s$: $\text{dist}[v] = \infty$.

Example: Red: S .



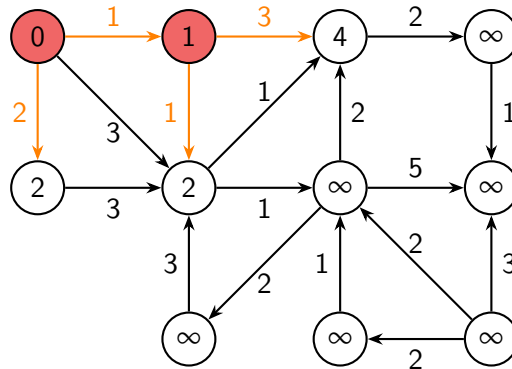
Numbers in the nodes are the $\text{dist}[\cdot]$ values.

Round:

Find some node $u \in V - S$ that minimizes $\text{dist}[v]$, $v \in V - S$ (need not be unique).

Add u to S . (Edges out of u start burning.)

The current value $\text{dist}[u]$ is "frozen" and will not change anymore.



What else has to be done?

On edge (u, v) could be a node $v \in V - S$ be reached that was not reachable before, or $v \in V - S$ can be reached faster.

For such nodes v we must check if $\text{dist}[u] + c(u, v) < \text{dist}[v]$, and if so, update the dist value:

$$\text{dist}[v] \leftarrow \min \{ \text{dist}[v], \text{dist}[u] + c(u, v) \}.$$

(The spark now also walks along the edge (u, v) .)

If $\text{dist}[v]$ switches from ∞ to some finite value, we say that v is *found* in this round.

The algorithm as described so far calculates the **lengths** of the shortest paths from s to all other nodes (The times at which the fire reaches the nodes, or ∞ if unreachable).

Rough Version of Dijkstras Algorithm

9 DIJKSTRA'S ALGORITHM

1: **procedure** *DIJKSTRADISTANCES*(G, s)

Input: weighted digraph $G = (V, E, c)$ with $c(e) \geq 0$, $V = \{1, \dots, n\}$, start node s

Output: lengths of the shortest paths from s to the nodes in G

Data structure: Array $\text{dist}[1 \dots n]$

2: $S \leftarrow \emptyset$;

▷ Initialization

3: $\text{dist}[s] \leftarrow 0$;

4: **for** $v \in V \setminus S$ **do** $\text{dist}[v] \leftarrow \infty$;

5: **while** $\exists u \in V \setminus S : \text{dist}[u] < \infty$ **do**

▷ round in which u is “scanned”

6: $u \leftarrow$ one such node u with minimal $\text{dist}[u]$;

7: $S \leftarrow S \cup \{u\}$;

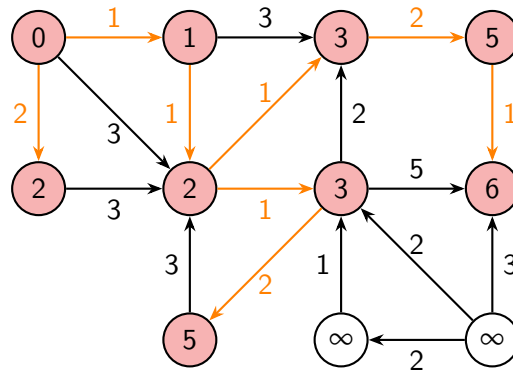
8: **for** $v \in V \setminus S$ with $(u, v) \in E$ **do**

9: $\text{dist}[v] \leftarrow \min\{\text{dist}[v], \text{dist}[u] + c(u, v)\}$;

10: **return** $\text{dist}[1 \dots n]$;

Dijkstra's Algorithm, rough version, in action

slide 9.17



Step by step walkthrough 📄 slides with crossfades.

Correctness of *DIJKSTRADISTANCES*

slide 9.18

Lemma 9.2. Algorithm *DIJKSTRADISTANCES* outputs, in $\text{dist}[1 \dots n]$, the value $d(s, v)$, for all $v \in V$.

Proof:

If node u is considered in lines 6-9, we say that u is *scanned*.

(Actually, the edges out of u are scanned.)

If $\text{dist}[v]$ is set to a value $< \infty$ for the first time, in line 2 or 8, we say that v is *found*.

We first deal with the simple case of the unreachable nodes.

By a simple induction, as in BFS, one can show that every node v that can be reached from s on a path (or walk) will be found at some time and will be scanned at some later time.

Nodes v not reachable from s will never be found, and they keep the value $\text{dist}[v] = \infty$.

Now we show the following invariants, which are valid at the end of each round.

slide 9.19

(I1): $\forall v \in V : \text{dist}[v] < \infty \Rightarrow$ there is a path from s to v of length at most $\text{dist}[v]$

(I2): $\forall v \in S : \text{dist}[v] = d(s, v)$.

9 DIJKSTRA'S ALGORITHM

slide 9.20

We proof **(I1)** by induction over rounds:

Basis: After initialization we have $\text{dist}[v] < \infty$ only for $v = s$, and there is a path from s to s of length 0.

Induction Step: Consider a round in which u is scanned, and an arbitrary node v . If $\text{dist}[v]$ does not change in the round, there is nothing to show. So assume $\text{dist}[v]$ changes. This implies $v \in V \setminus S$.

$\text{dist}[v]$ is changed to the new value $\text{dist}[u] + c(u, v)$.

By induction hypothesis there is a path p_u from s to u of length at most $\text{dist}[u]$. By extending p_u by edge (u, v) we obtain a path from s to v of length at most $\text{dist}[v]$.

slide 9.21

We proof **(I2)** by induction over rounds:

Basis: At the beginning S is empty. In the first round s is put into S , and there is a path of length $\text{dist}[s] = 0$ from s to s (the path with no edge). On the other hand we have $d(s, s) = 0$, since all edges have nonnegative weight, and walks with cycles do not help.

Induction Step: Consider a round in which $u \neq s$ is scanned.

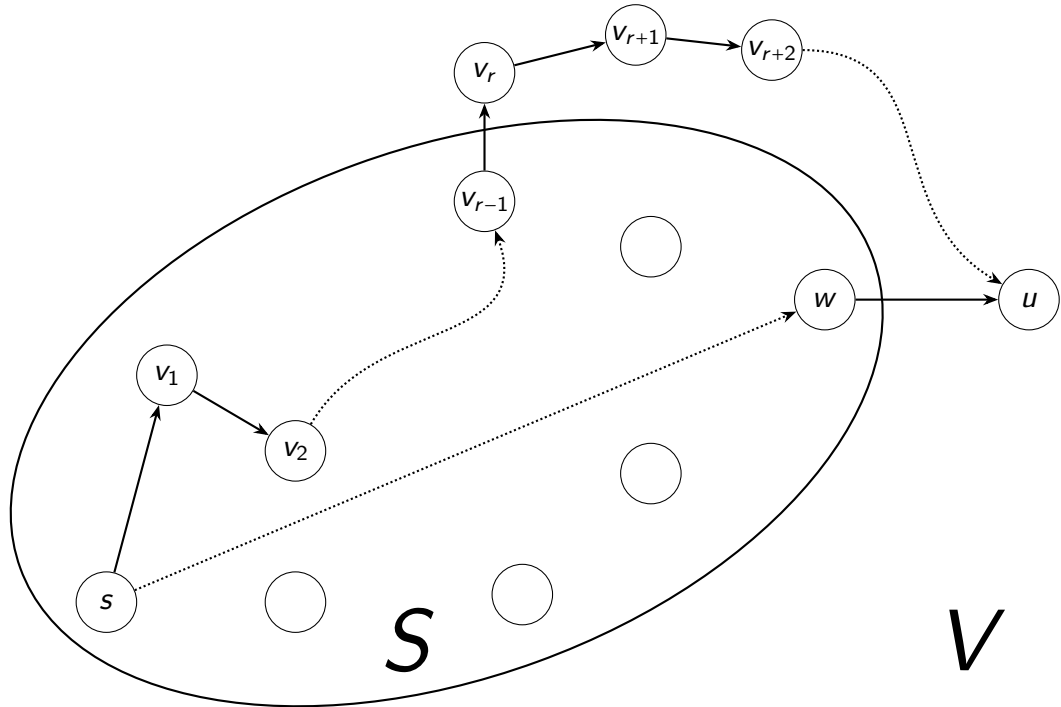
By **(I1)** there is a path from s to u of length at most $\text{dist}[u]$.

We must show:

There is no path from s to u shorter than $\text{dist}[u]$.

Let $p = (s = v_0, v_1, \dots, v_t = u)$ be an arbitrary path from s to u .

Since p starts in $v_0 = s \in S$ and ends in $v_t = u \in V \setminus S$, there is some r such that $s = v_0, v_1, \dots, v_{r-1} \in S$ and $v_r \notin S$.



slide 9.22

slide 9.23

We consider the initial segment $p_{r-1} = (v_0, \dots, v_{r-1})$ of p .

By the induction hypothesis for $v_{r-1} \in S$ we have $d(s, v_{r-1}) = \text{dist}[v_{r-1}]$.

By the definition of the distance function we have $d(s, v_{r-1}) \leq c(p_{r-1})$.

So: $\text{dist}[v_{r-1}] + c(v_{r-1}, v_r) \leq c(p_r)$, for the initial segment $p_r = (v_0, \dots, v_r)$ of p .

By the general assumption all **edge weights** are **nonnegative**.

9 DIJKSTRA'S ALGORITHM

In particular: $c(p) \geq c(p_r)$.

Thus:

$$(7) \quad c(p) \geq \text{dist}[v_{r-1}] + c(v_{r-1}, v_r)$$

Furthermore:

slide 9.24

$$(8) \quad \text{dist}[v_{r-1}] + c(v_{r-1}, v_r) \geq \text{dist}[v_r]$$

Why is this the case? We go back to the round in which v_{r-1} was scanned. In that round the algorithm compared $\text{dist}[v_{r-1}] + c(v_{r-1}, v_r)$ and $\text{dist}[v_r]$, and 8 is enforced. Afterwards $\text{dist}[v_r]$ may change, but it can only decrease.

Finally we observe:

$$(9) \quad \text{dist}[v_{r-1}] + c(v_{r-1}, v_r) \geq \text{dist}[v_r]$$

This is because the algorithm chooses a node in $V \setminus S$ with minimal $\text{dist}[\cdot]$ -value as u , and v_r is qualified for the competition.

Combining 7, 8 and 9 gives $c(p) \geq \text{dist}[u]$. □

Actually, we do not only want to calculate distances $d(s, v)$, but also find *shortest paths* from s to all other nodes. slide 9.25

Idea: For each node v we record the edge (w, v) on which “the spark” has reached node v . If we start in v and walk back step by step according to this “*predecessor*” information we get a shortest path.

Technically, for each node $v \notin S$ that has been found take down $w = p(v) \in S$ with $(w, v) \in E$ and $\text{dist}[v] = \text{dist}[w] + c(w, v)$. Whenever $\text{dist}[v]$ is decreased, update $p(v)$.

Data structure: $p[1 \dots n]$.

Modified Version of Dijkstras Algorithm

slide 9.26

```

1: procedure DIJKSTRATREE( $G, s$ )
   Input: weighted digraph  $G = (V, E, c)$  with  $c(e) \geq 0$ ,  $V = \{1, \dots, n\}$ , start node  $s$ 
   Output: length  $d(s, v)$  of the shortest paths, predecessor  $p(v)$  on shortest path

2:    $S \leftarrow \emptyset$ ;
3:    $\text{dist}[s] \leftarrow 0$ ;  $p[s] \leftarrow -2$ ;                                ▷ special value for root
4:   for  $v \in V \setminus S$  do  $\text{dist}[v] \leftarrow \infty$ ;  $p[v] \leftarrow -1$ ;        ▷ predecessor “undefined”
5:   while  $\exists u \in V \setminus S : \text{dist}[u] < \infty$  do
6:      $u \leftarrow$  one such node  $u$  with minimal  $\text{dist}[u]$ ;
7:      $S \leftarrow S \cup \{u\}$ ;
8:     for  $v \in V \setminus S$  with  $(u, v) \in E$  do
9:        $dd \leftarrow \text{dist}[u] + c(u, v)$ ;
10:      if  $dd < \text{dist}[v]$  then
11:         $\text{dist}[v] \leftarrow dd$ ;
12:         $p[v] \leftarrow u$ ;
13:   return  $\text{dist}[1 \dots n]$  and  $p[1 \dots n]$ ;
```

Correctness of DIJKSTRATREE

slide 9.27

Since exactly the reachable nodes are found we have that $p[v] \neq -1$ (“undefined”) holds at the end if and only if $\text{dist}[v] < \infty$.

$p[v] = -2$ is true only for $v = s$.

9 DIJKSTRA'S ALGORITHM

Definition 9.1. A path $(s = v_0, v_1, \dots, v_t)$ is called an *S-Path* if all nodes excepting maybe v_t are in S .

Claim: In addition to (I1) and (I2) Dijkstra's algorithm maintains the following invariants:

(I3): If $v \in S$ then $p[v] \neq -1$ and if in addition $v \neq s$ then $p[v]$ is the second-to-last node on a path from s to v that runs completely in S and has length $d(s, v)$.

(I4): If $v \notin S$ and $\text{dist}[v] < \infty$ then: $p[v] \in S$ and $p[v]$ the last S -node on an S -path from s to v of shortest length $\text{dist}[v]$.

Proof of (I3) and (I4): Induction on rounds. (Omitted.)

slide 9.28

Result:

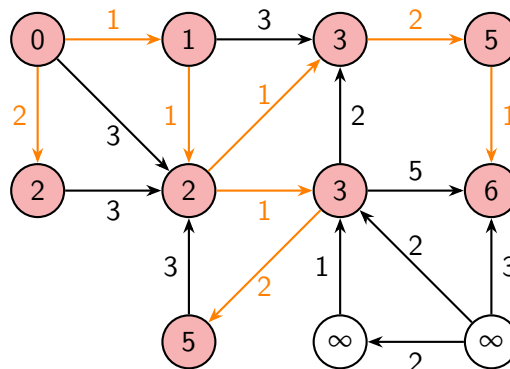
When Dijkstra's algorithm stops, iteratively following the $p[v]$ -pointers starting from w until s is reached will give a shortest path from s to w (in opposite direction).

The $p[v]$ -pointers cannot form a cycle, since in each round a new node u is attached to S , and the edge $(p[u], u)$ is fixed forever, the edges $(p[v], v)$ form a **tree** with root s , the so-called

shortest path tree

slide 9.29

Example: A shortest-path tree.



slide 9.30

Implementation Details

In a round, how do we **efficiently** find u with smallest value $\text{dist}[u]$?

Very simple solution: In each round scan the dist -array to find the node v with minimum value $\text{dist}[v]$ among nodes in $V \setminus S$.

Then each of the up to n round takes time $\Theta(n)$, and the total running time of Dijkstra's algorithm will be $\Theta(n^2)$, quadratic.

For "dense" graphs, meaning graphs with a number of edges close to n^2 , this is acceptable and actually not bad. If, however, we have a graph with $|E| \ll |V|^2$ quadratic running time is not good. One can do better.

slide 9.31

Efficient alternative: A clever data structure.

We maintain the set of vertices $v \in V \setminus S$ with values ("keys") $\text{dist}[v] < \infty$ in a

priority queue PQ.

9 DIJKSTRA'S ALGORITHM

A priority queue (for graph nodes) can be imagined to be a (variable) set of pairs (v, k) , $v \in V$, $k \in \mathbb{R}_0^+$, with the following operations (i.e. methods):

- **init()**: Initializes PQ to the empty set.
- **insert** (v, k) : Inserts a node $v \in V$ plus a key $k \in \mathbb{R}_0^+$
- **extractMin()**: Remove from PQ a pair (u, k) with minimum k , and return node u .
- **isEmpty()**: returns *true* if PQ is empty and *false* if PQ is not empty.
- **decreaseKey** (v, l) : assumes that (v, k) is in PQ, with $l < k$. (Otherwise illegal use of this operation.) Replace (v, k) by (v, l) in PQ.

Fact: One can implement a priority queue for graphs nodes in such a way that

slide 9.32

- **init()** takes time $O(n)$.
- **insert** (v, k) , **extractMin()**, **decreaseKey** (v, l) take time $O(\log n)$.
- **isEmpty()** takes time $O(1)$.

The name of an implementation with these properties is “binary heap”.

Dijkstras Algorithm with PQ

slide 9.33

```

1: procedure DIJKSTRAFULLWITHPQ( $G, s$ )
   Input: weighted digraph  $G = (V, E, c)$  with  $c(e) \geq 0$ ,  $V = \{1, \dots, n\}$ , start node  $s$ 
   Output: length  $d(s, v)$  of the shortest paths, predecessor  $p(v)$  on shortest path
   data structures: PQ: a priority queue for nodes; p, dist: as before; inS[1...n]:
   boolean

2:   for  $v$  from 1 to  $n$  do  $\text{dist}[v] \leftarrow \infty$ ;  $\text{inS}[v] \leftarrow \text{false}$ ;  $p[v] \leftarrow -1$ ;
3:   PQ.init();                                     ▷ set up empty priority queue
4:    $\text{dist}[s] \leftarrow 0$ ;  $p[s] \leftarrow -2$ ; PQ.insert( $s$ );
5:   while not PQ.isEmpty() do
6:      $u \leftarrow \text{PQ.extractMin}()$ ;  $\text{inS}[u] \leftarrow \text{true}$ ;                               ▷ now “scan”  $u$ 
7:     for node  $v$  with  $(u, v) \in E$  and not  $\text{inS}[v]$  do
8:        $dd \leftarrow \text{dist}[u] + c(u, v)$ ;
9:       if  $p[v] \geq 0$  and  $dd < \text{dist}[v]$  then
10:        PQ.decreaseKey( $v, dd$ );  $p[v] \leftarrow u$ ;  $\text{dist}[v] \leftarrow dd$ ;
11:      if  $p[v] = -1$  then                               ▷  $v$  not found before
12:         $\text{dist}[v] \leftarrow dd$ ;  $p[v] \leftarrow u$ ; PQ.insert( $v$ );                               ▷ “find”  $v$ 
13:   return  $\text{dist}[1 \dots n]$  and  $p[1 \dots n]$ .
```

Cost of Dijkstras Algorithm with PQ

slide 9.34

Maximum number of entries in PQ: $n - 1$.

Initialization: $O(n)$.

Let $V' =$ set of reachable nodes, $n' = |V'| \leq m + 1$.

There are $\leq n'$ executions of the **while** loop (1 execution = “scanning” a node).

One loop execution costs time $O(1)$ for loop organization plus $O(\log n)$ for **extractMin** plus the time for looking at the edges out of u . Each edge causes cost $O(\log n)$ (for **insert** or for **decreaseKey**). The number of such edges is $\text{outdeg}(u)$, so the total time for scanning u is $O((1 + \text{outdeg}(u)) \cdot \log n)$.

Initialization plus summing over all reachable u gives time bound $O(n + (n' + m) \log n) = O(n + m \log n)$.

Theorem 9.2. *Dijkstra’s algorithm with a priority queue, implemented as a binary heap, finds shortest paths from start node s in a weighted digraph with nonnegative edge weights in time $O(n + m \log n)$.*

Different Heap Variants

With simple scan to find minimum dist-value: $O(n^2)$.

With binary heap as priority queue: $O(n + m \log n)$.

With “Fibonacci heaps” (which offer cheaper decreaseKey operations): $O(m + n \log n)$.

With “ d -ary heaps”, $d = m/n$: $O(m \cdot \frac{\log n}{\log(m/n)})$.

This is $O(n \log n)$ for $m = O(n)$ and it is $O(m)$ for $m = n^{1+\epsilon}$, for any constant $\epsilon > 0$. This is almost as good as Fibonacci-Heaps.

Part IV

Hard Problems

10 The TSP and NP-completeness

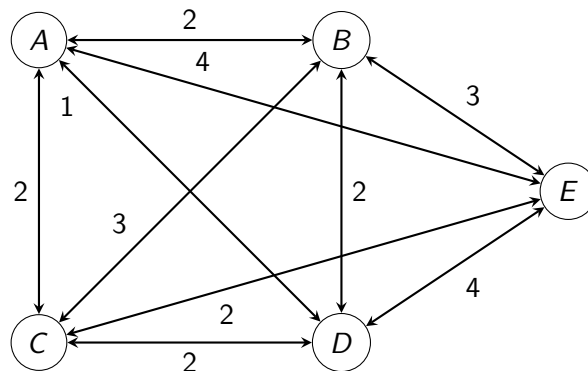
slide 10.1

10.1 Shortest path versus shortest round tour

10.1.1 TSP and the Bellman-Held-Karp Algorithm

Traveling Salesman Problem (TSP)

slide 10.2

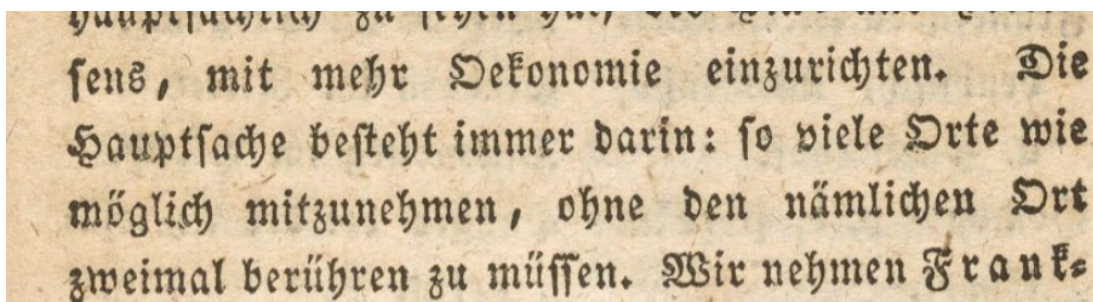


Shortest tour through all cities?: (A, B, E, C, D, A) of length 10.

Traveling Salesman Problem (TSP)

slide 10.3

Problem first mentioned in 1832:



“Die Hauptsache besteht immer darin: so viele Orte wie möglich [.3em] mitzunehmen, ohne den nämlichen Ort zweimal berühren zu müssen.”

Source: <https://nbn-resolving.org/urn:nbn:de:gbv:32-1-10036183634>

slide 10.4



slide 10.5

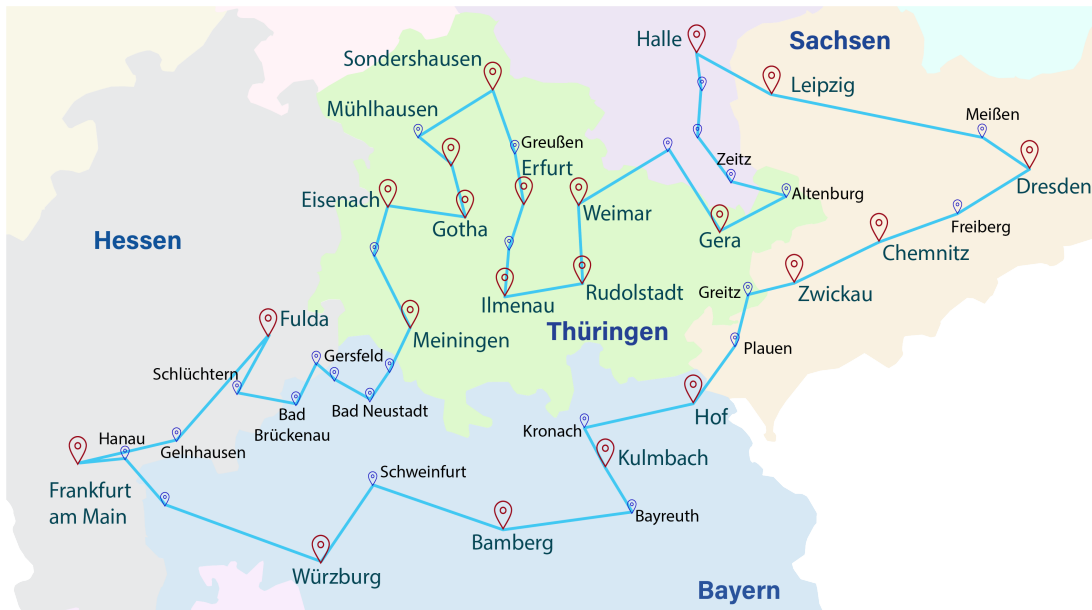
Der Handlungsreisende

The book also includes some tours and claims that they are optimal (but it does not prove optimality).

einige Vorschriften zu geben. Ein Jeder möge so viel davon benutzen, als er es seinem Zwecke für dienlich hält; so viel glauben wir aber davon versichern zu dürfen, daß es nicht wohl thunlich sein wird, die Touren durch Deutschland in Absicht der Entfernungen und, worauf der Reisende hauptsächlich zu sehen hat, des Hin- und Herreisens, mit mehr Dekonomie einzurichten. Die

slide 10.6

A shortest route in the Handlungsreisende book



Traveling Salesman Problem (TSP)

slide 10.7

- A *round tour* (also called Hamiltonian cycle) in a weighted digraph is a directed cycle that contains every vertex exactly once.
- The *weight* of a round tour is the sum of all edge weights on the cycle. (In the following, you may also think “cost” instead of “weight”.)

Problem: Traveling Salesman Problem

Input: A complete weighted digraph $G = (V, E = V^2, c)$ with non-negative edge weights $c: E \rightarrow \mathbb{R}_0^+$.

Output: A round tour of smallest weight.

By trying all possible round tours, the problem can be solved in time $O(n!)$ and space $O(n)$.

Contents

slide 10.8

Hard Problems

- ➔ Definition: Traveling Salesman Problem (TSP)
- ➔ Solving TSP with Dynamic Programming
- ➔ Complexity Theory
 - ➔ Decision Problems
 - ➔ The Class P – Polynomial Time
 - ➔ The Class NP – Non-deterministic Polynomial Time
 - ➔ NP-complete Problems
 - ➔ Proving that TSP is NP-hard

Solving TSP using dynamic programming [Bellman '62; Held and Karp '62]

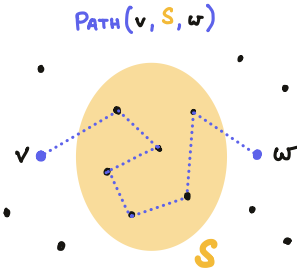
slide 10.9

Step 1: Formalization + recursive solution

10 THE TSP AND NP-COMPLETENESS

For $v, w \in V$ and $S \subseteq V \setminus \{v, w\}$, we **define**:

- $PATH(v, S, w) :=$ Weight of a shortest path that (1) starts in v ,
(2) visits all vertices from $S \cup \{v, w\}$ exactly once
(and no other vertices), and then (3) ends in w .
- Choose an *arbitrary start vertex* v_1 .
- Using the function $PATH$, we can express the *weight of a shortest round tour* as:



$$\textbf{Goal:} \quad \text{TSP result} = \min_{v_n \in V \setminus \{v_1\}} PATH(v_1, V \setminus \{v_1, v_n\}, v_n) + c(v_n, v_1)$$

To answer TSP, we need to compute these $PATH$ -values!

- **Recursive solution:**

$$PATH(v_1, S, w) = \begin{cases} c(v_1, w), & \text{if } S = \emptyset, \\ \min_{u \in S} PATH(v_1, S \setminus \{u\}, u) + c(u, w) & \text{else.} \end{cases}$$

slide 10.10

Step 2: Memoisation, Dependencies, Evaluation Order:

- For all $w \in V$ and all $S \subseteq V \setminus \{v_1, w\}$: Store the value $PATH(v_1, S, w)$ in an *array* $P[S, w]$.
- $PATH(v_1, S, \bullet)$ -values with $|S| = k$ depend on $PATH(v_1, S', \bullet)$ -values with $|S'| = k - 1$.
- Evaluation order: from smaller S to greater S !
- For $k = 0, \dots, n - 2$:
 - Compute $P[S, w]$ for all S, w with $|S| = k$.

slide 10.11

Input: Complete digraph $G = (V, E)$, weights $c: E \rightarrow \mathbb{R}_0^+$.

Output: The weight of a shortest round tour.

- 1: Choose any start vertex $v_1 \in V$.
- 2: **for** $v \in V \setminus \{v_1\}$ **do** $P[\emptyset, w] \leftarrow c(v_1, w)$ ▷ Compute values for $|S| = 0$.
- 3: **for** $k = 1, \dots, n - 2$ **do**
- 4: **for** $S \subseteq V \setminus \{v_1\}$ and $|S| = k$ **do** ▷ Compute values for $|S| = k$.
- 5: **for** $w \notin S \cup \{v_1\}$ **do**
- 6: $P[S, w] \leftarrow \min_{u \in S} P[S \setminus \{u\}, u] + c(u, w)$
- 7: **return** $\min_{v_n \in V \setminus \{v_1\}} P[V \setminus \{v_1, v_n\}, v_n] + c(v_n, v_1)$. ▷ v_n : choice of end vertex

- **Exercise:** Extend algorithm so that it outputs a minimum-weight tour!
- **Runtime:** $O(2^n n^2)$... much better than $O(n!)$ of brute-force approach.
- **Memory consumption:** $O(2^n n)$... much worse than $O(n)$ of brute-force approach.
- As of today no algorithm with runtime $O(1.999...9^n)$ known.

10.2 Theory of NP-completeness

Complexity Theory

- Finding a *shortest path*: Dijkstra with runtime $O(n + m \log n) = O(n^2 \log n)$
- Finding a *round tour*: Algorithm from before with runtime $O(2^n n^2)$

Question: Why is finding a round tour so much harder than finding a shortest path?

slide 10.12

- In this lecture, we often started with a very slow algorithm for a given problem, and then designed faster algorithms.
- What do we do when we tried really hard, but can't find a faster algorithm anymore? Is it because (a) we're stupid or is it because (b) there is no faster algorithm for that problem?

Question: For a given problem, what is the best possible algorithm runtime we can hope for?

These questions lead us to the field of **Complexity Theory**.

- **Complexity Theory** provides tools to classify and compare the difficulty of *computational problems*.
- In this lecture we focus on the theory of *NP-completeness* to distinguish between problems solvable in *polynomial time* and those for which it is commonly believed that no polynomial time algorithm exists.

Decision Problems

slide 10.13

Consider the output of these problems:

problem	output	"codomain"
Finding summer periods	Indices i, j	$\mathbb{N} \times \mathbb{N}$
Sorting	Sorted array	$(\mathbb{N})^n$ (length- n arrays over $\mathbb{N}$)
Max. Independent Set	size of maximum I.S.	$\mathbb{N}$
Traveling Salesman	a shortest route	V^n (length- n arrays over the vertices)

Definition 10.1. • A **Decision Problem** is a problem whose output is either YES or No.

- All other problems are called **Function Problems**.

- Decision problems output only a single bit, they "make a (binary) decision." Function problems "compute a function." (All problems in the table are function problems.)
- **In the following, we will restrict ourselves to decision problems.**
- Even though they can only output one bit (and are therefore very "simple"), they can be computationally quite hard!

Decision Variants of Function Problems

slide 10.14

If we are interested in a function problem, we can derive a decision problem for it:

Problem: Traveling Salesman Problem

Input: A complete weighted digraph $G = (V, E = V^2, c)$ with non-negative edge weights $c: E \rightarrow \mathbb{R}_0^+$.

Output: A round tour of smallest weight.

Problem: Traveling Salesman Decision Problem

Input: A complete weighted digraph $G = (V, E = V^2, c)$ with non-negative edge weights $c: E \rightarrow \mathbb{R}_0^+$ and a number ℓ .

Question: Is there a round tour in G of weight at most ℓ ?

- We call this also the *decision variant of TSP*.
- For many function problems that compute an integer, the decision variant is defined similarly!

Observation: Traveling Salesman Problem can be solved in polynomial time *if and only if* Traveling Salesman Decision Problem can be solved in polynomial time.

When is a problem tractable?

We want to classify problems into (a) those that are “easy” / “tractable” / “efficiently solvable”, and (b) those that are “hard” / “intractable.”

Definition 10.2 (Cobham/Edmonds thesis, 1965). A algorithm is *tractable* only if it runs in **polynomial time**.

A problem is *tractable* only if we can find at least a polynomial-time algorithm for it!

Polynomial time: Runtime in $O(n^c)$, for some constant c ! We also write $O(\text{poly}(n))$.

Definition 10.3. The class **P** (or **PTIME**) is the class of problems that admit a polynomial-time algorithm.

In practice, it is often not that easy:

- We have seen polynomial-time algorithms that are too slow to be useful! (eg. *fast matrix multiplication*, Slide 2.14 – lots of algorithms, but only Strassen's algorithm is used in practice!)
- There are non-polynomial algorithms that *are* useful! (eg. SAT solvers)
- For problems where we can't find an *exact solution* in polynomial time, often we can still find a useful *approximate solution* in polynomial time!

Still, when a computer scientist says “tractable”, they almost always mean the class P!

Comparing Problems: Polynomial time reductions

Definition 10.4 (Karp reduction). A **polynomial time reduction** from a decision problem A to a decision problem B

is a polynomial-time algorithm REDUCTION that

- transforms any instance x of A into an instance y of B , such that
- x is a YES-instance of A *if and only if* y is a YES-instance of B .

We write $A \leq B$ if there exists a polynomial time reduction from A to B .

What does that mean?

- We can use an algorithm for B as a “black box” to solve A ! $\text{ALGORITHMFORA}(x) := \text{ALGORITHMFORB}(\text{REDUCTION}(x))$
- If ALGORITHMFORB runs in $O(\text{poly}(n))$, then ALGORITHMFORA also runs in $O(\text{poly}(n))$!

slide 10.15

slide 10.16

Lemma 10.5. If $A \leq B$ and B is solvable in polynomial time, then A is solvable in polynomial time.

Attention: Whenever you show that $A \leq B$ for some problems A and B , don't get too excited! There are still three options: (1) both $A, B \notin P$ (2) $A \in P$ and $B \notin P$ (3) both $A, B \in P$

Example Reduction: Independent Set $\leq$ Clique

slide 10.17

Problem: Independent Set Decision

Input: A graph $G = (V, E)$
and a number $\ell \leq |V|$.

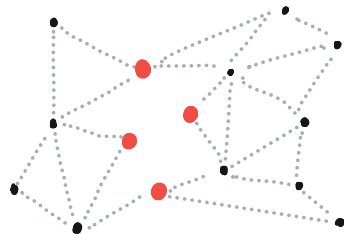
Question: Does G have an I.S. of size $\geq \ell$?

Problem: Clique Decision

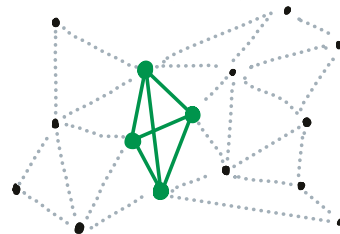
Input: A graph $G = (V, E)$
and a number $\ell \leq |V|$.

Output: Does G have a clique of size $\geq \ell$?

Instance $x = (G, \ell)$:



Instance $y = (G', \ell')$:



$\implies \text{REDUCTION}(G, \ell) := ???(G^C, \ell)$, where G^C is the complement graph of G .

Then we get $\text{ALGORITHMFORIS}(G, \ell) := \text{ALGORITHMFORCLIQUE}(\text{REDUCTION}(G, \ell))!$

Example Reduction: Independent Set $\leq$ Vertex Cover

slide 10.18

Problem: Independent Set Decision

Input: A graph $G = (V, E)$
and a number $\ell \leq |V|$.

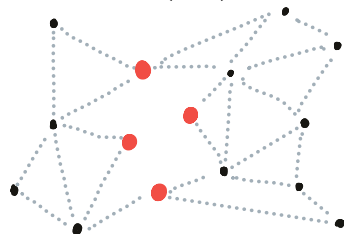
Question: Does G have an I.S. of size $\geq \ell$?

Problem: Vertex Cover Decision

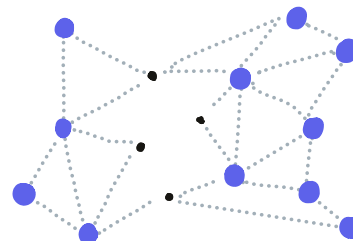
Input: A graph $G = (V, E)$
and a number $\ell \leq |V|$.

Output: Does G have a vertex cover of size $\leq \ell$?

Instance $x = (G, \ell)$:



Instance $y = (G', \ell')$:



$\implies \text{REDUCTION}(G, \ell) := ???(G, |V| - \ell)$

Then we get $\text{ALGORITHMFORIS}(G, \ell) := \text{ALGORITHMFORVERTEXCOVER}(\text{REDUCTION}(G, \ell))!$

The Class NP

slide 10.19

Definition 10.6. The class **NP** is the class of decision problems that can be solved by a *non-deterministic Turing machine* in polynomial time.

It can be shown that this is equivalent to:

Definition 10.7. The class **NP** is the class of decision problems A where

- for each YES-instance x of A , there exists a **polynomial-length witness/proof** w which proves that x is a YES-instance,
- and it can be **verified in polynomial time** that w is a witness for x .

decision problem	witness	VERIFIER(w)
CLIQUE	set of vertices $\{v_1, \dots, v_\ell\}$	Checks all edges between $v_1, \dots, v_\ell$
INDEPENDENT SET	set of vertices $\{v_1, \dots, v_\ell\}$	Checks: no edges between $v_1, \dots, v_\ell$
VERTEX COVER	set of vertices $\{v_1, \dots, v_\ell\}$	Checks: Is every edge incident to one v_i ?
3-SAT	an assignment of the variables	Checks: φ is TRUE under the assignment
TSP	tuple of vertices $(v_1, \dots, v_n)$	Checks: $(v_1, \dots, v_n)$ is tour with weight $\leq \ell$

slide 10.20

The Class NP: Is it tractable?

- We have seen on the last slide: CLIQUE, INDEPENDENT SET, VERTEX COVER, 3-SAT, and TSP are **all in NP**!
- It turns out: A **huge number** of interesting problems are in NP!
- So, the obvious question is: Are the problems in NP tractable? Or, because we said that P is the class of tractable problems:

What is the relation between P and NP?

- Obviously: $P \subseteq NP$. *Proof:* If a problem is in P, we can verify the YES-answer simply by re-computing it! (We don't even need a witness to help us.)

The question that remains:

Is $P \subsetneq NP$ or is $P = NP$?

That is the **P versus NP problem** and it is *the* unsolved problem in computer science!

slide 10.21

NP-complete Problems

Definition 10.8. A problem L is **NP-hard** if all problems $P \in NP$ are $P \leq L$.

Lemma 10.9. If L is NP-hard and L could be solved in polynomial time, then *all* problems in NP could be solved in polynomial time.

(We could use ALGORITHMFORL as a blackbox to solve any problem in NP with the reduction!)

Definition 10.10. A problem L is **NP-complete** if it is NP-hard and also in NP.

Theorem 10.11 (Cook-Levin 1971/73). 3-SAT is NP-complete.

(Without proof.)

The Cook-Levin Theorem tells us: If we find a polynomial-time algorithm for 3-SAT, we directly get polynomial-time algorithms for all problems in NP!

How to prove that a problem is NP-hard

slide 10.22

- Say you would want to show $P = NP$. You would have to show that *all problems in NP* have an algorithm in polynomial time.
- Why are **NP-hard** problems cool? If you have an NP-hard problem L , you “only” need to find *one polynomial-time algorithm* for your problem L , and you directly get polynomial-time algorithms for *all problems in NP*!

Lemma 10.12. If $P_1 \leq P_2$ and $P_2 \leq P_3$, then $P_1 \leq P_3$.

(Proof: Say REDUCTION1 proves $P_1 \leq P_2$ and REDUCTION2 proves $P_2 \leq P_3$. Then you can chain these reductions: REDUCTION2(REDUCTION1(x)) proves $P_1 \leq P_3$!)

Lemma 10.13. If P_2 is NP-hard and $P_2 \leq P_3$, then P_3 is NP-hard.

(Proof: “ P_2 is NP-hard” means that all problems $P_1 \in NP$ are $P_1 \leq P_2$. But because also $P_2 \leq P_3$, it follows that all problems $P_1 \in NP$ are $P_1 \leq P_2 \leq P_3$. Therefore, P_3 is NP-hard!)

$\Rightarrow$ To show that a problem B is NP-hard, you take some NP-hard problem A and show $A \leq B$!

NP-hardness of Independent Set

slide 10.23

- Let’s show NP-hardness for Independent Set!

Theorem 10.14. 3-SAT $\leq$ Independent Set

Proof: Reminder: A 3-SAT formula ϕ is a conjunction of clauses $c_i = (\lambda_{i_1} \vee \lambda_{i_2} \vee \lambda_{i_3})$ of length 3.

We will now construct a graph G_ϕ from ϕ such that ϕ is satisfiable iff. G_ϕ has an Independent set of size k , where k is the number of clauses in ϕ .

Construction: For each clause c in ϕ we add a triangle to the graph G_ϕ . Each node represents a literal λ from c .

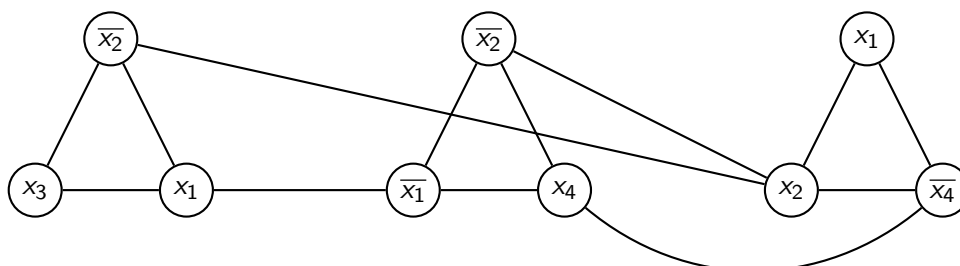
Additionally we add edges between each pair of nodes representing a literal and its negation.

NP-hardness of Independent Set

slide 10.24

Example: $\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4) \wedge (x_1 \vee x_2 \vee \overline{x_4})$

G_ϕ :



slide 10.25

NP-hardness of Independent Set**Correctness:** We need to show:

$$\phi \text{ is satisfiable} \Leftrightarrow G_\phi \text{ has an IS of size } k$$

" $\Rightarrow$ ": ϕ is satisfiable $\Rightarrow$ There is a truth assignment α mapping at least one literal from each clause to true.

To obtain an Independent Set we pick a node from each triangle whose literal is assigned to true.

(If there are multiple pick an arbitrary one)

This yields an Independent set of size k , since we pick only one node from each triangle and never nodes corresponding to x and $\bar{x}$. (since α is truth assignment)

slide 10.26

NP-hardness of Independent Set

" $\Leftarrow$ ": Now we assign all literals corresponding to nodes in the Independent Set to true and all others to false.

An Independent Set of size k has to pick a node from each triangle (since it can't pick 2 from the same one).

The edges between nodes corresponding to λ and $\bar{\lambda}$ ensure that λ and $\bar{\lambda}$ aren't both assigned to true.

Thus α is a valid truth assignment satisfying ϕ . □

slide 10.27

NP-hardness of Independent Set

Proof  blackboard

Corollary 10.15. Independent Set, Clique and Vertex Cover are NP-complete.

(*Proof:* We showed earlier that Independent Set $\leq$ Clique and Independent Set $\leq$ Vertex Cover; with the previous Theorem 10.14 it follows that all three of them are NP-hard. But we already know that all three problems are in NP! Because they are in NP *and* NP-hard, they are NP-complete.)

- Why are **NP-complete** problems cool? In general, an NP-hard problem L that is not in NP might be *strictly harder than all problems in NP*. In that case, even if $P = NP$, your L will not be in P, and you would never be able to prove $P = NP$ by trying to find a polynomial-time algorithm for your problem L !

slide 10.28

NP-hardness of TSP (decision version)

Problem: Directed Hamiltonian Cycle **Decision**

Input: A digraph $G = (V, E)$.

Question: Is there a round tour in G (visiting every vertex exactly once)?

Theorem 10.16. 1. Vertex Cover $\leq$ Directed Hamiltonian Cycle
2. Directed Hamiltonian Cycle $\leq$ Traveling Salesman Decision Problem

- *Traveling Salesman Decision Problem* is NP-complete!

- If there would be a polynomial-time algorithm for TSP, then there would also be polynomial-time algorithms for *all* problems in NP. (incl. 3-SAT, Vertex Cover, Independent Set, Clique, and many many more ...).
- This provides strong evidence that TSP is *inherently hard*, and that there might not be a polynomial-time algorithm.

Algorithms lecture summary

slide 10.29

1. Prelude: runtime analysis, correctness proofs, difference between problems and algorithms
2. **Divide-and-Conquer Algorithms**
 - **Techniques:** divide-conquer-merge design strategy, solving recurrences
 - **Problems:** sorting, selection, integer mult., matrix mult., polynomial mult.
 - **Algorithms:** mergesort, quicksort, quickselect, mom-select, Karatsuba, Strassen, triangle counting, FFT, IFFT, polynomial multiplication using FFT
3. **Dynamic Programming**
 - **Techniques:** dynamic programming design strategy, memoisation
 - **Problems:** text segmentation, longest increasing subsequence, edit distance, optimal binary search trees, matrix chain multiplication, maximum independent set on trees, TSP
 - **Algorithms:** One algorithm for each problem.
4. **Greedy**
 - **Problems:** storing files on tape, event scheduling, minimum cost prefix code, MST, SSSP
 - **Algorithms:** files on tape, scheduling, Huffman, Jarnik/Prim, Kruskal, Dijkstra
5. **NP-completeness**
 - **Definitions:** decision problem, NP-hard, NP-complete, reduction, basic properties
 - **Decision problems:** TSdP, 3-SAT, IS, VC, Clique, Hamiltonian cycle
 - **Reduction algorithms:** several reductions between above decision problems

Picture credits

Titel: *algorists* vs. *abacists* from Gregor Reisch (1503 CE). The source of the digital copy is "Typ 520.03.736, Houghton Library, Harvard University". (CC PDM 1.0; public domain) 1

1 Edsger W. Dijkstra. Von: Hamilton Richards - manuscripts of Edsger W. Dijkstra, University Texas at Austin <https://commons.wikimedia.org/w/index.php?curid=4204157> (CC BY-SA 3.0). 64

Details on the mentioned Creative Commons (CC) licenses are available at <https://creativecommons.org/>.

References

[CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009. URL: <http://mitpress.mit.edu/books/introduction-algorithms> (cit. on p. 3).

[Eri19] J. Erickson. *Algorithms*. Online edition, 2019. URL: <http://algorithms.wtf> (cit. on p. 3).